Smarter Regular Expression Matching

From Theory to Tools

Ilian Kohl

October 13, 2026

Contents

1 Introduction **3**
- 1.1 Motivation . . . 3
- 1.2 Problem Statement . . . 4
- 1.3 Contributions . . . 4
- 1.4 Thesis Outline . . . 5

2 Regular Expressions **6**
- 2.1 Syntax . . . 6
- 2.2 Semantics . . . 7
- 2.3 Rust Representation . . . 8

3 Derivatives and Partial Derivatives **9**
- 3.1 Naive Matching . . . 9
- 3.2 Brzozowski Derivatives . . . 11
- 3.3 Antimirov Partial Derivatives . . . 17

4 Matching and Parsing **22**
- 4.1 Matching: A Boolean Answer . . . 22
- 4.2 Parsing: Parse Trees as Matching Evidence . . . 22

5 Derivative-Based Parser **25**
- 5.1 The Standard Derivative-Based Parser . . . 25
  - 5.1.1 Building the Empty-Word Tree . . . 26
  - 5.1.2 Reinserting Characters via Injection . . . 27
  - 5.1.3 Parser . . . 30
  - 5.1.4 Worked Example: Ambiguous Expression . . . 30
- 5.2 The Bit-Coded Derivative-Based Parser . . . 32
  - 5.2.1 Annotated Expressions . . . 33
  - 5.2.2 Fusing Bit-Lists . . . 34
  - 5.2.3 Internalizing Expressions . . . 35
  - 5.2.4 Extracting Bits . . . 36
  - 5.2.5 Annotated Derivatives . . . 37
  - 5.2.6 Decoding Bits . . . 39
  - 5.2.7 Parser . . . 42
  - 5.2.8 Worked Example: Ambigious Expression . . . 42

6 Partial-Derivative-Based Parser **44**
- 6.1 The Standard Partial-Derivative Parser . . . 44

6.1.1 Partial Derivatives with Injections . . . 46
6.1.2 State Sets and Selection . . . 48
6.1.3 Parser . . . 49
6.1.4 Worked Example: Ambiguous Expression . . . 49
6.2 The Bit-Coded Partial-Derivative Parser . . . 51
6.2.1 Bit-Coded Partial Derivatives . . . 51
6.2.2 Extracting Bits . . . 54
6.2.3 State Sets and Selection . . . 55
6.2.4 Parser . . . 55
6.2.5 Worked Example: Ambiguous Expression . . . 56

7 External To Internal Regex **58**
7.1 Two ASTs . . . 59
7.2 Parsing Surface Syntax . . . 60
7.3 Translating `ExtPat` into `Regex` . . . 61
7.4 Anchors and the Two Entry Points . . . 63

8 Evaluation **68**
8.1 Parser Agreement . . . 68
8.2 Dataset Preparation . . . 69
8.2.1 The Real-World Corpus . . . 69
8.3 Performance Analysis . . . 72
8.3.1 Performance on Synthetic Patterns . . . 72
8.3.2 Evaluation on the Mamouras Regex Dataset . . . 73
8.4 Discussion of Results . . . 73
8.4.1 Optimization Challenges and Insights . . . 73

9 Related Work and Conclusion **74**
9.1 Related Work . . . 74
9.2 Conclusion . . . 74

A Diagnostics **76**
A.1 `deriv_std_rec`, match . . . 76
A.2 `deriv_bc`, match . . . 78
A.3 `pderiv_std`, match . . . 79
A.4 `pderiv_bc`, match . . . 81
A.5 `deriv_std_rec`, no match . . . 82
A.6 `deriv_bc`, no match . . . 83
A.7 `pderiv_std`, no match . . . 84
A.8 `pderiv_bc`, no match . . . 86

1 Introduction

1.1 Motivation

Regular expressions are a fundamental formalism for describing regular languages and are among the most widely used abstractions in computer science. They underpin applications including lexical analysis, input validation, log processing, search tools, and network security. Across these applications, the problem is regular expression matching. Given a regular expression and an input string, determine whether the string belongs to the language denoted by the expression. Formally, this is the *membership problem*.

For example, the expression `(ab|a)*` matches `"aba"`, as it decomposes into `"ab"` followed by `"a"`, but does not match `"abb"`, since no sequence of subexpressions can produce it.

For many applications, a Boolean answer is enough. An input validator may only need to accept or reject a password or URL. Others require more. For instance, a lexer must determine how input decomposes into tokens. Capturing groups, schema validation, and information extraction require identifying which input parts correspond to parts of the expression. Recognition alone is not enough; the matcher must also recover the *structure*.

Efficiently solving these problems remains an active research area. The choice of algorithm influences performance, memory consumption, and security. Once matching becomes parsing, the way in which match structure is recovered also influences correctness and raises the question of how to handle ambiguity when multiple matches are possible.

Existing approaches include backtracking and finite-automata-based techniques, such as nondeterministic (NFA) and deterministic (DFA) automata. These approaches have different trade-offs in implementation complexity, memory consumption, and worst-case performance. Backtracking engines are flexible and widely used, but may exhibit exponential running time on constructed inputs, leading to regular expression denial-of-service (ReDoS) vulnerabilities [Cox, 2007]. Automata-based approaches provide stronger worst-case guarantees but may require preprocessing or suffer from state-space explosion.

Derivative-based algorithms provide an alternative by transforming the regular expression as input symbols are consumed. Each derivative represents the language remaining after an input symbol has been consumed. By maintaining a correspondence between an expression and its successive residuals, derivative-based methods provide an elegant foundation for regular expression matching and, with suitable extensions, structured parsing.

However, derivative-based techniques are less common in practical regular expression tooling. Their use for practical matching, parsing, and diagnostic tools motivates this thesis: "From Theory to Tools".

1.2 Problem Statement

Derivative-based approaches provide a principled foundation for regular expression matching and parsing. Brzozowski introduced regular expression derivatives as a way of transforming a regular expression after consuming an input symbol [Brzozowski, 1964]. Antimirov later introduced partial derivatives, which represent sets of residual expressions and are closely related to nondeterministic automata [Antimirov, 1996].

Building on Brzozowski's derivatives, Sulzmann and Lu developed a derivative-based algorithm for POSIX regular expression parsing [Sulzmann and Lu, 2014]. Their work also presents a bit-coded variant that represents parse information as a compact bit sequence, which can be decoded after a successful match rather than maintaining an explicit parse tree throughout the computation. A Haskell implementation of the POSIX parser is available in Sulzmann's *Programming Paradigms* teaching material [Sulzmann, 2026].

Sulzmann and Lu also studied partial derivatives in the context of regular expression sub-matching [Sulzmann and Lu, 2012]. While sub-matching is not the focus of this thesis, their work provides relevant background for the partial-derivative approach. The *Programming Paradigms* teaching material also presents a bit-coded partial-derivative-based parser in Haskell.

These constructions have primarily been presented in a theoretical and functional programming setting, leaving open practical questions about execution time, memory consumption, and data representation.

This thesis investigates these questions through implementations in Rust. As a systems programming language, Rust provides fine-grained control over memory allocation and data representation, allowing the derivative-based and partial-derivative-based parsers, including their bit-coded variants, to be evaluated in practice.

The implementation also provides traced versions of the parsing algorithms that record intermediate derivative computations during matching. These traces show the transformations performed by the algorithms and help explain successful and unsuccessful matches.

1.3 Contributions

The following contributions are being made:

- **Rust implementation of derivative-based parsers.** The thesis implements derivative-based and partial-derivative-based parsing constructions in Rust, including direct and bit-coded variants. The implementations are based on the constructions described in the literature and the corresponding Haskell material, resulting in four parser variants in total.
- **Empirical evaluation of the parser variants.** The four implementations are evaluated with respect to execution time and memory consumption using synthetic benchmarks and a corpus of real-world regular expressions from Snort/Suricata, SpamAssassin, and RegexLib, following the source categories investigated by Mamouras et al. [Mamouras et al., 2024]. They are also compared with Rust's `regex` crate and Google's RE2.

1.4 Thesis Outline

The remainder of this thesis follows the parsing constructions from their formal concepts through their Rust implementation to their evaluation. Theory and implementation are presented together.

- **Chapter 2 - Regular Expressions.** This chapter introduces the syntax and semantics of regular expressions and the Rust representation used throughout the implementation.
- **Chapter 3 - Derivatives.** This chapter introduces Brzozowski's derivatives and Antimirov's partial derivatives, together with their basic matching algorithms and Rust implementations.
- **Chapter 4 - Matching vs. Parsing.** This chapter distinguishes regular expression matching from parsing and introduces the disambiguation problem.
- **Chapter 5 - Derivative Parser.** This chapter presents Sulzmann and Lu's derivative-based POSIX parser and its Rust implementation in direct and bit-coded variants.
- **Chapter 6 - Partial-Derivative Parser.** This chapter presents the partial-derivative-based parser and its Rust implementation in direct and bit-coded variants. It also discusses the differences between the two derivative-based approaches.
- **Chapter 7 - External versus Internal Syntax.** This chapter describes the parsing of regular expression strings into an external AST and their translation into the internal representation used by the parsers.
- **Chapter 8 - Evaluation.** This chapter evaluates the four parser variants with respect to execution time, memory consumption, and correctness. The evaluation uses synthetic benchmarks and a corpus of real-world regular expressions, and compares the implementations with Rust's `regex` crate and Google's RE2. It also examines optimization challenges and the insights gained from the evaluation, including the derivative traces produced by the implementations.
- **Chapter 9 - Conclusion.** Finally, this chapter summarizes the findings of the thesis and discusses the practical implications of the results and directions for future work.

2 Regular Expressions

2.1 Syntax

Let Σ be an alphabet. Regular expressions over Σ are defined inductively by the grammar:

$$r, s ::= x \mid \varepsilon \mid \emptyset \mid r + s \mid r \cdot s \mid r^* \qquad (x \in \Sigma). \tag{2.1}$$

In the grammar, r and s range over regular expressions, while x ranges over the alphabet Σ and denotes an arbitrary symbol, such as a, b, or c. The constants ε and $\emptyset$ represent the *empty-word* and *empty-language* expressions. The operators $+$, $\cdot$, and * denote alternation, concatenation, and Kleene star, respectively.

Concatenation is written implicitly, so rs denotes $r \cdot s$. Given this notation, we adopt the precedence and associativity conventions. Unless parentheses are shown explicitly, Kleene star binds most tightly, followed by concatenation, followed by alternation. For example,

$$ab^*c = (a \cdot (b^*)) \cdot c,$$
$$a + bc^* = a + (b \cdot (c^*)).$$

For repeated operators we use left association, so

$$a + b + c = (a + b) + c,$$
$$abc = (ab)c.$$

The grammar above is the core syntax. Most regular expression libraries also offer one-or-more, optional, and bounded repetition (r^+, $r?$, $r^{\{m,n\}}$), but these add no expressive power since each can be defined by the core constructs: $r^+ = r \cdot r^*$, $r? = r + \varepsilon$, and

$$r^{\{m,n\}} = \underbrace{r \cdot \ldots \cdot r}_{m \text{ times}} \cdot \underbrace{(r + \varepsilon) \cdot \ldots \cdot (r + \varepsilon)}_{n-m \text{ times}}.$$

In the implementation, expressions using such operators are translated into the core constructs before they reach the parsing algorithms.

As a running example, consider the regular expression over $\Sigma = \{0, 1\}$

$$r_{01} = (0 + 1)^* \cdot 0 \cdot 1. \tag{2.2}$$

By the conventions above, its syntax tree is $((0 + 1)^* \cdot 0) \cdot 1$. This tree represents the hierarchical structure of the expression: a Kleene star applied to the alternation $0 + 1$, followed by concatenation with the literals 0 and 1.

The syntactic structure must be given a semantic interpretation.

2.2 Semantics

A semantic interpretation assigns a language to each regular expression. A *word* over Σ is a finite sequence of symbols from Σ. The set of all words over Σ, including the empty word ε, is denoted by Σ^*. A *language* over Σ is a subset of Σ^*.

For a regular expression r, we write $L(r)$ for the language it denotes, where $L(r) \subseteq \Sigma^*$. The definition follows the structure of the syntax tree by *structural recursion*. Each case corresponds to one production of the grammar in (2.1):

$$\begin{aligned} L(x) &= \{x\}, \\ L(\varepsilon) &= \{\varepsilon\}, \\ L(\emptyset) &= \emptyset, \\ L(r + s) &= L(r) \cup L(s), \\ L(r \cdot s) &= \{vw \mid v \in L(r),\ w \in L(s)\}, \\ L(r^*) &= \{\varepsilon\} \cup \{w_1 \cdots w_n \mid n \geq 1,\ w_1, \ldots, w_n \in L(r)\}. \end{aligned} \tag{2.3}$$

Table 2.1 applies these definitions to small expressions. The literal a denotes the language containing only the word a. The expression ε denotes the language containing only the empty word, while $\emptyset$ denotes the language containing no words. The alternation $a + b$ combines the languages of a and b to give $\{a, b\}$, and concatenation ab combines words from the languages of a and b to give $\{ab\}$. The Kleene star a^* allows zero or more concatenations of words from the language of a, giving $\{\varepsilon, a, aa, aaa, \ldots\}$.

Expression r	Language $L(r)$
a	$\{a\}$
ε	$\{\varepsilon\}$
$\emptyset$	$\emptyset$
$a + b$	$\{a, b\}$
ab	$\{ab\}$
a^*	$\{\varepsilon, a, aa, aaa, \ldots\}$

Table 2.1: Languages denoted by basic expressions.

For example $r_{01} = (0 + 1)^* \cdot 0 \cdot 1$ from (2.2), we compute its language step by step:

$$\begin{aligned} L(0) &= \{0\}, \\ L(1) &= \{1\}, \\ L(0 + 1) &= L(0) \cup L(1) = \{0, 1\}, \\ L((0 + 1)^*) &= \{0, 1\}^*, \\ L(r_{01}) &= L((0 + 1)^*) \cdot L(0) \cdot L(1) = \{\, w \cdot 0 \cdot 1 \mid w \in \{0, 1\}^* \,\}. \end{aligned} \tag{2.4}$$

This is the set of binary words ending in 01. For example, the words 01, 001, and 1101 belong to $L(r_{01})$, whereas ε, 0, and 010 do not.

This is the question of whether a word *matches* a regular expression: w *matches* r if $w \in L(r)$. Determining this is the *membership problem*.

2.3 Rust Representation

The grammar from (2.1) is represented directly as a Rust enum with one variant for each syntactic construct, as shown in Figure 2.1:

```
#[derive(Debug, Clone, PartialEq, Eq, Hash)]
pub enum Regex {
    Phi, Eps, Lit(char),
    Seq(Box<Regex>, Box<Regex>),  // r . s
    Alt(Box<Regex>, Box<Regex>),  // r + s
    Star(Box<Regex>),              // r*
}

impl Regex {
    pub fn seq(r: Regex, s: Regex) -> Regex {
        Regex::Seq(Box::new(r), Box::new(s))
    }
    pub fn alt(r: Regex, s: Regex) -> Regex {
        Regex::Alt(Box::new(r), Box::new(s))
    }
    pub fn star(r: Regex) -> Regex { Regex::Star(Box::new(r)) }
    pub fn lit(c: char) -> Regex { Regex::Lit(c) }
}
```

Figure 2.1: The `Regex` enum and constructors. Derived traits support operations needed by parsing: `Clone` for duplication, `PartialEq`/`Eq` for comparison, `Hash` for collections.

The type carries no precedence or associativity information, as these are determined by the syntax tree. Rust requires explicit indirection because enum sizes must be known at compile time. Recursive variants therefore use `Box<Regex>`. A direct definition such as `Seq(Regex, Regex)` would have infinite size, whereas `Box` stores recursive values separately, providing finite size [The Rust Project Developers, 2026a,b]. This differs from Haskell's heap-allocated data types.

The running example $r_{01} = (0 + 1)^* \cdot 0 \cdot 1$ from (2.2) is represented by the syntax tree:

$$\mathrm{Seq}\Big(\mathrm{Seq}(\mathrm{Star}(\mathrm{Alt}(0, 1)), 0), 1\Big). \tag{2.5}$$

In Rust, this is represented as shown in Figure 2.2, with the outermost `Seq` containing `Lit('1')` and `Seq(Star(Alt(0,1)), 0)` as its right and left children.

```
Regex::seq(
    Regex::seq(
        Regex::star(Regex::alt(Regex::lit('0'), Regex::lit('1'))),
        Regex::lit('0')),
    Regex::lit('1'))
```

Figure 2.2: Rust representation of r_{01}.

3 Derivatives and Partial Derivatives

Chapter 2 introduced the syntax and semantics of regular expressions, and defined the *membership problem*: Decide whether a word w belongs to $L(r)$. This chapter introduces three algorithms for solving it, each returning a Boolean answer: match or no match.

The first is a naive recursive matcher that follows the structure of the regular-expression semantics. The other two transform the regular expression as input symbols are consumed. Brzozowski derivatives transform it into a single residual expression, while Antimirov's partial derivatives represent the possible residual expressions as a set.

3.1 Naive Matching

A direct way to decide whether $w \in L(r)$ is to interpret the regular-expression semantics as recursive matching rules, by following the structure of the expression and searching for possible decompositions of the input. Let $w[i:j]$ denote the substring from position i to j. A recursive procedure $\mathrm{match}(w, i, j, r)$ determines whether $w[i:j] \in L(r)$:

$$\begin{aligned}
\mathrm{match}(w,i,j,\varepsilon) &\iff i = j,\\
\mathrm{match}(w,i,j,x) &\iff j = i+1 \wedge w[i] = x,\\
\mathrm{match}(w,i,j,r+s) &\iff \mathrm{match}(w,i,j,r) \vee \mathrm{match}(w,i,j,s),\\
\mathrm{match}(w,i,j,r\cdot s) &\iff \exists k \in [i,j].\ \mathrm{match}(w,i,k,r) \wedge \mathrm{match}(w,k,j,s),\\
\mathrm{match}(w,i,j,r^*) &\iff i = j \vee \exists k \in (i,j].\ \mathrm{match}(w,i,k,r) \wedge \mathrm{match}(w,k,j,r^*).
\end{aligned} \tag{3.1}$$

Each rule corresponds directly to the semantics. Concatenation does not specify where to split, so the matcher must consider all possible split points k.

For matching $(a \cdot a) \cdot a$ against `aaa`, the split points $k \in \{0,1,2,3\}$ are:

$$\begin{aligned}
&\underbrace{\mathrm{match}(w,0,0,a\cdot a)}_{a\cdot a \text{ against } w[0:0]=\epsilon} \wedge \underbrace{\mathrm{match}(w,0,3,a)}_{a \text{ against } w[0:3]=\texttt{aaa}}\\
\vee\ &\underbrace{\mathrm{match}(w,0,1,a\cdot a)}_{a\cdot a \text{ against } w[0:1]=\texttt{a}} \wedge \underbrace{\mathrm{match}(w,1,3,a)}_{a \text{ against } w[1:3]=\texttt{aa}}\\
\vee\ &\underbrace{\mathrm{match}(w,0,2,a\cdot a)}_{a\cdot a \text{ against } w[0:2]=\texttt{aa}} \wedge \underbrace{\mathrm{match}(w,2,3,a)}_{a \text{ against } w[2:3]=\texttt{a}}\\
\vee\ &\underbrace{\mathrm{match}(w,0,3,a\cdot a)}_{a\cdot a \text{ against } w[0:3]=\texttt{aaa}} \wedge \underbrace{\mathrm{match}(w,3,3,a)}_{a \text{ against } w[3:3]=\epsilon}.
\end{aligned} \tag{3.2}$$

At $k = 2$, both operands match, triggering a recursive search on $(a \cdot a)$ against `aa` with split points $k \in \{0, 1, 2\}$, where $k = 1$ succeeds:

$$\begin{aligned} & \underbrace{\text{match}(w, 0, 0, a)}_{a \text{ against } w[0:0]=\epsilon} \wedge \underbrace{\text{match}(w, 0, 2, a)}_{a \text{ against } w[0:2]=\texttt{aa}} \\ \vee\ & \underbrace{\text{match}(w, 0, 1, a)}_{a \text{ against } w[0:1]=\texttt{a}} \wedge \underbrace{\text{match}(w, 1, 2, a)}_{a \text{ against } w[1:2]=\texttt{a}} \\ \vee\ & \underbrace{\text{match}(w, 0, 2, a)}_{a \text{ against } w[0:2]=\texttt{aa}} \wedge \underbrace{\text{match}(w, 2, 2, a)}_{a \text{ against } w[2:2]=\epsilon}. \end{aligned} \tag{3.3}$$

This completes the original match through the nested split points $k = 2$ and $k = 1$.

In Rust. The recursive rules translate directly into the corresponding Rust match arms, shown in Figure 3.1. Alternation tries both branches on the same input, while concatenation and star search possible split points. The star arm clones the body to construct the recursive `Star` expression. This is necessary because the recursive call needs to own the newly constructed expression. The range-based search short-circuits at the first successful split. Nested concatenations repeatedly search smaller substrings, exploring multiple paths before succeeding. This leads to exponential backtracking in the worst case.

```
Regex::Alt(r, s) => {
    match_naive_range(word, i, j, r)
        || match_naive_range(word, i, j, s)
}
Regex::Seq(r, s) => {
    (i..=j).any(|k| {
        match_naive_range(word, i, k, r)
            && match_naive_range(word, k, j, s)
    })
}
Regex::Star(r) => {
    i == j || (i + 1..=j).any(|k| {
        match_naive_range(word, i, k, r)
            && match_naive_range(word, k, j,
                &Regex::star(*r.clone()))
    })
}
```

Figure 3.1: Alternation, concatenation, and star arms of the naive matcher.

Although inefficient, this matcher is useful as a reference implementation because it follows the regular-expression semantics directly without any intermediate transformation. The cost is visible in the worked examples: nested concatenations repeatedly search smaller substrings, each exploring all possible split points before finding a match.

The derivative approach takes a different approach. Rather than searching for a decomposition of w while keeping r fixed, it transforms r as each character of w is consumed. The resulting expression describes the language that can still match the remaining suffix.

3.2 Brzozowski Derivatives

Brzozowski's derivative of r with respect to a symbol x, written $r\backslash x$, is the expression describing exactly what remains of r's language after consuming the leading symbol x of a word $w \in L(r)$ [Brzozowski, 1964]:

$$L(r\backslash x) \;=\; \{\, w \mid xw \in L(r) \,\}. \tag{3.4}$$

For $w = x_1 x_2 \cdots x_n$, consume one symbol at a time by repeatedly taking derivatives:

$$r, \quad r\backslash x_1, \quad (r\backslash x_1)\backslash x_2, \quad \ldots$$

Nullability. After all symbols have been consumed, the remaining input is ε. Thus, the resulting expression must accept ε for the original word to belong to $L(r)$. This is the question of whether the expression is *nullable*. Let $n(r)$ be true exactly when $\varepsilon \in L(r)$:

$$\begin{array}{llll} n(\emptyset) = \text{false} & (\mathsf{N}_\emptyset) & n(r+s) = n(r) \vee n(s) & (\mathsf{N}_+) \\ n(\varepsilon) = \text{true} & (\mathsf{N}_\varepsilon) & n(r \cdot s) = n(r) \wedge n(s) & (\mathsf{N}_{\text{seq}}) \\ n(x) = \text{false} & (\mathsf{N}_{\text{lit}}) & n(r^*) = \text{true} & (\mathsf{N}_*) \end{array} \tag{3.5}$$

The empty language $\emptyset$ is not nullable because it contains no words, including ε, while a literal cannot match the empty word. ε and r^* always match ε. An alternation is nullable when either branch is nullable, and a concatenation only when both components are.

In Rust. The definition is transcribed one `Regex` variant at a time, using the enum of Figure 2.1. The case split of (3.5) becomes the arms of a `match`, shown in Figure 3.2.

```
pub fn nullable(r: &Regex) -> bool {
    match r {
        Regex::Phi => false,
        Regex::Eps => true,
        Regex::Lit(_) => false,
        Regex::Alt(r, s) => nullable(r) || nullable(s),
        Regex::Seq(r, s) => nullable(r) && nullable(s),
        Regex::Star(_) => true,
    }
}
```

Figure 3.2: `nullable` implementation and its six cases: $\mathsf{N}_\emptyset$, N_ε, N_{lit}, N_+, N_{seq} and N_*.

The expression is passed by reference because `nullable` only inspects r and does not modify or consume it. The underscores in N_{lit} and N_* indicate that the corresponding operands are irrelevant. A literal is never nullable, regardless of its symbol, while a Kleene-star expression is always nullable, regardless of its body. The short-circuiting operators `||` and `&&` avoid evaluating the second operand when the result is already determined, while preserving the corresponding logical semantics.

Derivative Rules. The rules specify the expression that remains after consuming x:

$$\begin{array}{llll} \emptyset\backslash x = \emptyset & (\mathsf{D}_\emptyset) & (r+s)\backslash x = (r\backslash x) + (s\backslash x) & (\mathsf{D}_+) \\ \varepsilon\backslash x = \emptyset & (\mathsf{D}_\varepsilon) & r^*\backslash x = (r\backslash x)\cdot r^* & (\mathsf{D}_*) \\ y\backslash x = \begin{cases} \varepsilon & x = y \\ \emptyset & x \neq y \end{cases} & (\mathsf{D}_{\text{lit}}) & (r\cdot s)\backslash x = \begin{cases} (r\backslash x)\cdot s + s\backslash x & n(r) = \text{true} \\ (r\backslash x)\cdot s & n(r) = \text{false} \end{cases} & \begin{array}{l}(\mathsf{D}^n_{\text{seq}}) \\ (\mathsf{D}_{\text{seq}})\end{array} \end{array} \tag{3.6}$$

The empty language and ε cannot consume x. A literal y succeeds exactly when $x = y$, leaving ε. Alternation retains both derivatives and for r^*, one occurrence of r consumes x while further occurrences remain available.

Concatenation is the only rule that depends on nullability. If r is not nullable, r must consume the leading x, leaving $r\backslash x$ as the remainder of r while s remains untouched. If r is nullable, it may either consume x giving $(r\backslash x)\cdot s$, or r matches ε and s consumes x leaving $s\backslash x$. Hence the nullable case produces the alternation $(r\backslash x)\cdot s + s\backslash x$.

In Rust. Figure 3.3 implements (3.6) similarly, one variant at a time.

```
pub fn deriv(r: &Regex, x: char) -> Regex {
    match r {
        Regex::Phi => Regex::Phi,
        Regex::Eps => Regex::Phi,
        Regex::Lit(c) => if *c == x { Regex::Eps } else { Regex::Phi },
        Regex::Alt(r1, r2) =>
            Regex::alt(deriv(r1, x), deriv(r2, x)),
        Regex::Seq(r1, r2) => {
            let dr1 = Regex::seq(deriv(r1, x), *r2.clone());
            if nullable(r1) {
                Regex::alt(dr1, deriv(r2, x))
            } else { dr1 }
        }
        Regex::Star(r1) =>
            Regex::seq(deriv(r1, x), Regex::star(*r1.clone())),
    }
}
```

Figure 3.3: `deriv` implementation with six cases.

`deriv` differs from `nullable` in that it borrows its input but returns an owned `Regex`, since a derivative is a new expression. The `if nullable(r1)` implements the case split of (3.6), with branches in order $\mathsf{D}^n_{\text{seq}}$ then D_{seq}. This is the only place where a derivative rule consults nullability and where the output can be larger than the input, since the nullable branch produces an alternation while the other produces a concatenation.

The cloning in `*r2.clone()` is necessary because s appears untouched in both branches of the concatenation case, and the star arm must preserve r^*'s body both derived and underived. This copying mechanism drives expression growth, which justifies simplification after every derivative step.

Example: Growth without simplification. Consider $(x+y)^*$ against the word $\mathtt{xx}$: The expression is a star, so D_* applies with body $r = x + y$:

$$\begin{aligned} (x+y)^*\backslash\mathtt{x} &= ((x+y)\backslash\mathtt{x}) \cdot (x+y)^* && \mathsf{D}_*,\ r = x+y \\ &= (\varepsilon + \emptyset) \cdot (x+y)^* && \mathsf{D}_+,\ \mathsf{D}_{\text{lit}} \end{aligned} \tag{3.7}$$

The expression is now the concatenation $(\varepsilon + \emptyset) \cdot (x+y)^*$, and deriving it consumes the second $\mathtt{x}$. The result is an alternation because $\mathsf{D}^n_{\text{seq}}$ applies with $r = \varepsilon + \emptyset$ and $s = (x+y)^*$, its nullable case being selected since $n(r) = n(\varepsilon + \emptyset) = \text{true}$ by N_+ and N_ε:

$$\begin{aligned} &((\varepsilon + \emptyset) \cdot (x+y)^*)\backslash\mathtt{x} \\ &= ((\varepsilon + \emptyset)\backslash\mathtt{x}) \cdot (x+y)^* + (x+y)^*\backslash\mathtt{x} && \mathsf{D}^n_{\text{seq}} \\ &= (\emptyset + \emptyset) \cdot (x+y)^* + (x+y)^*\backslash\mathtt{x} && \mathsf{D}_\varepsilon \\ &= (\emptyset + \emptyset) \cdot (x+y)^* + (\varepsilon + \emptyset) \cdot (x+y)^* && \mathsf{D}_*,\ \mathsf{D}_+,\ \mathsf{D}_{\text{lit}} \end{aligned} \tag{3.8}$$

After both symbols are consumed, the result is checked for nullability:

$$\begin{aligned} &n((\emptyset + \emptyset) \cdot (x+y)^* + (\varepsilon + \emptyset) \cdot (x+y)^*) \\ &= n((\emptyset + \emptyset) \cdot (x+y)^*) \vee n((\varepsilon + \emptyset) \cdot (x+y)^*) && \mathsf{N}_+ \\ &= (n(\emptyset + \emptyset) \wedge n((x+y)^*)) \vee (n(\varepsilon + \emptyset) \wedge n((x+y)^*)) && \mathsf{N}_{\text{seq}} \\ &= (\text{false} \wedge \text{true}) \vee (\text{true} \wedge \text{true}) && \mathsf{N}_+,\ \mathsf{N}_\emptyset,\ \mathsf{N}_\varepsilon,\ \mathsf{N}_* \\ &= \text{true}. \end{aligned} \tag{3.9}$$

Thus $(x+y)^*$ matches $\mathtt{xx}$.

Simplification. Derivatives can grow unboundedly. Repeatedly deriving $(x+y)^*$ by the same symbol produces expressions of increasing size. This growth results from the nullable branch of $\mathsf{D}^n_{\text{seq}}$, which introduces an alternation as a redundant structure that is carried into subsequent derivatives. ε and $\emptyset$ accumulate without bound:

$$\begin{aligned} & (x+y)^* && 4 \\ \xrightarrow{\mathtt{x}}\ & (\varepsilon + \emptyset) \cdot (x+y)^* && 8 \\ \xrightarrow{\mathtt{x}}\ & (\emptyset + \emptyset) \cdot (x+y)^* \ +\ (\varepsilon + \emptyset) \cdot (x+y)^* && 17 \\ \xrightarrow{\mathtt{x}}\ & (\emptyset + \emptyset) \cdot (x+y)^* \ +\ \left((\emptyset + \emptyset) \cdot (x+y)^* \ +\ (\varepsilon + \emptyset) \cdot (x+y)^*\right) && 26 \end{aligned} \tag{3.10}$$

To counter the growth, expressions are simplified after each derivative step, using identities that preserve the denoted language:

$$\begin{aligned} \emptyset \cdot r &= \emptyset && (\mathsf{S}_1) \\ \varepsilon \cdot r &= r && (\mathsf{S}_2) \\ \emptyset + r &= r && (\mathsf{S}_3) \end{aligned} \tag{3.11}$$

S_1 eliminates dead concatenations with $\emptyset$, S_2 eliminates identity concatenations with ε, and S_3 eliminates $\emptyset$ from alternations. Each holds equally with the unit on the right.

Applying them to (3.8), after each step, returns the expression back to $(x+y)^*$, keeping the size bounded regardless of input length:

$$\begin{aligned}
&(\emptyset + \emptyset) \cdot (x+y)^* + (\varepsilon + \emptyset) \cdot (x+y)^* \\
&= \emptyset \cdot (x+y)^* + \varepsilon \cdot (x+y)^* && \mathsf{S}_3,\ \text{twice} \\
&= \emptyset + (x+y)^* && \mathsf{S}_1,\ \mathsf{S}_2 \\
&= (x+y)^* && \mathsf{S}_3
\end{aligned} \tag{3.12}$$

In Rust. The identities are implemented recursively, applying the rules bottom-up over the expression tree. Each arm of the inner match implements an identity from (3.11).

`simplify` takes ownership of its input and returns simplified subexpressions directly, unlike `deriv`, which borrows and constructs. This model is visible where the two compose:

```
current = simplify(deriv(&current, c));
```

`deriv` borrows `current`, constructs a new expression, and `simplify` takes ownership of that result to return it without additional copying. The recursive calls ensure that all dead code is eliminated before an expression is returned, keeping size bounded at each step. Together, derive-then-simplify forms the core loop of the matcher, transforming and cleaning the expression at every input symbol.

```
pub fn simplify(r: Regex) -> Regex {
    match r {
        Regex::Seq(r, s) => {
            let r = simplify(*r);
            let s = simplify(*s);
            match (&r, &s) {
                (Regex::Phi, _) | (_, Regex::Phi) => Regex::Phi,
                (Regex::Eps, _) => s,
                (_, Regex::Eps) => r,
                _ => Regex::seq(r, s),
            }
        }
        Regex::Alt(r, s) => {
            let r = simplify(*r);
            let s = simplify(*s);
            match (&r, &s) {
                (Regex::Phi, _) => s,
                (_, Regex::Phi) => r,
                _ if r == s => r,
                _ => Regex::alt(r, s),
            }
        }
        Regex::Star(r) => match simplify(*r) {
            Regex::Eps | Regex::Phi => Regex::Eps,
            r => Regex::star(r),
        },
        other => other,
    }
}
```

Figure 3.4: Simplification rules: S_1 and S_2 in concatenation, S_3 in both branches.

Running example: $r_{01} = (0+1)^* \cdot 0 \cdot 1$. Recall from (2.2) that it is left-associated as $\Big((0+1)^* \cdot 0\Big) \cdot 1$, so the outer operands are $r = (0+1)^* \cdot 0$ and $s = 1$. We derive against $w = \mathtt{101}$ one symbol at a time.

1. **Derive by $\mathtt{1}$.** At the outer node, $r = (0+1)^* \cdot 0$ and $s = 1$. This r is not nullable:

$$n\Big((0+1)^* \cdot 0\Big) = \text{true} \wedge \text{false} = \text{false},$$

by N_{seq}, N_* and N_{lit}. Thus D_{seq} applies without a second summand:

$$r_{01}\backslash\mathtt{1} = \Big(((0+1)^* \cdot 0)\backslash\mathtt{1}\Big) \cdot 1.$$

At the middle node, the left operand $(0+1)^*$ is nullable, so $\mathsf{D}^n_{\text{seq}}$ applies. First,

$$\begin{aligned}
(0+1)^*\backslash\mathtt{1} &= \Big((0+1)\backslash\mathtt{1}\Big) \cdot (0+1)^* && \mathsf{D}_* \\
&= \Big((0\backslash\mathtt{1}) + (1\backslash\mathtt{1})\Big) \cdot (0+1)^* && \mathsf{D}_+ \\
&= (\emptyset + \varepsilon) \cdot (0+1)^* && \mathsf{D}_{\text{lit}} \\
&= (0+1)^* && \mathsf{S}_3,\ \mathsf{S}_2.
\end{aligned}$$

Hence

$$\begin{aligned}
((0+1)^* \cdot 0)\backslash\mathtt{1} &= (0+1)^* \cdot 0 + (0\backslash\mathtt{1}) && \mathsf{D}^n_{\text{seq}} \\
&= (0+1)^* \cdot 0 + \emptyset && \mathsf{D}_{\text{lit}} \\
&= (0+1)^* \cdot 0 && \mathsf{S}_3.
\end{aligned}$$

Substituting back into the outer node gives

$$r_{01}\backslash\mathtt{1} = \Big((0+1)^* \cdot 0\Big) \cdot 1 = r_{01}. \tag{3.13}$$

Thus a $\mathtt{1}$ not yet part of the final $\mathtt{01}$ is absorbed by the leading star.

2. **Derive by $\mathtt{0}$.** The expression is unchanged. Again, $(0+1)^*\backslash\mathtt{0}$ simplifies to $(0+1)^*$, but now the second summand at the middle node survives:

$$\begin{aligned}
((0+1)^* \cdot 0)\backslash\mathtt{0} &= (0+1)^* \cdot 0 + (0\backslash\mathtt{0}) && \mathsf{D}^n_{\text{seq}} \\
&= (0+1)^* \cdot 0 + \varepsilon && \mathsf{D}_{\text{lit}}.
\end{aligned}$$

No simplification applies. The outer node is still non-nullable, so

$$r_{01}\backslash\mathtt{1}\backslash\mathtt{0} = \Big((0+1)^* \cdot 0 + \varepsilon\Big) \cdot 1. \tag{3.14}$$

After $\mathtt{10}$, either the trailing $\mathtt{01}$ is still owed in full, represented by $(0+1)^* \cdot 0$, or the consumed $\mathtt{0}$ was already its first symbol, represented by ε.

3. **Derive by $\mathtt{1}$.** The outer binding has now changed. The left operand is $(0+1)^* \cdot 0 + \varepsilon$, which is nullable by N_+ and N_ε. So $\mathsf{D}^n_{\text{seq}}$ applies and contributes the second summand $1\backslash\mathtt{1} = \varepsilon$. For the first summand, D_+ applies, reusing the result from step 1:

$$\begin{aligned}
((0+1)^* \cdot 0 + \varepsilon)\backslash\mathtt{1} &= ((0+1)^* \cdot 0)\backslash\mathtt{1} + (\varepsilon\backslash\mathtt{1}) && \mathsf{D}_+ \\
&= (0+1)^* \cdot 0 + \emptyset && \text{step 1; } \mathsf{D}_\varepsilon \\
&= (0+1)^* \cdot 0 && \mathsf{S}_3.
\end{aligned}$$

Therefore

$$r_{01}\backslash\mathtt{1}\backslash\mathtt{0}\backslash\mathtt{1} = \Big((0+1)^* \cdot 0\Big) \cdot 1 + \varepsilon = r_{01} + \varepsilon. \tag{3.15}$$

The final expression is nullable by N_+ and N_ε, so $\mathtt{101} \in L(r_{01})$. The ε summand witnesses that the match is complete. For comparison, $w = \mathtt{10}$ leaves $\Big((0+1)^* \cdot 0 + \varepsilon\Big) \cdot 1$. Neither branch is nullable: $(0 + 1)^* \cdot 0$ still requires a 0, while $\varepsilon \cdot 1$ still requires a 1. Hence $\mathtt{10} \notin L(r_{01})$.

The matcher. The implementation can be summarized by the sequence of residual expressions produced while consuming the input. For $w = x_1 x_2 \cdots x_n$, the initial residual is $r_0 = r$, and each subsequent residual is obtained by deriving with respect to the next symbol and simplifying the result:

$$r_0 = r, \qquad r_i = \mathrm{simplify}(r_{i-1} \backslash x_i) \quad (1 \le i \le n). \tag{3.16}$$

The residual r_i denotes exactly the set of suffixes that complete the consumed prefix $x_1 \cdots x_i$ to a word in $L(r)$. Since simplification preserves the represented language, each r_i is equivalent to the unsimplified derivative after consuming that prefix. After the final symbol has been consumed, the matcher accepts when the remaining residual is nullable:

$$w \in L(r) \iff n(r_n) = \mathrm{true}.$$

So $r_0 \xrightarrow{x_1} r_1 \xrightarrow{x_2} \cdots \xrightarrow{x_n} r_n$ can be viewed as a path through a lazily constructed DFA. Each residual expression represents a state, and $\xrightarrow{x}$ denotes the transition obtained by taking the derivative with respect to x. Unlike an explicitly constructed DFA, only the states reached by the input are constructed. A residual is an accepting state exactly when it is nullable, so acceptance is determined by whether the final residual is nullable.

In Rust. Figure 3.5 implements (3.16) by composing `deriv`, `simplify`, and `nullable`. The implementation follows the mathematical definition directly. The current residual is transformed at each step, and acceptance is determined after the complete input has been consumed.

```
pub fn match_deriv(input: &str, r: &Regex) -> bool {
    let mut current = r.clone();
    for c in input.chars() {
        current = simplify(deriv(&current, c));
    }
    nullable(&current)
}
```

Figure 3.5: `match_deriv`: the core matcher loop. For each input character, derive the current residual expression and simplify it in place. Accept if the final residual is nullable.

The loop implements the residual sequence. At each iteration, `current` contains the current residual, which is then derived and simplified before processing the next input symbol. Only the current residual is retained, since the previous residual is dropped when `current` is reassigned. In contrast to parsing, where intermediate results may be required to reconstruct a parse tree, Boolean matching requires only the final residual.

The derivative-based matcher avoids the exponential backtracking of naive matching by transforming the expression rather than searching for decompositions. Simplification keeps the resulting expressions compact, preventing the growth shown in (3.10).

3.3 Antimirov Partial Derivatives

The derivative of Section 3.2 combines alternative residuals into a single expression using $+$. Antimirov's partial derivatives instead keep them as a *set* of expressions, one per strand of nondeterminism [Antimirov, 1996]. Writing $pd(r, x)$ for this set:

$$
\begin{aligned}
pd(\emptyset, x) &= \{\} && (\mathsf{P}_\emptyset)\\
pd(\varepsilon, x) &= \{\} && (\mathsf{P}_\varepsilon)\\
pd(y, x) &= \begin{cases} \{\varepsilon\} & x = y \\ \{\} & x \neq y \end{cases} && (\mathsf{P}_{\text{lit}})\\
pd(r + s, x) &= pd(r, x) \cup pd(s, x) && (\mathsf{P}_+)\\
pd(r^*, x) &= \{\, smartC(r', r^*) \mid r' \in pd(r, x) \,\} && (\mathsf{P}_*)\\
pd(r \cdot s, x) &= \begin{cases} \{\, smartC(r', s) \mid r' \in pd(r, x) \,\} \cup pd(s, x) & n(r) = \text{true} \\ \{\, smartC(r', s) \mid r' \in pd(r, x) \,\} & n(r) = \text{false} \end{cases} && \begin{matrix}(\mathsf{P}^n_{\text{seq}})\\ (\mathsf{P}_{\text{seq}})\end{matrix}
\end{aligned}
\tag{3.17}
$$

The empty language and ε produce the empty set, since neither has a residual with respect to x: where $\mathsf{D}_\emptyset$ and D_ε returned $\emptyset$ as a node in the result, $\mathsf{P}_\emptyset$ and P_ε return no residual at all. A literal y gives $\{\varepsilon\}$ when $x = y$ and $\{\}$ otherwise. Alternation unions the two residual sets, and for r^* one iteration of r consumes x, with each resulting residual placed in front of r^* so the star may continue. None of these alternatives are combined with $+$; each stays a separate element of the set.

Concatenation depends on nullability for the same reason as D_{seq}. If r is not nullable it must consume x, and each of its residuals is concatenated with the untouched s. If r is nullable, r may instead match ε and let s consume x, in which case $pd(s, x)$ joins the set. Where $\mathsf{D}^n_{\text{seq}}$ combined the two cases with $+$ inside one expression, $\mathsf{P}^n_{\text{seq}}$ keeps them as two elements of one set.

In Rust. Figure 3.6 presents the implementation of (3.17), with each `match` arm corresponding to one of its cases. The `Phi`, `Eps`, and `Lit` variants implement $\mathsf{P}_\emptyset$, P_ε, and P_{lit} directly, returning `Vec::new()` where the equation gives $\{\}$. `Alt` concatenates the two child vectors, implementing the union of P_+. `Star` maps `smart_seq` over r's own residuals, matching P_*; `Regex::star(*r.clone())` clones because `Regex::star` needs an owned `Regex` and `r` is only borrowed, once per residual since the closure runs once for each.

`Seq` implements the case split directly. The `map` applies `smart_seq` to each residual of the left operand, matching P_{seq}, and the guarded `out.extend(pderiv(s, x))` adds $pd(s, x)$ for $\mathsf{P}^n_{\text{seq}}$. Although the mathematical definition uses sets, the implementation uses `Vec<Regex>` throughout the recursion and defers deduplication to a single `HashSet` insertion at the top, rather than deduplicating at every `Alt` or `Seq` node on the way down.

Every union in `pderiv` is therefore a vector `extend`, not a set operation. Duplicate residuals reached by different strands are only merged once inserted into the `HashSet` used by the matcher below to keep the residual generation separate from deduplication.

```
fn pderiv(r: &Regex, x: char) -> Vec<Regex> {
    match r {
        Regex::Phi => Vec::new(),
        Regex::Eps => Vec::new(),
        Regex::Lit(c) => {
            if *c == x { vec![Regex::Eps] } else { Vec::new() }
        }
        Regex::Alt(r, s) => {
            let mut out = pderiv(r, x);
            out.extend(pderiv(s, x));
            out
        }
        Regex::Seq(r, s) => {
            let mut out: Vec<Regex> = pderiv(r, x)
                .into_iter()
                .map(|r_prime| smart_seq(r_prime, s))
                .collect();
            if nullable(r) {
                out.extend(pderiv(s, x));
            }
            out
        }
        Regex::Star(r) => {
            pderiv(r, x)
                .into_iter()
                .map(|r_prime| smart_seq(r_prime, &Regex::star(*r.clone())))
                .collect()
        }
    }
}
```

Figure 3.6: $pd(r, x)$ in full, corresponding arm for arm to (3.17).

The smart constructor. The concatenation in P_{seq} is constructed using the auxiliary constructor *smartC*, denoted by SC:

$$smartC(r', s) = \begin{cases} s & r' = \varepsilon \\ r' \cdot s & \text{otherwise} \end{cases} \tag{3.18}$$

If $r' = \varepsilon$, the consumed symbol completed the left operand and there is nothing left to concatenate with s, so the residual is s alone. Otherwise r' stays in front of s. *smartC* therefore states the concatenation rule without ever constructing $\varepsilon \cdot s$.

A set element is one possible residual, not one branch of a sum. When several alternatives can consume the same symbol, they stay as separate elements, and *smartC* is applied to each independently. One strand may finish the left operand while another still carries a non-empty residual in front of s. The constructor is therefore applied independently to each residual produced by $pd(r, x)$, preserving the distinct continuations of the concatenation without introducing redundant $\varepsilon \cdot s$ expressions.

In Rust. Figure 3.7 is *smartC*. `smart_seq` takes `r` by value and `s` by reference, matching (3.18). The residual r' comes from the recursive derivative and is consumed, while s must survive into the result. The $r' = \varepsilon$ branch clones only `s` and drops ε, so $\varepsilon \cdot s$ is never built.

```
pub fn smart_seq(r: Regex, s: &Regex) -> Regex {
    match r {
        Regex::Eps => s.clone(),
        r => Regex::seq(r, s.clone()),
    }
}
```

Figure 3.7: `smart_seq`, the smart constructor *smartC*

Example: No growth under *pd*. Consider $(x+y)^*$ against `xx` under *pd*.

P_* applies with star body $r = x + y$:

$$\begin{aligned} pd((x+y)^*, \mathtt{x}) &= \{\, smartC(r', (x+y)^*) \mid r' \in pd(x+y, \mathtt{x}) \,\} && \mathsf{P}_* \\ &= \{\, smartC(r', (x+y)^*) \mid r' \in \{\varepsilon\} \,\} && \mathsf{P}_+,\ \mathsf{P}_{\text{lit}} \text{ twice} \\ &= \{(x+y)^*\} && \mathsf{SC},\ r' = \varepsilon. \end{aligned} \tag{3.19}$$

$pd(x+y, \mathtt{x}) = \{\varepsilon\} \cup \{\} = \{\varepsilon\}$ by P_+, since `x` matches the left literal and misses the right, and *smartC* absorbs the ε straight back into $(x+y)^*$.

The set is $\{(x+y)^*\}$, so deriving by the second `x` repeats the computation:

$$pd((x+y)^*, \mathtt{x}, \mathtt{x}) = \{(x+y)^*\}.$$

Since $(x+y)^*$ is nullable by N_*, the final partial derivative accepts ε, so $\mathtt{xx} \in L((x+y)^*)$.

For comparison, applying the Brzozowski derivative to the same expression gives the intermediate alternation in (3.8), $(\emptyset+\emptyset)\cdot(x+y)^* + (\varepsilon+\emptyset)\cdot(x+y)^*$. Only after simplification in (3.12) does this reduce to $(x+y)^*$. Under *pd*, the residual set is $\{(x+y)^*\}$ after both steps, since $\mathsf{P}_\emptyset$ and P_ε produce no residual at all. So redundant terms are avoided at construction time rather than built and simplified afterwards

Running example: $r_{01} = (0+1)^* \cdot 0 \cdot 1$. Derive r_{01} against $w = \mathtt{101}$, tracking the residual set. The concatenations bind as in Section 3.2, with $(r, s) = ((0+1)^* \cdot 0, 1)$ at the outer node and $(r, s) = ((0+1)^*, 0)$ at the inner node; the star body is $0 + 1$. The derivation proceeds from the inside outward.

1. **Derive by `1`.** At the star body, P_+ distributes the derivative over two alternatives:

$$\begin{aligned} pd(0{+}1,\ \mathtt{1}) &= pd(0, \mathtt{1}) \cup pd(1, \mathtt{1}) && \mathsf{P}_+,\ r = 0,\ s = 1 \\ &= \{\} \cup \{\varepsilon\} && \mathsf{P}_{\text{lit}},\ \text{both branches} \\ &= \{\varepsilon\}. \end{aligned} \tag{3.20}$$

   The star rule then applies *smartC* to the resulting residual:

$$\begin{aligned} pd\Big((0+1)^*,\ \mathtt{1}\Big) &= \{\, smartC(r', (0+1)^*) \mid r' \in \{\varepsilon\} \,\} && \mathsf{P}_*,\ r = 0+1 \\ &= \{(0+1)^*\} && \mathsf{SC},\ r' = \varepsilon. \end{aligned} \tag{3.21}$$

At the middle concatenation, the left operand is nullable by N_*, so the nullable case contributes both the derivatives through the left and right operand:

$$\begin{aligned} pd\Big((0+1)^*\cdot 0,\ \mathtt{1}\Big) &= \{\, smartC(r',0) \mid r' \in pd\Big((0+1)^*,\mathtt{1}\Big)\,\} \cup pd(0,\mathtt{1}) && \mathsf{P}^n_{\text{seq}} \\ &= \{smartC((0+1)^*,0)\} \cup \{\} && \text{above; } \mathsf{P}_{\text{lit}} \\ &= \{(0+1)^*\cdot 0\} && \mathsf{SC},\ r' \neq \varepsilon. \end{aligned} \tag{3.22}$$

At the outer node, the left operand is not nullable, so only one branch contributes:

$$\begin{aligned} pd(r_{01},\ \mathtt{1}) &= \{\, smartC(r',1) \mid r' \in \{(0+1)^*\cdot 0\}\,\} && \mathsf{P}_{\text{seq}},\ r=(0+1)^*\cdot 0,\ s=1 \\ &= \{(0+1)^*\cdot 0\cdot 1\} = \{r_{01}\} && \mathsf{SC}. \end{aligned} \tag{3.23}$$

The first symbol leaves $\{r_{01}\}$, returning to the starting expression since the consumed `1` belongs to the leading star.

2. **Derive by `0`.** The current set is still $\{r_{01}\}$. As before, $pd\Big((0+1)^*,\mathtt{0}\Big) = \{(0+1)^*\}$. At the middle concatenation, however, the right operand is now the matching literal 0, so P_{lit} contributes $\{\varepsilon\}$:

$$\begin{aligned} pd\Big((0+1)^*\cdot 0,\ \mathtt{0}\Big) &= \{smartC((0+1)^*,0)\} \cup pd(0,\mathtt{0}) && \mathsf{P}^n_{\text{seq}},\ r=(0+1)^*,\ s=0 \\ &= \{(0+1)^*\cdot 0\} \cup \{\varepsilon\} && \mathsf{P}_{\text{lit}},\ \text{since } \mathtt{0}=0 \\ &= \{(0+1)^*\cdot 0,\ \varepsilon\}. \end{aligned} \tag{3.24}$$

The outer concatenation applies $smartC$ separately to both residuals:

$$\begin{aligned} pd(r_{01},\mathtt{1},\mathtt{0}) &= \{\, smartC(r',1) \mid r' \in \{(0+1)^*\cdot 0,\varepsilon\}\,\} && \mathsf{P}_{\text{seq}},\ s=1 \\ &= \{(0+1)^*\cdot 0\cdot 1,\ 1\} && \mathsf{SC} \\ &= \{r_{01},\ 1\}. \end{aligned} \tag{3.25}$$

The two residuals are the possible strands after `10`: r_{01} keeps both symbols in the leading star, while 1 means the `0` began the final `01`. Since $smartC(\varepsilon,1)=1$, the latter is represented directly as 1.

3. **Derive by `1`.** The two residuals are derived independently and their results are unioned:

$$\begin{aligned} pd(r_{01},\mathtt{1},\mathtt{0},\mathtt{1}) &= pd(r_{01},\mathtt{1}) \cup pd(1,\mathtt{1}) && \text{union over the set} \\ &= \{r_{01}\} \cup \{\varepsilon\} && \text{step 1; } \mathsf{P}_{\text{lit}} \\ &= \{r_{01},\ \varepsilon\}. \end{aligned} \tag{3.26}$$

The final set contains ε, so one surviving residual is nullable and $\mathtt{101} \in L(r_{01})$. This agrees with the conclusion reached by the Brzozowski derivative in (3.15).

For $w = \mathtt{10}$, the first two steps leave $\{r_{01}, 1\}$, which represents the same alternatives as $\Big((0+1)^*\cdot 0+\varepsilon\Big)\cdot 1$ in (3.14). Either the trailing `01` remains to be matched, or the consumed `0` was its first symbol. Neither residual is nullable, so $\mathtt{10} \notin L(r_{01})$.

Finiteness. For a fixed regular expression r, the set of reachable partial derivatives is finite and bounded by $\|r\| + 1$, where $\|r\|$ denotes the number of literal occurrences in r [Antimirov, 1996, Theorem 3.4, p. 309]. For r_{01}, $\|r_{01}\| = 4$ (the four literals: $0, 1, 0, 1$), so the bound is $4 + 1 = 5$ distinct residuals across all possible inputs. The example with $w = \texttt{101}$ reaches only three of them: $\{r_{01}\}$, $\{r_{01}, 1\}$, and $\{r_{01}, \varepsilon\}$. Other residuals in the theoretical set are reachable only by different input sequences.

For ordinary derivatives, finiteness requires identifying equivalent expressions [Brzozowski, 1964, Theorem 4.3], yielding an exponential bound $d_R \leq 2^{d_P} - 1$. Simplification rules eliminate only units and redundant $\emptyset$ expressions, not all equivalent forms. The linear bound for partial derivatives applies directly without simplification.

The matcher. A *frontier* of residuals is updated one symbol at a time by taking pd of every element and unioning the results. The frontier F_i represents all residuals reachable after consuming the first i input symbols—every possible state the computation could be in. For frontier F_i after i symbols,

$$F_0 = \{r\}, \qquad F_i = \bigcup_{r' \in F_{i-1}} pd(r', x_i) \quad (1 \leq i \leq n). \tag{3.27}$$

The matcher accepts when some residual in the final frontier is nullable. Where Brzozowski's $r_0 \to r_1 \to \cdots \to r_n$ traces one path through a lazily constructed DFA, $F_0 \to F_1 \to \cdots \to F_n$ traces all states an NFA explores simultaneously, holding each state at once rather than committing to a single residual expression.

In Rust. Figure 3.8 implements (3.27) using a `HashSet<Regex>` frontier, where `Regex`'s `Hash` and `Eq` implementations allow residuals from different paths to collapse. Where the Brzozowski matcher holds one `Regex`, this holds a set. The early return when the frontier empties follows from construction: no later derivative produces residuals from an empty set. The final `any(nullable)` checks whether some residual is nullable. The matcher trades a single growing expression for a set of small ones that never need simplification.

```
pub fn match_pderiv(input: &str, r: &Regex) -> bool {
    let mut states: HashSet<Regex> = HashSet::new();
    states.insert(r.clone());
    for c in input.chars() {
        let mut next: HashSet<Regex> = HashSet::new();
        for state in &states {
            next.extend(pderiv(state, c));
        }
        if next.is_empty() { return false; }
        states = next;
    }
    states.iter().any(nullable)
}
```

Figure 3.8: The partial-derivative matcher.

4 Matching and Parsing

Chapter 3 established matching with a Boolean answer: $w \in L(r)$. A matcher only answers whether the input is accepted by the regular expression. Parsing identifies the evidence for the match and *parse trees* act as their structured witnesses.

4.1 Matching: A Boolean Answer

Consider $r = (a + ab)(b + \varepsilon)$ and $w = \texttt{ab}$. The matcher can determine $w \in L(r)$ using the algorithms of Chapter 3. There are, however, two ways in which r can match w:

$$\underbrace{a}_{a+ab}\ \underbrace{b}_{b+\varepsilon} \quad \text{and} \quad \underbrace{\texttt{ab}}_{a+ab}\ \underbrace{\varepsilon}_{b+\varepsilon} \tag{4.1}$$

Both choices establish the same membership result, but assign the input differently to the subexpressions of r. The Boolean result alone cannot distinguish these cases, so the matcher cannot reveal which decomposition was chosen.

Parsing makes this additional structure explicit, by showing which alternatives were chosen and which parts of the input were consumed by the corresponding subexpressions.

4.2 Parsing: Parse Trees as Matching Evidence

Parse Tree. A *parse tree* provides a structured witness for such a derivation. Sulzmann and Lu view regular expressions as types and *parse trees* as their values [Sulzmann and Lu, 2014]. The judgment $\vdash v : r$ states that v is a valid parse tree for r, recording which alternatives were chosen and how each part of the input was consumed during matching:

$$\frac{}{\vdash () : \varepsilon}\ (\mathsf{T}_\varepsilon) \qquad \frac{\vdash v_1 : r_1}{\vdash \mathit{Left}\ v_1 : r_1 + r_2}\ (\mathsf{T}^L_+)$$

$$\frac{l \in \Sigma}{\vdash l : l}\ (\mathsf{T}_{\text{lit}}) \qquad \frac{\vdash v_2 : r_2}{\vdash \mathit{Right}\ v_2 : r_1 + r_2}\ (\mathsf{T}^R_+) \tag{4.2}$$

$$\frac{\vdash v_1 : r_1 \qquad \vdash v_2 : r_2}{\vdash (v_1, v_2) : r_1 r_2}\ (\mathsf{T}_{\text{seq}}) \qquad \frac{}{\vdash [\,] : r^*}\ (\mathsf{T}^0_*)$$

$$\frac{\vdash v : r \qquad \vdash vs : r^*}{\vdash (v : vs) : r^*}\ (\mathsf{T}_*)$$

In Rust. Figure 4.1 shows the parse-tree type, with one variant for each constructor.

```
#[derive(Debug, Clone, PartialEq, Eq)]
pub enum ParseTree {
    Empty,                                  // ()
    Char(char),                             // l
    Pair(Box<ParseTree>, Box<ParseTree>),   // (v1, v2)
    Left(Box<ParseTree>),                   // Left v
    Right(Box<ParseTree>),                  // Right v
    Star(Vec<ParseTree>),                   // [v1, ..., vn]
}
```

Figure 4.1: The parse tree type, representing the structure of a parse.

The definitions above translate directly to Rust, except for the star case. $\mathsf{T}\varepsilon$ corresponds to `Empty` for the empty parse, T_{lit} to `Char` for a literal, T_{seq} to `Pair` for a concatenation of parse trees, and T_{+}^{L} and T_{+}^{R} to `Left` and `Right`, recording the chosen branch of an alternation. Both star rules are represented by `Star(vec![])`.

Flattening. The typing judgment describes the structure of a match, but a parse tree must also be related to the word that it matches. For this purpose, they define a *flattening* function $|v|$ that maps a parse tree to the word represented by its structure:

$$\begin{aligned} |()| &= \varepsilon, & |(v_1, v_2)| &= |v_1||v_2|, \\ |l| &= l, & |\mathit{Left}\ v| &= |v|, \\ |[]| &= \varepsilon & |\mathit{Right}\ v| &= |v|, \\ & & |v : vs| &= |v||vs|. \end{aligned} \tag{4.3}$$

The flattening function directly connects parse trees to $L(r)$, and thus validates that a parse tree witnesses membership in $L(r)$, serving as an correctness check:

$$L(r) = \{\, |v| \mid\mid\!\vdash v : r \,\}. \tag{4.4}$$

In Rust. Figure 4.2 shows how `flatten` recursively traverses the parse tree and concatenates its leaf characters. The function borrows the tree to allow subsequent inspection and comparison. The `Pair` case concatenates the results of flattening both children, while `Left` and `Right` delegate to their child, and `Star` concatenates the results for each element.

```
pub fn flatten(v: &ParseTree) -> String {
    match v {
        ParseTree::Empty        => String::new(),
        ParseTree::Char(c)      => c.to_string(),
        ParseTree::Pair(v1, v2) => format!("{}{}", flatten(v1), flatten(v2)),
        ParseTree::Left(v)      => flatten(v),
        ParseTree::Right(v)     => flatten(v),
        ParseTree::Star(vs)     => vs.iter().map(flatten).collect(),
    }
}
```

Figure 4.2: The `flatten` function extracts the matched word from a parse tree.

Unambiguous example. Consider $r = ab$ and $w = \texttt{ab}$. Since each subexpression is a literal, the rule T_{lit} gives each literal a unique witness. Then T_{seq} requires the parse tree for a to be followed by the parse tree for b, combining them into a parse tree for ab:

$$\frac{a \in \Sigma}{\vdash a : a}\,(\mathsf{T}_{\text{lit}}) \qquad \frac{b \in \Sigma}{\vdash b : b}\,(\mathsf{T}_{\text{lit}}) \qquad \frac{\vdash a : a \qquad \vdash b : b}{\vdash (a, b) : ab}\,(\mathsf{T}_{\text{seq}}) \tag{4.5}$$

The parse tree $v = (a, b)$ flattens to $|(a, b)| = \texttt{ab}$.

Ambiguous Example. Recall $r = (a + ab)(b + \varepsilon)$ and $w = \texttt{ab}$ from Section 4.1. The word admits two distinct decompositions, corresponding to different parse trees.

The first decomposition uses the right alternatives of both subexpressions:

$$\underbrace{\frac{\dfrac{a \in \Sigma}{\vdash a : a}\,(\mathsf{T}_{\text{lit}}) \qquad \dfrac{b \in \Sigma}{\vdash b : b}\,(\mathsf{T}_{\text{lit}})}{\vdash (a, b) : ab}\,(\mathsf{T}_{\text{seq}})}{\vdash \mathit{Right}(a, b) : a + ab}\,(\mathsf{T}^R_+)}_{a + ab} \qquad \underbrace{\frac{\dfrac{b \in \Sigma}{\vdash b : b}\,(\mathsf{T}_{\text{lit}}) \qquad \vdash () : \varepsilon\,(\mathsf{T}_\varepsilon)}{\vdash \mathit{Right}() : b + \varepsilon}\,(\mathsf{T}^R_+)}_{b + \varepsilon}$$

By T_{seq}, these derivations combine into the parse tree:

$$\frac{\dfrac{\vdash a : a \qquad \vdash b : b}{\vdash (a, b) : ab}}{\vdash \mathit{Right}(a, b) : a + ab} \qquad \frac{\vdash () : \varepsilon}{\vdash \mathit{Right}() : b + \varepsilon} \tag{4.6}$$

$$v_1 = \Big(\mathit{Right}(a, b), \mathit{Right}()\Big), \qquad |v_1| = \texttt{ab}.$$

The second decomposition uses the left alternatives of both subexpressions:

$$\underbrace{\frac{\dfrac{a \in \Sigma}{\vdash a : a}\,(\mathsf{T}_{\text{lit}})}{\vdash \mathit{Left}\, a : a + ab}\,(\mathsf{T}^L_+)}_{a + ab} \qquad \underbrace{\frac{\dfrac{b \in \Sigma}{\vdash b : b}\,(\mathsf{T}_{\text{lit}})}{\vdash \mathit{Left}\, b : b + \varepsilon}\,(\mathsf{T}^L_+)}_{b + \varepsilon}$$

By T_{seq}, these derivations combine into the second parse tree:

$$\frac{\vdash a : a}{\vdash \mathit{Left}\, a : a + ab} \qquad \frac{\vdash b : b}{\vdash \mathit{Left}\, b : b + \varepsilon} \tag{4.7}$$

$$v_2 = \Big(\mathit{Left}\, a, \mathit{Left}\, b\Big), \qquad |v_2| = \texttt{ab}.$$

Both trees are valid parse trees for the same expression and word, yet they differ in structure. The typing judgment alone does not determine which parse tree should be selected. A *disambiguation policy* specifies this choice.

5 Derivative-Based Parser

Chapter 3 showed how a Brzozowski-derivative matcher determines whether $w \in L(r)$ and Chapter 4 showed that a matching word may admit multiple valid parse trees. A parser must therefore select a single parse tree according to a disambiguation policy.

Following Sulzmann and Lu [Sulzmann and Lu, 2014], this chapter presents two derivative-based methods for constructing POSIX parse trees. The direct method performs a forward derivative pass followed by a backward pass for tree reconstruction. The bit-coded method records parse-tree information during the forward pass, avoiding a separate backward pass.

5.1 The Standard Derivative-Based Parser

Consider the ambiguous example from Chapter 4, $r = (a + ab)(b + \varepsilon)$ with input $w = \texttt{ab}$, and ask whether a parse tree can be recovered from the final derivative.

$$\begin{aligned} r &\xrightarrow{a} (\varepsilon + \varepsilon b)(b + \varepsilon) \\ &\xrightarrow{b} (\phi + (\phi b + \varepsilon))(b + \varepsilon) + (\varepsilon + \phi). \end{aligned} \tag{5.1}$$

The final derivative is nullable, so $w \in L(r)$ and both disjuncts accept the empty word. Its first disjunct, $(\phi + (\phi b + \varepsilon))(b + \varepsilon)$, reflects $ab \cdot \varepsilon$, and its second, $(\varepsilon + \phi)$, reflects $a \cdot b$.

Since a derivative step only records the remaining input language and not how consumed characters were matched, the final derivative alone does not indicate which decomposition was selected, and therefore cannot be used to recover the parse tree.

However, the derivation sequence itself contains this information. Each step consumed one character and is invertible given that character, so reversing the steps recovers the tree. Figure 5.1 shows the forward pass as the derivation sequence and the backward pass as the reconstruction.

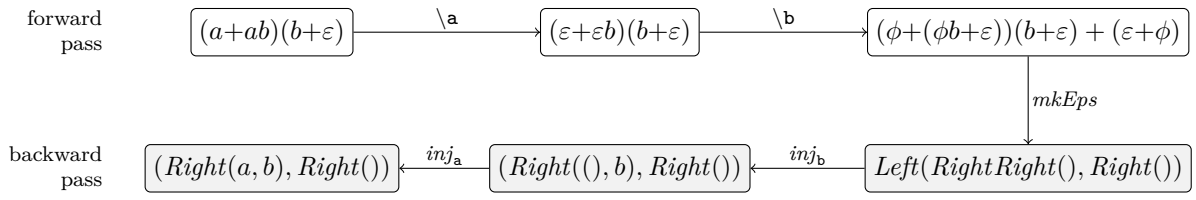


Figure 5.1: Overview of the standard derivative-based parser on the ambiguous example. The forward pass computes derivatives, while the backward pass uses *mkEps* to construct a parse tree for the final nullable derivative and *inj* to reconstruct the original parse tree.

The two-pass pipeline. The direct construction uses two passes. The forward pass applies `deriv` (Figure 3.3) and `nullable` (Figure 3.2) to process the input.

The backward pass applies two new functions to walk the derivation sequence in reverse and reconstruct the parse tree. First *mkEps* builds a parse tree for the empty word from the final nullable expression. Then injection steps are applied, with *inj* reinserting one consumed character into the parse tree, moving in w from right to left. Each injection step takes the current parse tree, the corresponding derivative step, and its consumed character, and returns the parse tree for the preceding expression.

As shown in Figure 5.1 after the first injection of b, the tree is $(\mathit{Right}((), b), \mathit{Right}())$, where $()$ marks the position still to be filled. After the second injection of a, that position is replaced by the subtree $\mathit{Right}(a, b)$, giving the complete parse tree $(\mathit{Right}(a, b), \mathit{Right}())$.

In the following the operations for the standard derivative parser are shown in more detail.

5.1.1 Building the Empty-Word Tree

The backward pass begins with *mkEps*, which reads a parse tree for the empty word off the final nullable expression. The function follows [Sulzmann and Lu, 2014, Fig. 3] and is defined recursively:

$$\begin{aligned}
\mathit{mkEps}_{\varepsilon} &= () && (\mathsf{E}_{\varepsilon}) \\
\mathit{mkEps}_{r^*} &= [\,] && (\mathsf{E}_{*}) \\
\mathit{mkEps}_{r_1 r_2} &= (\mathit{mkEps}_{r_1}, \mathit{mkEps}_{r_2}) && (\mathsf{E}_{\mathrm{seq}}) \\
\mathit{mkEps}_{r_1 + r_2} &= \begin{cases} \mathit{Left}\ \mathit{mkEps}_{r_1} & \text{if } n(r_1) \quad (\mathsf{E}_{+}^{L}) \\ \mathit{Right}\ \mathit{mkEps}_{r_2} & \text{otherwise} \quad (\mathsf{E}_{+}^{R}) \end{cases}
\end{aligned} \tag{5.2}$$

The first three rules follow from the structure of the expression: $(\mathsf{E}_{\varepsilon})$ gives ε an empty parse tree, (E_{*}) treats a star matching the empty word as zero iterations, and $(\mathsf{E}_{\mathrm{seq}})$ matches a concatenation against ε only when both factors do. No rules are needed for $\emptyset$ or literals, since neither can match the empty word. The alternation rule is where disambiguation occurs. If exactly one of r_1 and r_2 is nullable, that branch is the only possible witness. If both are, (E_{+}^{L}) breaks the tie by preferring the left one, following POSIX's leftmost-longest policy [Sulzmann and Lu, 2014]. In the example, the final expression is:

$$(\phi + (\phi b + \varepsilon))(b + \varepsilon) + (\varepsilon + \phi).$$

At top-level $+$, both summands are nullable, so (E_{+}^{L}) applies and picks the left one. Within the left summand's first factor $\phi + (\phi b + \varepsilon)$, only the right alternative is nullable, so (E_{+}^{R}) applies, and within it only the right sub-alternative ε is nullable, so (E_{+}^{R}) applies again, contributing $\mathit{RightRight}()$. The second factor $b + \varepsilon$ contributes $\mathit{Right}()$ via (E_{+}^{R}), giving:

$$\mathit{Left}(\mathit{RightRight}(), \mathit{Right}()).$$

Because disambiguation is deferred to *mkEps*, the forward pass cannot simplify. The second injection recovers $\mathit{Right}(a, b)$ only because the forward pass preserved the branch that consumed a. Had it been simplified away, the backward pass would have had no node to reinsert a into. For example, simplifying at each step collapses the derivation to:

$$r \xrightarrow{a} (\varepsilon + b)(b + \varepsilon) \xrightarrow{b} \varepsilon,$$

This still witnesses $w \in L(r)$ but keeps no record of which alternative consumed a or b. The matcher accepts, but a parser cannot recover $\mathit{Right}(a, b)$ from ε.

In Rust. The recursive definition of *mkEps* (5.2) is implemented directly in Rust, as shown in Figure 5.2, following the same cases for each expression form.

```
pub fn mk_eps(r: &Regex) -> ParseTree {
    match r {
        Regex::Eps          => ParseTree::Empty,
        Regex::Star(_)      => ParseTree::Star(Vec::new()),
        Regex::Seq(r1, r2) => ParseTree::Pair(
                               Box::new(mk_eps(r1)), Box::new(mk_eps(r2))),
        Regex::Alt(r1, r2) =>
            if nullable(r1)   {ParseTree::Left(Box::new(mk_eps(r1)))}
            else              {ParseTree::Right(Box::new(mk_eps(r2)))},
        Regex::Lit(c)       => panic!("mk_eps called on Lit('{}')", c),
        Regex::Phi          => panic!("mk_eps called on Phi"),
    }
}
```

Figure 5.2: `mk_eps`: construct a parse tree for the empty word.

The expression r is borrowed rather than consumed because the backward pass needs it again during injection. The alternation arm encodes disambiguation as a conditional, examining `r2` only when `nullable(r1)` is false. The panic arms enforce the precondition that *mkEps* applies only to nullable expressions, rejecting the non-nullable ϕ and literals.

5.1.2 Reinserting Characters via Injection

After *mkEps* constructed the parse tree v for the empty input, the reconstruction starts from this initial tree and proceeds backwards through the derivation sequence. Each application of *inj* takes the current tree, the corresponding derivative step, and its consumed character, returning a tree for the preceding expression. *inj* recursively follows the structure of the residual until it reaches the subexpression containing the literal that consumed the character. It reinserts the character there and then unwinds to rebuild the surrounding tree. After all characters have been reinserted, the parse tree for the original expression and input is recovered. The injection function is defined by the following rules [Sulzmann and Lu, 2014, Fig. 3]:

$$
\begin{aligned}
\mathit{inj}_{x\backslash x}() &= x && (\mathsf{I}_{\mathrm{lit}}) \\
\mathit{inj}_{(r_1+r_2)\backslash x}(\mathit{Left}\ v_1) &= \mathit{Left}\ (\mathit{inj}_{r_1\backslash x}\ v_1) && (\mathsf{I}^{L}_{+}) \\
\mathit{inj}_{(r_1+r_2)\backslash x}(\mathit{Right}\ v_2) &= \mathit{Right}\ (\mathit{inj}_{r_2\backslash x}\ v_2) && (\mathsf{I}^{R}_{+}) \\
\mathit{inj}_{r^*\backslash x}(v, vs) &= (\mathit{inj}_{r\backslash x}\ v) : vs && (\mathsf{I}_{*}) \\
\mathit{inj}_{(r_1 r_2)\backslash x}(v_1, v_2) &= (\mathit{inj}_{r_1\backslash x}\ v_1,\ v_2) && (\mathsf{I}_{\mathrm{seq}}),\ n(r_1) = \text{false} \\
\mathit{inj}_{(r_1 r_2)\backslash x}(\mathit{Left}(v_1, v_2)) &= (\mathit{inj}_{r_1\backslash x}\ v_1,\ v_2) && (\mathsf{I}^{L}_{\mathrm{seq}}),\ n(r_1) = \text{true} \\
\mathit{inj}_{(r_1 r_2)\backslash x}(\mathit{Right}\ v_2) &= (\mathit{mkEps}_{r_1},\ \mathit{inj}_{r_2\backslash x}\ v_2) && (\mathsf{I}^{R}_{\mathrm{seq}}),\ n(r_1) = \text{true}
\end{aligned}
\tag{5.3}
$$

Rules (I_{lit}) and (I_{*}) invert their derivative rules directly, reconstructing the consumed character and the current iteration, the latter prepended to the remaining ones.

Since the consumed character lies in exactly one branch of an alternation, the incoming tree is either *Left* v_1 or *Right* v_2. Rule (I_{+}^{L}) recurses into v_1 and rebuilds the *Left* node, and (I_{+}^{R}) recurses into v_2 and rebuilds the *Right* node.

The concatenation rules cover the three cases of the sequence derivative. If $n(r_1) = \text{false}$, r_1 cannot match ε and must consume x, so (I_{seq}) recurses into v_1. If $n(r_1) = \text{true}$, the derivative recorded which component consumed x as a tag: ($\mathsf{I}_{\text{seq}}^{L}$) handles an incoming $\mathit{Left}(v_1, v_2)$ tag and recurses into v_1, while ($\mathsf{I}_{\text{seq}}^{R}$) handles a *Right* v_2 tag by reconstructing r_1's empty match with mkEps_{r_1} and recursing into v_2.

Since ϕ consumes no characters, *inj* is never applied with ϕ as the residual.

```
pub fn inject(r: &Regex, l: char, v: ParseTree) -> ParseTree {
    match r {
        Regex::Lit(c) => { assert!(*c == l);
            ParseTree::Char(l) }
        Regex::Alt(r1, r2) => match v {
            ParseTree::Left(v1) =>
                ParseTree::Left(Box::new(inject(r1, l, *v1))),
            ParseTree::Right(v2) =>
                ParseTree::Right(Box::new(inject(r2, l, *v2))),
            _ => panic!("inject/Alt: want Left|Right"),
        },
        Regex::Star(r1) => match v {
            ParseTree::Pair(v1, vs) => {
                let mut it = vec![inject(r1, l, *v1)];
                if let ParseTree::Star(rest) = *vs { it.extend(rest); }
                ParseTree::Star(it)
            }
            _ => panic!("inject/Star: want Pair"),
        },
        Regex::Seq(r1, r2) => if !nullable(r1) {
            match v {
                ParseTree::Pair(v1, v2) => ParseTree::Pair(
                    Box::new(inject(r1, l, *v1)), v2),
                _ => panic!("inject/Seq: want Pair"),
            }
        } else {
            match v {
                ParseTree::Left(vp) => match *vp {
                    ParseTree::Pair(v1, v2) => ParseTree::Pair(
                        Box::new(inject(r1, l, *v1)), v2),
                    o => panic!("inject/SeqL: want Pair, got {:?}", o),
                },
                ParseTree::Right(v2) => ParseTree::Pair(
                    Box::new(mk_eps(r1)),
                    Box::new(inject(r2, l, *v2))),
                _ => panic!("inject/Seq: want Left|Right"),
            }
        },
        Regex::Eps | Regex::Phi => panic!("inject called on Eps/Phi"),
    }
}
```

Figure 5.3: `inject`: reinsert a character into a parse tree.

In Rust. `inject(r, l, v)` takes three arguments because *inj* is indexed by both an expression and a character. Figure 5.3 shows the complete implementation.

The function matches on the expression `r` first and on the tree `v` second, following the order in which (5.3) indexes its rules. This order matters because a tree tag is interpreted relative to the expression that produced it. Under an alternation, `Left` selects the left disjunct, while under a nullable concatenation it indicates that the derivative continued inside the left operand. The `Lit` arm reconstructs a leaf after asserting that the consumed

character matches, and the `Star` arm inverts (I_*), reinserting the character into the current iteration and prepending it to the remaining ones.

The expression `r` is borrowed and read only, while the tree `v` is consumed and decomposed. In the concatenation arms where the right operand was not derived, `v2` moves directly into the new pair, since (5.3) leaves it unchanged. The `Right` arm of concatenation is the only concatenation case that synthesizes content, calling `mk_eps(r1)` because the derivative skipped `r1` and the forward pass produced no tree for that half of the pair.

Calling `nullable(r1)` in the `Seq` arm recomputes a decision the forward pass already made when it chose between D_{seq} and $\mathsf{D}^n_{\text{seq}}$. However, carrying this as a flag at every step would require additional storage, while `nullable` is cheap to evaluate on demand.

5.1.3 Parser

The parser composes `mkEps` and `inject` by storing every forward-pass expression and reconstructing the tree in reverse. Since `inject` takes the preceding expression as an explicit argument, each call needs r_{i-1}. Re-deriving it from r through the same prefix would duplicate the forward pass, so storing it avoids computing the derivation again.

Figure 5.4 implements this iteratively, using a `Vec` that grows dynamically on the heap.

```
pub fn parse_deriv_std_loop(input: &str, r: &Regex) -> Option<ParseTree>
{
    let chars: Vec<char> = input.chars().collect();
    let mut expressions = Vec::with_capacity(chars.len() + 1);
    expressions.push(r.clone());
    for &c in &chars {
        expressions.push(deriv(expressions.last().unwrap(), c));
    }
    if !nullable(expressions.last().unwrap()) {
      return None;
      }
    let mut tree = mk_eps(expressions.last().unwrap());
    for i in (0..chars.len()).rev() {
        tree = inject(&expressions[i], chars[i], tree);
    }
    Some(tree)
}
```

Figure 5.4: The standard derivative-based parser. The forward pass stores the residual after each character in `expressions`, entry i being the residual after i characters, and returns `None` if the final entry is not nullable. The backward pass starts from `mk_eps` of the final entry and applies `inject` to the preceding entries in reverse, yielding the parse tree for r.

5.1.4 Worked Example: Ambiguous Expression

Recall the ambiguous example of Section 4.2, $r_0 = (a \mid ab)(b \mid \varepsilon)$ on input `ab`.

Forward pass. There are two derivative steps, one per input symbol. After deriving $\mathtt{a}$, the left factor $\varepsilon + \varepsilon b$ is nullable, so the second derivation on $\mathtt{b}$ uses $\mathsf{D}^n_{\text{seq}}$ rather than D_{seq}:

$$r_0 = (a + ab)(b + \varepsilon) \xrightarrow[\mathsf{D}_{\text{seq}}]{a} r_1 = (\varepsilon + \varepsilon b)(b + \varepsilon)$$
$$\xrightarrow[\mathsf{D}^n_{\text{seq}}]{b} r_2 = \underbrace{(\phi + (\phi b + \varepsilon))(b + \varepsilon)}_{X} + \underbrace{(\varepsilon + \phi)}_{Y}.$$

The resulting final expression is r_2, both X and Y are nullable. The disjunct X is the residual of the ab alternative in the first factor, and Y is the residual of the a alternative.

Backward pass: mkEps. The backward pass starts by building the parse tree for the empty word: $mkEps(r_2)$. Since $r_2 = X + Y$ and both disjuncts are nullable, E^L_+ selects X. As X is a concatenation, E_{seq} combines the trees for its two factors:

$$mkEps(r_2) = mkEps(X + Y) \xrightarrow{\mathsf{E}^L_+} Left\, mkEps(X)$$

$$mkEps(X) \xrightarrow{\mathsf{E}_{\text{seq}}} \Big(mkEps(\phi + (\phi b + \varepsilon)), mkEps(b + \varepsilon)\Big).$$

Since ϕ and ϕb are not nullable, the first factor selects ε by applying the rule E^R_+ twice, while the second factor selects ε through E^R_+ since b is not nullable, giving v_2:

$$mkEps(\phi + (\phi b + \varepsilon)) \xrightarrow{\mathsf{E}^R_+} Right\, mkEps(\phi b + \varepsilon) \xrightarrow{\mathsf{E}^R_+} Right\, Right\, mkEps(\varepsilon) \xrightarrow{\mathsf{E}_\varepsilon} Right\, Right\, ()$$

$$mkEps(b + \varepsilon) \xrightarrow{\mathsf{E}^R_+} Right\, mkEps(\varepsilon) \xrightarrow{\mathsf{E}_\varepsilon} Right\, ()$$

$$v_2 = mkEps(r_2) = Left\Big(Right\, Right\, (), Right\, ()\Big).$$

Backward pass: Inj. Reinsertion begins with $\mathtt{b}$ and maps v_2 to a tree for $r_1 = (\varepsilon + \varepsilon b)(b + \varepsilon)$. Since v_2 carries a *Left* tag, $\mathsf{I}^L_{\text{seq}}$ recurses into $\varepsilon + \varepsilon b$, leaving the second factor's tree $Right\, ()$ unchanged, as all input characters belong to the first factor under the $ab \cdot \varepsilon$ decomposition. Inside $\varepsilon + \varepsilon b$, I^R_+ selects εb, the residual of the ab alternative. Since ε is nullable, $\mathsf{I}^R_{\text{seq}}$ supplies $mkEps(\varepsilon) = ()$ and recurses into b, where I_{lit} reinserts $\mathtt{b}$.

$$inj_{r_1 \backslash \mathtt{b}}(v_2) = inj_{r_1 \backslash \mathtt{b}}\Big(\underbrace{Left(Right\, Right\, (),\ Right\, ())}_{v_2}\Big)$$
$$\xrightarrow{\mathsf{I}^L_{\text{seq}}} \Big(inj_{(\varepsilon+\varepsilon b)\backslash \mathtt{b}}(Right\, Right\, ()),\ Right\, ()\Big)$$
$$\xrightarrow{\mathsf{I}^R_+} \Big(Right\, inj_{\varepsilon b \backslash \mathtt{b}}(Right\, ()),\ Right\, ()\Big)$$
$$\xrightarrow{\mathsf{I}^R_{\text{seq}}} \Big(Right\, (mkEps(\varepsilon),\ inj_{b\backslash \mathtt{b}}()),\ Right\, ()\Big)$$
$$\xrightarrow{\mathsf{E}_\varepsilon,\ \mathsf{I}_{\text{lit}}} \underbrace{\Big(Right((), b),\ Right\, ()\Big)}_{v_1}$$

Reinsertion of $\mathtt{a}$ maps v_1 to a parse tree for $r_0 = (a + ab)(b + \varepsilon)$. The outer sequence's first factor $a + ab$ is not nullable, so I_{seq} fires directly on v_1's first component, again leaving the

second component $Right\,()$ untouched. That first component, $Right((), b)$, carries a $Right$ tag, so I_{+}^{R} selects ab. Inside $ab = Seq(a, b)$ the first factor a is again not nullable, so I_{seq} fires once more and recurses into a, where I_{lit} reinserts the character:

$$\begin{aligned} inj_{r_0\backslash \mathtt{a}}(v_1) &= inj_{r_0\backslash \mathtt{a}}\Big(\underbrace{(Right((), b),\ Right\,())}_{v_1}\Big) \\ &\xrightarrow{\mathsf{I}_{\text{seq}}} \Big(inj_{(a+ab)\backslash \mathtt{a}}(Right((), b)),\ Right\,()\Big) \\ &\xrightarrow{\mathsf{I}_{+}^{R}} \Big(Right\, inj_{ab\backslash \mathtt{a}}((), b)),\ Right\,()\Big) \\ &\xrightarrow{\mathsf{I}_{\text{seq}}} \Big(Right\,(inj_{a\backslash \mathtt{a}}()),\ b),\ Right\,()\Big) \\ &\xrightarrow{\mathsf{I}_{\text{lit}}} \underbrace{\Big(Right(a, b),\ Right\,()\Big)}_{v_0} \end{aligned}$$

This implementation stores three intermediate expressions, r_0, r_1, and r_2, one per input character plus the initial expression, for the backward pass to reconstruct from.

5.2 The Bit-Coded Derivative-Based Parser

The direct construction of Sulzmann and Lu [Sulzmann and Lu, 2014] reconstructs the tree only after the full input, since each injection step needs the preceding expression. Their bit-coded optimization records this information during a single pass instead.

Each subexpression carries the bit-sequence it has committed to so far, written $bs@r$. P and Q denote the annotated factors for $r_0 = (a + ab)(b + \varepsilon)$. Deriving $w = \mathtt{ab}$ gives:

$$\begin{aligned} ri_0 &= []@\Big(\underbrace{[0]@a \oplus [1]@(a\,b)}_{P} \cdot \underbrace{[0]@b \oplus [1]@\varepsilon}_{Q}\Big) \\ &\xrightarrow{a} ri_1 = []@\Big(\underbrace{[0]@\varepsilon \oplus [1]@b}_{P'} \cdot Q\Big) \\ &\xrightarrow{b} ri_2 = []@\Big([1]@Q \ \oplus\ [0,0]@\varepsilon\Big). \end{aligned} \tag{5.4}$$

Figure 5.5 shows this derivation together with the extraction step it feeds into:

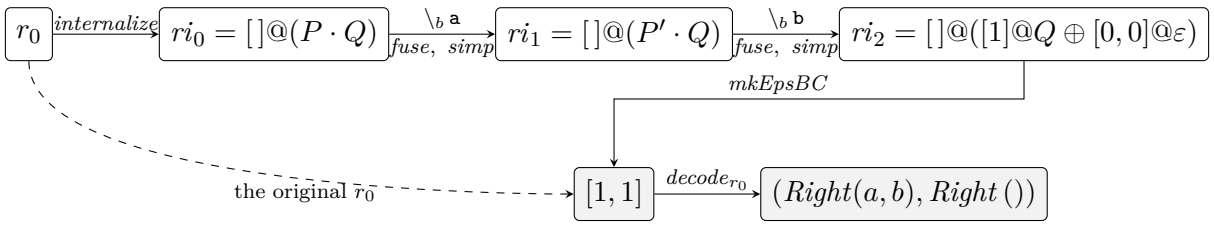


Figure 5.5: Overview of the bit-coded derivative parser on the ambiguous example. *internalize* introduces the bits, each derivative step threads them forward, so consumed factors' bits transfer to survivors. *mkEpsBC* extracts them and $decode_{r_0}$ reads them against the original pattern.

The bit-coded pipeline. *internalize* tags each alternation branch with a bit that records its preference, producing $ri_0 = []@(P \cdot Q)$ with $P = [0]@a \oplus [1]@(a\,b)$ and $Q = [0]@b \oplus [1]@\varepsilon$, where 0 selects the left branch and 1 the right.

Deriving by `a` consumes P's leading a, giving P', while Q remains untouched. Deriving by `b` consumes the rest. Since P''s ε branch is nullable, both decompositions survive. In the first, the derivative continues inside P', which finishes and folds its bit 1 onto Q. In the second, Q matches `b` instead, and P''s completion bit fuses with Q's match into $[0,0]@\varepsilon$. This yields $ri_2 = []@([1]@Q \oplus [0,0]@\varepsilon)$. The helper *fuse* performs this folding and fusing of bits onto surviving nodes, prepending a bit-sequence to a node's annotation without descending into its children. Both $deriv_{bc}$ and *simp* rely on it.

Unlike the direct construction, the bit-coded construction requires simplification to prevent unbounded growth of annotated expressions. The function *simp* discards dead branches and collapses finished subexpressions, while *fuse* preserves their bits on surviving nodes, as when P collapses into P' and then onto Q.

Once `ab` is exhausted, *mkEpsBC* walks ri_2 the way *mkEps* walks a plain expression, taking the leftmost nullable branch at each remaining choice: $[1]@Q$ is nullable, so it contributes its 1, and inside Q the ε branch is nullable, contributing a second 1, giving $bits = [1,1]$. $decode_{r_0}$ then replays these two bits against the original r_0. The first 1 selects $a+ab$'s right branch and the second selects $b+\varepsilon$'s right branch, giving the tree $(Right(a,b), Right\,())$.

This shows that the bit-coded construction reconstructs the parse tree in a single step, while the direct construction reverses residuals step by step.

5.2.1 Annotated Expressions

Building parse-tree information during the forward pass extends the derivative computation with bit-sequence annotations, propagated by the bit-threaded derivative $ri\backslash_b l$. An annotated expression ri extends r by associating each non-$\emptyset$ node with a bit-sequence bs recording the reconstruction choices made during the forward pass.

$$ri ::= \emptyset \mid (bs@\varepsilon) \mid (bs@x) \mid (bs@ri_1 \oplus ri_2) \mid (bs@ri_1\,ri_2) \mid (bs@ri^*). \tag{5.5}$$

In Rust. Figure 5.6 defines `ARegex` as the annotated counterpart of `Regex`. Every variant except `Phi` carries a leading `Vec<bool>` corresponding to (5.5), with one `bool` per bit. `Phi` has no annotation, since it cannot contribute to a successful parse.

```
pub enum ARegex {
    Phi,                        // no annotation slot
    Eps(Vec<bool>), Lit(Vec<bool>, char),
    Alt(Vec<bool>, Box<ARegex>, Box<ARegex>),
    Seq(Vec<bool>, Box<ARegex>, Box<ARegex>),
    Star(Vec<bool>, Box<ARegex>),
}
```

Figure 5.6: The annotated-expression type extends `Regex` with a leading bit-list.

All following operations act on this `ARegex`. Since derivative and simplification steps remove nodes while the bits attached to them must survive, each operation needs a way to transfer those bits onto the node that remains. The helper *fuse* provides this transfer.

5.2.2 Fusing Bit-Lists

fuse, defined in [Sulzmann and Lu, 2014, Fig. 5], transfers bits from a node that is about to be removed onto the node that survives in its place. It prepends bs to the top node's annotation bs' without descending into its children, so outer reconstruction choices precede those already recorded in the subtree.

$$\begin{aligned} \mathit{fuse}(bs, \emptyset) &= \emptyset \\ \mathit{fuse}(bs, (bs'@\varepsilon)) &= (bs +\!\!+ bs')@\varepsilon \\ \mathit{fuse}(bs, (bs'@x)) &= (bs +\!\!+ bs')@x \\ \mathit{fuse}(bs, (bs'@ri_1 \oplus ri_2)) &= (bs +\!\!+ bs')@(ri_1 \oplus ri_2) \\ \mathit{fuse}(bs, (bs'@ri_1\, ri_2)) &= (bs +\!\!+ bs')@(ri_1\, ri_2) \\ \mathit{fuse}(bs, (bs'@ri^*)) &= (bs +\!\!+ bs')@ri^*. \end{aligned} \tag{5.6}$$

In Rust. Figure 5.7 implements the six rules of (5.6), one per variant. The helper `combine` corresponds to the concatenation $+\!\!+$ and places `bs` before `p`, so the newly prepended bits precede those already recorded:

```
pub fn fuse(bs: &[bool], ri: ARegex) -> ARegex {
    if bs.is_empty() { return ri; }
    match ri {
        ARegex::Phi => ARegex::Phi,
        ARegex::Eps(p) => ARegex::Eps(combine(bs, p)),
        ARegex::Lit(p, c) => ARegex::Lit(combine(bs, p), c),
        ARegex::Alt(p, r1, r2) => ARegex::Alt(combine(bs, p), r1, r2),
        ARegex::Seq(p, r1, r2) => ARegex::Seq(combine(bs, p), r1, r2),
        ARegex::Star(p, r) => ARegex::Star(combine(bs, p), r),
    }
}

fn combine(bs: &[bool], p: Vec<bool>) -> Vec<bool> {
    let mut new = bs.to_vec();
    new.extend(p);
    new
}
```

Figure 5.7: `fuse`: prepend a bit-list to the annotation of the top node.

The `Phi` arm is the only case that returns without calling `combine`, because `Phi` represents the empty language and carries no annotation slot, so there is no bit-list for `fuse` to prepend to. Every other arm rebuilds the top node with a combined annotation and moves its two children, or single child in the star case, unchanged. Since `fuse` prepends `bs` only to the top node's annotation, the bits carried by its children are not touched, and the operation rewrites exactly one node regardless of how large the subtree below it is.

The function is therefore non-recursive, and no subtree needs to be copied. The implementation realizes this directly, since `ri` is consumed by value and its untouched children move into the rebuilt node without a clone. The helper `combine` allocates a fresh `Vec`, at a cost proportional to the combined length of the two bit-lists rather than to the size of the subtree below. This allocation happens on every call with a non-empty `bs`. An empty

`bs` is the one case that skips it, since combining an empty prefix with `p` gives `p` and the rebuilt node would be identical to the input.

5.2.3 Internalizing Expressions

The forward pass operates on annotated expressions, but the input is a plain expression, so r must first be converted into its annotated counterpart ri. The function *internalize*, following [Sulzmann and Lu, 2014, Fig. 5], performs this conversion recursively, and it is the only step that introduces new bits:

$$\begin{aligned}
\mathit{internalize}(\emptyset) &= \emptyset \\
\mathit{internalize}(\varepsilon) &= [\,]@\varepsilon \\
\mathit{internalize}(x) &= [\,]@x \\
\mathit{internalize}(r_1 + r_2) &= [\,]@\Big(\mathit{fuse}([0], \mathit{internalize}(r_1)) \oplus \mathit{fuse}([1], \mathit{internalize}(r_2))\Big) \\
\mathit{internalize}(r_1\, r_2) &= [\,]@\Big(\mathit{internalize}(r_1)\; \mathit{internalize}(r_2)\Big) \\
\mathit{internalize}(r^*) &= [\,]@\mathit{internalize}(r)^*.
\end{aligned} \tag{5.7}$$

The tags [0] and [1] on the two branches of $r_1 + r_2$ mark the left and right branch respectively, so the decoder can later recover which one was taken. Every other case introduces no new bit and uses only the empty list $[\,]$. The two calls to *fuse* in (5.7) are the only points in the construction where new bits are introduced.

In Rust. Figure 5.8 directly maps the constructors of `Regex` to their annotated counterparts. Each recursive call applies `internalize` to the corresponding subexpression, preserving the structure of the original expression in the annotated representation.

```
pub fn internalize(r: &Regex) -> ARegex {
    match r {
        Regex::Phi => ARegex::Phi,
        Regex::Eps => ARegex::Eps(vec![]),
        Regex::Lit(c) => ARegex::Lit(vec![], *c),
        Regex::Alt(r1, r2) => {
            let ri1 = fuse(&[false], internalize(r1));
            let ri2 = fuse(&[true], internalize(r2));
            ARegex::Alt(vec![], Box::new(ri1), Box::new(ri2))
        }
        Regex::Seq(r1, r2) => ARegex::Seq(vec![],
            Box::new(internalize(r1)), Box::new(internalize(r2))),
        Regex::Star(r1) => ARegex::Star(vec![], Box::new(internalize(r1))),
    }
}
```

Figure 5.8: `internalize` converts a `Regex` into an `ARegex`. Each alternation branch receives a tag recording its side, and the two `fuse` calls are the only points where new bits are introduced.

The `Alt` case calls `fuse` with a non-empty bit-list, tagging the left operand with `[false]` and the right with `[true]` before wrapping them in a `Alt` with an empty outer bit-list. Every later bit originates from one of these two tags and is propagated during derivation.

The `Phi`, `Eps`, `Lit`, `Seq`, and `Star` cases use an empty bit-list, since they introduce no disambiguation choice at this stage. Star iteration choices are instead determined during derivative computation.

5.2.4 Extracting Bits

mkEpsBC, from [Sulzmann and Lu, 2014, Fig. 5], mirrors *mkEps* but extracts the corresponding bit-list, using the same left preference at alternation and a closing 1 for a star. Unlike *mkEps*, it constructs no parse-tree nodes. Each rule prepends the current node's bits to those returned recursively.

$$\begin{aligned}
mkEpsBC_{(bs@\varepsilon)} &= bs \\
mkEpsBC_{(bs@ri^*)} &= bs +\!\!+ [1] \\
mkEpsBC_{(bs@ri_1\, ri_2)} &= bs +\!\!+ mkEpsBC_{ri_1} +\!\!+ mkEpsBC_{ri_2} \\
mkEpsBC_{(bs@ri_1 \oplus ri_2)} &= bs +\!\!+ \begin{cases} mkEpsBC_{ri_1} & \varepsilon \in L(ri_1) \\ mkEpsBC_{ri_2} & \text{otherwise.} \end{cases}
\end{aligned} \tag{5.8}$$

The rules cover the four expression forms that can be nullable, as in *mkEps*. No rules are needed for $\emptyset$ or literals, since neither can match the empty word, and *mkEpsBC* is called only on nullable expressions, so both are handled as preconditions.

In Rust. Figure 5.9 mirrors `mk_eps` (Figure 5.2), with one difference. Where `mk_eps` builds a `ParseTree`, `mk_eps_bc` concatenates bit-lists. The `Star` arm also does not recurse, since a star matching the empty word uses zero iterations, so it only appends the closing `true` and never examines the child expression:

```
pub fn mk_eps_bc(ri: &ARegex) -> Vec<bool> {
    match ri {
        ARegex::Phi => panic!("mk_eps_bc called on Phi"),
        ARegex::Lit(_, c) => panic!("mk_eps_bc called on Lit('{}')", c),
        ARegex::Eps(bs) => bs.clone(),
        ARegex::Alt(bs, r1, r2) => {
            let mut result = bs.clone();
            if nullable_bc(r1) {
                result.extend(mk_eps_bc(r1));
            } else {
                result.extend(mk_eps_bc(r2));
            }
            result
        }
        ARegex::Seq(bs, r1, r2) => {
            let mut result = bs.clone();
            result.extend(mk_eps_bc(r1));
            result.extend(mk_eps_bc(r2));
            result
        }
        ARegex::Star(bs, _) => {
            let mut result = bs.clone();
            result.push(true);
            result
        }
    }
}
```

Figure 5.9: `mk_eps_bc` implements (5.8). `Phi` and `Lit` panic, since neither can be nullable.

5.2.5 Annotated Derivatives

$ri\backslash_b l$ has the same case structure as the plain derivative (3.6), but extended with bits threaded through the rules. Each rule corresponds to its unannotated counterpart:

$$
\begin{aligned}
\emptyset\backslash_b x &= \emptyset && (\mathsf{B}_\emptyset)\\
(bs@\varepsilon)\backslash_b x &= \emptyset && (\mathsf{B}_\varepsilon)\\
(bs@c)\backslash_b x &= \begin{cases} bs@\varepsilon & c = x \\ \emptyset & \text{otherwise} \end{cases} && (\mathsf{B}_{\text{lit}})\\
(bs@ri_1 \oplus ri_2)\backslash_b x &= (bs@(ri_1\backslash_b x) \oplus (ri_2\backslash_b x)) && (\mathsf{B}_+)\\
(bs@ri_1\ ri_2)\backslash_b x &= (bs@(ri_1\backslash_b x)\ ri_2) && (\mathsf{B}_{\text{seq}}),\ \varepsilon \notin L(ri_1)\\
(bs@ri_1\ ri_2)\backslash_b x &= bs@\Big(([\,]@(ri_1\backslash_b x)\ ri_2) \oplus \mathit{fuse}(\mathit{mkEpsBC}_{ri_1}, ri_2\backslash_b x)\Big) && (\mathsf{B}^n_{\text{seq}}),\ \varepsilon \in L(ri_1)\\
(bs@ri^*)\backslash_b x &= (bs@\mathit{fuse}([0], ri\backslash_b x)\ ([\,]@ri^*)) && (\mathsf{B}_*).
\end{aligned}
\tag{5.9}
$$

The rules B_+, B_{seq}, and B_* preserve the outer bs, since derivation changes the residual but not the choices already recorded above it.

The nullable sequence case $\mathsf{B}^n_{\text{seq}}$ is the exception. Since ri_1 is nullable, x may be matched

by either factor, so the rule introduces a new alternation. Its left branch continues into ri_1, while its right branch recurses into ri_2, with the empty match's bits extracted by *mkEpsBC* and fused onto ri_2's derivative, so the choice inside ri_1 survives. This performs during the forward pass what $\mathsf{I}^R_{\text{seq}}$ performs during injection. The outer *bs* attaches to the new alternation, since the alternation as a whole is what $ri_1\ ri_2$ derived into.

In Rust. Figure 5.10 implements (5.9), following the case structure of plain `deriv` with bits threaded through the constructors. The base cases `Phi`, `Eps`, and `Lit` match plain `deriv`, except that `Lit` carries its bit-list onto the resulting `Eps`.

The `Alt` arm derives both operands and rebuilds the alternation with the outer `bs` on the wrapper, not pushed into either branch. This matches B_+, where the recorded choice belongs to the alternation itself and derivation does not change it, so the bit stays on the outer constructor until the decoder reads it.

The `Seq` arm reflects B_{seq} and $\mathsf{B}^n_{\text{seq}}$. In the non-nullable case, `r1` must consume the character, so only its derivative is needed and `r2` moves directly into the rebuilt sequence. In the nullable case, both alternatives are possible. The bits of `r1`'s empty match are extracted by `mk_eps_bc` *before* `r1` is consumed by `deriv_bc`, since after that call the original `r1` is gone. Those bits are then fused onto the derivative of `r2`, so the choice made inside `r1` reaches the surviving branch. The left branch is wrapped in a fresh `Seq` with an empty bit-list, since the outer `bs` moves to the new `Alt` wrapper, matching $\mathsf{B}^n_{\text{seq}}$. Two clones are therefore needed. `r2.clone()` provides one copy for the left branch and one for its own derivative, and `nullable_bc` borrows `r1` before `deriv_bc` consumes it.

The `Star` arm follows the same pattern as the nullable case of `Seq`. The body is needed both for its derivative and for the reconstructed star, so `r.clone()` provides the second copy, and the reconstructed star is wrapped in a fresh `Star` with an empty bit-list for the same reason as the left branch above. The leading `[false]` prepended to the derivative marks one more iteration, matching the outermost bit in B_*

```
pub fn deriv_bc(ri: ARegex, l: char) -> ARegex {
    match ri {
        // ...
        ARegex::Alt(bs, r1, r2) => {
            let d1 = deriv_bc(*r1, l);
            let d2 = deriv_bc(*r2, l);
            ARegex::Alt(bs, Box::new(d1), Box::new(d2))
        }
        ARegex::Seq(bs, r1, r2) => {
            if nullable_bc(&r1) {
                let eps_bits = mk_eps_bc(&r1);
                let d1 = deriv_bc(*r1, l);
                let d2 = deriv_bc(*r2.clone(), l);
                let left_branch = ARegex::Seq(vec![], Box::new(d1), r2);
                let right_branch = fuse(&eps_bits, d2);
                ARegex::Alt(bs, Box::new(left_branch), Box::new(right_branch))
            } else {
                let d1 = deriv_bc(*r1, l);
                ARegex::Seq(bs, Box::new(d1), r2)
            }
        }
        ARegex::Star(bs, r) => {
            let d = deriv_bc(*r.clone(), l);
            let fused = fuse(&[false], d);
            let new_star = ARegex::Star(vec![], r);
            ARegex::Seq(bs, Box::new(fused), Box::new(new_star))
        }
    }
}
```

Figure 5.10: Annotated derivative `deriv_bc`, implementing $\mathsf{B}_{\emptyset}$, B_{ε}, B_{lit}, B_{+}, $\mathsf{B}_{\text{seq}}/\mathsf{B}^{n}_{\text{seq}}$, B_{*}.

5.2.6 Decoding Bits

$decode_r$ reconstructs a `ParseTree` from the bit sequence produced by $mkEpsBC$. It walks the original, unannotated r and uses one bit per alternation to choose a branch:

$$decode_r(bs) = v \quad \text{if } decodeInner_r(bs) = (v, [\,])$$

$$\begin{aligned}
decodeInner_{\varepsilon}(bs) &= (Empty,\ bs)\\
decodeInner_{x}(bs) &= (Char(x),\ bs)\\
decodeInner_{r_1+r_2}(0{:}bs) &= (Left(v),\ bs_1) \quad \text{if } decodeInner_{r_1}(bs) = (v, bs_1)\\
decodeInner_{r_1+r_2}(1{:}bs) &= (Right(v),\ bs_1) \quad \text{if } decodeInner_{r_2}(bs) = (v, bs_1)\\
decodeInner_{r_1 r_2}(bs) &= (Pair(v_1, v_2),\ bs_2)\\
&\qquad \text{if } decodeInner_{r_1}(bs) = (v_1, bs_1) \text{ and } decodeInner_{r_2}(bs_1) = (v_2, bs_2)\\
decodeInner_{r^*}(1{:}bs) &= (Star([\,]),\ bs)\\
decodeInner_{r^*}(0{:}bs) &= (Star(v{:}vs),\ bs_2)\\
&\qquad \text{if } decodeInner_{r}(bs) = (v, bs_1) \text{ and } decodeInner_{r^*}(bs_1) = (Star(vs), bs_2)
\end{aligned} \tag{5.10}$$

The annotated expression has been reduced to a bare bit sequence, so $decode_r$ walks the original r and uses the bits only to choose between alternation branches. Its worker *decodeInner* returns both the tree for the current subexpression and the unconsumed bits, so the recursion can continue at the correct position.

In Rust. Figure 5.11 shows `decode` and the compound arms of its worker `decode_inner`. The worker returns a pair, `(ParseTree, &[bool])`, holding the tree for the current subexpression together with whatever bits remain, which is what lets the recursion continue at the correct position in the bit-string. Rust's slice patterns make the leading-bit test direct, since `[false, rest @ ..]` and `[true, rest @ ..]` both check the bit and name the remainder in one pattern.

```
pub fn decode(r: &Regex, bs: &[bool]) -> ParseTree {
    let (v, rest) = decode_inner(r, bs);
    assert!(rest.is_empty(),
        "decode: {} leftover bit(s) after decoding {:?}", rest.len(), r);
    v
}

fn decode_inner<'a>(r: &Regex, bs: &'a [bool]) -> (ParseTree, &'a [bool]) {
    match r {
        // ...
        Regex::Alt(r1, r2) => match bs {
            [false, rest @ ..] => {
                let (v, remaining) = decode_inner(r1, rest);
                (ParseTree::Left(Box::new(v)), remaining)
            }
            [true, rest @ ..] => {
                let (v, remaining) = decode_inner(r2, rest);
                (ParseTree::Right(Box::new(v)), remaining)
            }
            // ...
        },
        Regex::Seq(r1, r2) => {
            let (v1, after_r1) = decode_inner(r1, bs);
            let (v2, after_r2) = decode_inner(r2, after_r1);
            (ParseTree::Pair(Box::new(v1), Box::new(v2)), after_r2)
        }
        Regex::Star(r1) => {
            let mut items = Vec::new();
            let mut remaining = bs;
            loop {
                match remaining {
                    [true, rest @ ..] => { remaining = rest; break; }
                    [false, rest @ ..] => {
                        let (v, after) = decode_inner(r1, rest);
                        items.push(v);
                        remaining = after;
                    }
                    // ...
                }
            }
            (ParseTree::Star(items), remaining)
        }
    }
}
```

Figure 5.11: `decode` and the `Alt`, `Seq`, and `Star` arms of `decode_inner`.

`decode` calls the worker and asserts that no bits remain, which is exactly the side condition that $decode_r$ be defined only when the worker consumes the entire bit-string. The `Alt` arm tests the leading bit with a slice pattern and recurses into the branch it names, returning the remaining bits unchanged, since only the leading bit belongs to the alternation.

The `Seq` arm shows why the worker returns a pair. It recurses into the first factor, receives both a tree and the bits the first factor left unconsumed, and passes those bits to the second factor. The second factor then returns its own tree and the bits that follow it, which are the bits for any enclosing expression. Returning only the tree would leave the second recursion with no way to know where to start.

The `Star` arm loops, since the number of iterations is not known in advance and is itself encoded in the bits. Each leading `false` corresponds to one iteration, so the arm recurses into the body and then continues the loop from whatever bits the body left behind. The loop ends at the leading `true` that `mk_eps_bc` appended. The `Eps`, `Lit`, and `Phi` arms are elided, since the first two consume no bits and the third is unreachable.

5.2.7 Parser

Figure 5.12 implements the bit-coded driver. It internalizes r, applies `deriv_bc` and `simp` to each input character, checks nullability and, if successful, calls `mk_eps_bc` and `decode`.

```
pub fn parse_bitcoded_loop(input: &str, r: &Regex) -> Option<ParseTree> {
    let mut ri = internalize(r);
    for l in input.chars() { ri = simp(deriv_bc(ri, l)); }
    if !nullable_bc(&ri) { return None; }
    Some(decode(r, &mk_eps_bc(&ri)))
}
```

Figure 5.12: `parse_bitcoded_loop`: the bit-coded parser implementation.

Unlike the direct implementation, which has a growing `Vec<Regex>` for backward reconstruction, the bit-coded implementation needs only the current `ri`. The recursive entry point has the same three-step computation but differs only in how intermediate expressions are stored.

5.2.8 Worked Example: Ambigious Expression

Consider the same input `ab` and regular expression $r_0 = (a \mid ab)(b \mid \varepsilon)$.

Internalize. Before consuming input, *internalize* tags each alternative with [0] for the left branch and [1] for the right branch:

$$\mathit{internalize}(r_0) = ri_0 = [\,]@\Big([\,]@([0]@a \oplus [1]@(a\,b)) \cdot [\,]@([0]@b \oplus [1]@\varepsilon)\Big). \tag{5.11}$$

Forward pass. The two input symbols are consumed by successive bit-threaded derivatives with *simp* applied after each step. The first derivative uses B_{seq}. After consuming `a`, the first factor becomes nullable, so the second derivative uses $\mathsf{B}^{n}_{\text{seq}}$, which propagates bits from the empty match of the first factor into the derivative of the second:

$$\begin{aligned} ri_0 &\xrightarrow[\mathsf{B}_{\text{seq}}]{a} ri_1 = [\,]@\Big([\,]@([0]@\varepsilon \oplus [1]@b) \cdot [\,]@([0]@b \oplus [1]@\varepsilon)\Big) \\ ri_1 &\xrightarrow[\mathsf{B}^{n}_{\text{seq}}]{b} ri_2 = [\,]@\Big([1]@([0]@b \oplus [1]@\varepsilon) \oplus [0,0]@\varepsilon\Big). \end{aligned} \tag{5.12}$$

The [1] records the right alternative ab of the first factor. The $[0, 0]$ on the second branch results from fusing bits during the derivative. Unlike the direct construction, where *inj* recovers this information later, the bit-coded method propagates it during the forward pass.

Extracting and decoding the bits. Both disjuncts of ri_2 are nullable, so *mkEpsBC* applies the left-preference rule and selects the left disjunct. Inside, b is not nullable while ε is, so EB_+^R contributes its [1] tag:

$$mkEpsBC(ri_2) \xrightarrow{\mathsf{EB}_+^L} mkEpsBC\Big([1]@([0]@b \oplus [1]@\varepsilon)\Big) \xrightarrow{\mathsf{EB}_+^R} [1, 1]. \tag{5.13}$$

Unlike *mkEps*, which constructs a partial parse tree, *mkEpsBC* extracts only the committed choices as a bit-string. The decoder then interprets $[1, 1]$ against the original r_0: the first bit selects the right branch of $a + ab$ (producing ab), and the second selects the right branch of $b + \varepsilon$:

$$decode_{r_0}([1, 1]) = \Big(Right(a, b), Right(\,)\Big) = v_0. \tag{5.14}$$

Flattening validates the result:

$$flatten(v_0) = a \cdot b \cdot \varepsilon = \mathtt{ab}. \tag{5.15}$$

The bit-coded construction produces the same parse tree as the direct construction but stores only the final annotated expression ri_2, not the intermediate expressions r_0 and r_1.

6 Partial-Derivative-Based Parser

Chapter 5 presented two derivative-based methods for constructing POSIX parse trees. Both keep the structure of the original expression throughout the forward pass, defer disambiguation to a final *mkEps* step and, in the standard variant, reconstruct the tree in a backward pass. This chapter presents the partial-derivative-based alternative of Sulzmann and Lu [Sulzmann and Lu, 2012]. Instead of one residual per step, the partial derivative produces a set of states, each pairing a residual expression with the reconstruction information for one path through the original expression. The parser maintains this set as an NFA state set, transitions every state on the next input character, and selects the first accepting state at the end.

The defining difference from the derivative-based parsers is the disambiguation policy. Because partial derivatives break apart the structure of the original expression—alternations are split into separate states, and concatenations are expanded without preserving the outer shape—the NFA traversal yields the greedy leftmost parse tree rather than the POSIX one. On the running ambiguous example from Chapter 5, $r_0 = (a+ab)(b+\varepsilon)$ with input $\mathtt{ab}$, the derivative-based parsers return $(\mathit{Right}(a, b), \mathit{Right}())$, whereas the partial-derivative parsers return $(\mathit{Left}(a), \mathit{Left}(b))$.

As in Chapter 5, two variants are presented. The standard partial-derivative parser attaches an injection closure to each configuration, composing reconstruction steps during the forward pass. The bit-coded partial-derivative parser replaces these closures with bit-sequences carried alongside the residuals. Both compute the same greedy parse tree; they differ only in how the reconstruction information is represented.

6.1 The Standard Partial-Derivative Parser

Consider again the ambiguous example $r_0 = (a + ab)(b + \varepsilon)$ with input $w = \mathtt{ab}$. The partial derivative of r_0 with respect to $\mathtt{a}$ produces two states, one for each way $\mathtt{a}$ can be consumed:

$$\begin{aligned} \mathit{pderiv}(r_0, \mathtt{a}) = \big\{ &(\, b + \varepsilon, \ \lambda(_, v_2).\ (\mathit{Left}\, a, \ v_2) \ ), \\ &(\, b\,(b + \varepsilon), \ \lambda((v_1, v_2), v_2').\ (\mathit{Right}(a, v_2), \ v_2') \ ) \big\}. \end{aligned} \tag{6.1}$$

The first state comes from the left branch a of the first factor: the character is matched by a, and the injection places it in the left injection of the alternation. The second state comes from the right branch ab: the character is matched by the leading a of ab, and the injection places it in the right injection, leaving the residual b to be matched later. Two notational conventions appear in the closures. The underscore in the first injection marks

a binder that is never used: the residual is ε, whose only tree carries no information, so the closure discards its argument. The leading ε inside the second state's residual has already been removed by the smart constructor, since $\varepsilon\, b$ and b describe the same language; the closure's pair-binder reflects the shape the residual would have had before the simplification.

Transitioning both states on the second character `b` gives a new state set:

$$\begin{aligned} \mathit{pderiv}(r_0, \mathtt{a}, \mathtt{b}) = \big\{ &(\ \varepsilon,\ \lambda_.\ (\mathit{Left}\, a,\ \mathit{Left}\, b)\ ), \\ &(\ b + \varepsilon,\ \lambda(_, v_2).\ (\mathit{Right}(a, b),\ v_2)\ ) \big\}. \end{aligned} \tag{6.2}$$

The first configuration has residual ε and is accepting; the second has residual $b + \varepsilon$, which is also accepting. Selection walks the state set in priority order and returns the first accepting configuration. Applying its injection to $\mathit{mkEps}_\varepsilon = ()$ yields $(\mathit{Left}\, a, \mathit{Left}\, b)$, the greedy parse tree.

Figure 6.1 shows the two transitions and the selection on the running example. Each arrow is labelled by the function that performs the step: $\mathit{pderiv}(-, \mathtt{a})$ and $\mathit{pderiv}(-, \mathtt{b})$ advance the state set by one character, *select* picks the first accepting configuration.

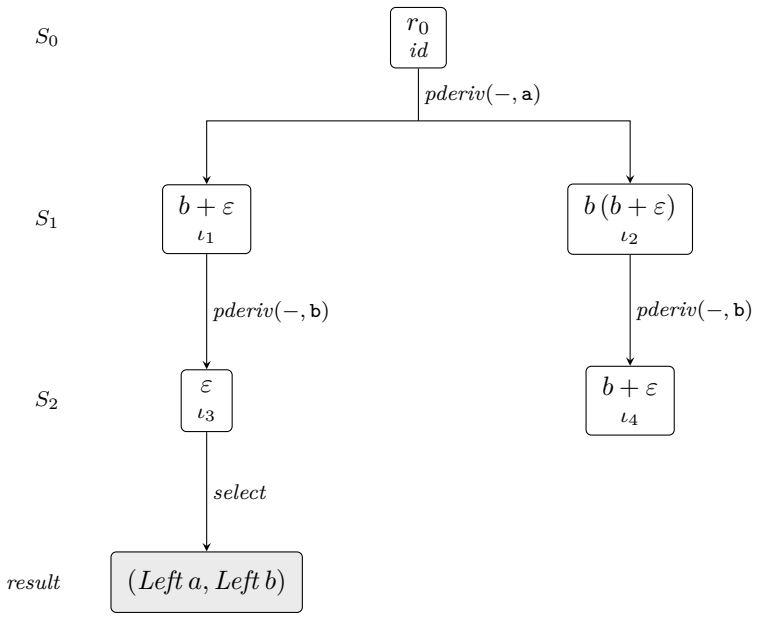


Figure 6.1: The standard partial-derivative parser on the ambiguous example $r_0 = (a + ab)(b + \varepsilon)$ with input `ab`. Each box is a configuration: a residual expression and an injection ι_i. The initial state set S_0 contains (r_0, id). Applying $\mathit{pderiv}(-, \mathtt{a})$ advances every configuration by the character `a`, producing S_1 with two configurations. Applying $\mathit{pderiv}(-, \mathtt{b})$ advances S_1 to S_2, in which the first configuration has residual ε and is accepting. Selecting that configuration's injection and applying it to $\mathit{mkEps}_\varepsilon = ()$ yields the greedy parse tree $(\mathit{Left}\, a, \mathit{Left}\, b)$.

The order of configurations in the state set encodes the greedy preference. When a state produces several successors, they are appended in the order the rules generate them, with the left alternative first. When two configurations share a residual, *nub* keeps the first, discarding the later one and its injection. Both mechanisms favour the leftmost path through the expression.

6.1.1 Partial Derivatives with Injections

A partial derivative of an expression r with respect to a character x is not a single expression but a set of pairs. Each pair consists of a residual expression r' whose language is one component of the left quotient, together with an injection closure *inj* that maps a parse tree for r' back to a parse tree for r. The function *pderiv*, following [Sulzmann and Lu, 2012, Fig. 10], is defined recursively:

$$
\begin{aligned}
\mathit{pderiv}(\phi, x) &= \emptyset \\
\mathit{pderiv}(\varepsilon, x) &= \emptyset \\
\mathit{pderiv}(c, x) &= \begin{cases} \{(\varepsilon,\ \lambda_.\, c)\} & c = x \\ \emptyset & \text{otherwise} \end{cases} \\
\mathit{pderiv}(r_1 + r_2, x) &= \{\, (r_1',\ \lambda v.\, \mathit{Left}\,(\mathit{inj}_1\, v)) \mid (r_1', \mathit{inj}_1) \in \mathit{pderiv}(r_1, x) \,\} \\
&\quad \cup \{\, (r_2',\ \lambda v.\, \mathit{Right}\,(\mathit{inj}_2\, v)) \mid (r_2', \mathit{inj}_2) \in \mathit{pderiv}(r_2, x) \,\} \\
\mathit{pderiv}(r_1\, r_2, x) &= \{\, (r_1'\, r_2,\ \lambda(v_1, v_2).\, (\mathit{inj}_1\, v_1,\ v_2)) \mid (r_1', \mathit{inj}_1) \in \mathit{pderiv}(r_1, x) \,\} \\
&\quad \cup \begin{cases} \{\, (r_2',\ \lambda v.\, (\mathit{mkEps}_{r_1},\ \mathit{inj}_2\, v)) \mid (r_2', \mathit{inj}_2) \in \mathit{pderiv}(r_2, x) \,\} & n(r_1) \\ \emptyset & \text{otherwise} \end{cases} \\
\mathit{pderiv}(r^*, x) &= \{\, (r'\, r^*,\ \lambda(v, \mathit{Star}(vs)).\, \mathit{Star}([\mathit{inj}\, v] +\!\!+ vs)) \mid (r', \mathit{inj}) \in \mathit{pderiv}(r, x) \,\}.
\end{aligned}
\tag{6.3}
$$

After expansion, the list is deduplicated by residual, keeping the first occurrence. This is the step that breaks apart the original expression: where the derivative of $r_1 + r_2$ produces a single alternation, the partial derivative produces separate states for each branch, and the outer alternation disappears from the residual.

The injection in each pair records the path taken through the expression. Rule (P_+) wraps the recursive injection in a *Left* or *Right* node, reinserting the alternation choice. Rule (P_{seq}) for a non-nullable first factor passes the outer pair through unchanged, with the recursive injection applied to the first component only. When the first factor is nullable, the rule generates two kinds of states: those that continue inside the first factor, and those that skip the first factor entirely, using mkEps_{r_1} to synthesise its empty match. Rule (P_*) reconstructs the current iteration by applying the recursive injection and prepending it to the remaining iterations.

No rules are needed for ϕ or ε beyond their empty results, since neither can consume a character.

In Rust. The recursive definition (6.3) is implemented directly in Rust. The type *Inj* is a reference-counted closure, which allows a single parent injection to be shared when a transition fans out into several states.

```
pub(crate) type Inj = Rc<dyn Fn(ParseTree) -> ParseTree>;

pub(crate) fn pderiv_tree(r: &Regex, x: char) -> Vec<(Regex, Inj)> {
    match r {
        Regex::Eps | Regex::Phi => vec![],

        Regex::Lit(c) => {
            if *c == x {
                vec![(Regex::Eps,
                      Rc::new(move |_| ParseTree::Char(x))) as Inj]
            } else { vec![] }
        }

        Regex::Alt(r1, r2) => {
            let mut out: Vec<(Regex, Inj)> = pderiv_tree(r1, x)
                .into_iter()
                .map(|(r1_, inj1)| {
                    let f: Inj = Rc::new(move |v| {
                        ParseTree::Left(Box::new(inj1(v)))
                    });
                    (r1_, f)
                })
                .collect();
            out.extend(pderiv_tree(r2, x).into_iter().map(|(r2_, inj2)| {
                let f: Inj = Rc::new(move |v| {
                    ParseTree::Right(Box::new(inj2(v)))
                });
                (r2_, f)
            }));
            nub_tree(out)
        }

        Regex::Seq(r1, r2) => {
            if nullable(r1) {
                let mut out: Vec<(Regex, Inj)> = pderiv_tree(r1, x)
                    .into_iter()
                    .map(|(r1_, inj1)| {
                        let r2c = (**r2).clone();
                        let f: Inj = Rc::new(move |v| match v {
                            ParseTree::Pair(v1, v2) =>
                                ParseTree::Pair(Box::new(inj1(*v1)), v2),
                            _ => panic!("pderiv_tree/Seq: want Pair"),
                        });
                        (Regex::seq(r1_, r2c), f)
                    })
                    .collect();

                let v1_eps = mk_eps(r1);
                out.extend(pderiv_tree(r2, x).into_iter().map(|(r2_, inj2)| {
                    let v1c = v1_eps.clone();
                    let f: Inj = Rc::new(move |v2| {
                        ParseTree::Pair(Box::new(v1c.clone()),
                                        Box::new(inj2(v2)))
                    });
                    (r2_, f)
                }));
                nub_tree(out)
```

The `Alt` arm is where the partial derivative visibly breaks apart the expression: each branch produces its own states, tagged by *Left* or *Right* in the injection. The `Seq` arm's nullable case is the richest: the first factor is consumed by `deriv`-style recursion, and the first factor's empty match is computed once with `mk_eps` and captured by each state from the second factor.

The `Star` arm follows the same pattern, with the injection inserting the reconstructed iteration at the front of the star's list. Note that the residual is built with the plain `Regex::seq` constructor rather than a smart constructor. Keeping the unsimplified shape matters because the injection pattern-matches on the tree's structure; a simplified residual would lose the outer sequence node the injection expects.

6.1.2 State Sets and Selection

The parser maintains a state set of (residual, injection) configurations. For each input character, every configuration in the state set is transitioned with `pderiv_tree`, and the resulting configurations are collected into a new state set. A configuration's parent injection is composed with the transition injection, so that at the end the state set's injections map residuals all the way back to the original expression.

The composition is direct. When a parent configuration (r, inj_p) transitions to (r', inj_s), the new configuration's injection is $\lambda v.\, inj_p(inj_s(v))$.

```
pub(super) fn step_frontier(
    frontier: Vec<(Regex, Inj)>, x: char
) -> Vec<(Regex, Inj)> {
    let mut out = Vec::new();
    for (r, inj_parent) in frontier {
        for (r_next, inj_step) in pderiv_tree(&r, x) {
            let parent = inj_parent.clone();
            let combined: Inj = Rc::new(move |v| parent(inj_step(v)));
            out.push((r_next, combined));
        }
    }
    out
}
```

Figure 6.3: `step_frontier`: one transition of the NFA state set on a single character. The function name retains the older "frontier" terminology; it computes the successor state set $\delta(S, x)$.

After the input is exhausted, selection walks the state set in priority order and returns the tree for the first accepting configuration:

```
pub(super) fn select(frontier: &[(Regex, Inj)]) -> Option<ParseTree> {
    for (r, inj) in frontier {
        if nullable(r) {
            return Some(inj(mk_eps(r)));
        }
    }
    None
}
```

Figure 6.4: `select`: return the tree for the first accepting configuration in the state set.

Because the state set lists configurations in the order the rules generated them, and because `nub_tree` keeps the first of each duplicate residual, selection favours the configuration that committed earliest to the leftmost alternative. This is the greedy leftmost policy.

6.1.3 Parser

The parser composes `step_frontier` over the input and then calls `select`.

```
pub fn parse_pderiv_std(input: &str, r: &Regex) -> Option<ParseTree> {
    let mut frontier: Vec<(Regex, Inj)> =
        vec![(r.clone(), identity_inj())];
    for c in input.chars() {
        frontier = step_frontier(frontier, c);
        if frontier.is_empty() {
            return None;
        }
    }
    select(&frontier)
}
```

Figure 6.5: The standard partial-derivative parser.

The initial state set has a single configuration: the original expression with the identity injection. The early return on an empty state set corresponds to the case where no configuration can consume the current character, so the input cannot be matched. The final `select` returns `None` if no configuration is accepting, and otherwise returns the tree for the first accepting one.

6.1.4 Worked Example: Ambiguous Expression

Recall the ambiguous example $r_0 = (a + ab)(b + \varepsilon)$ on input `ab`.

Forward pass. The initial state set holds a single configuration:

$$S_0 = \big[(r_0,\ id)\big].$$

Transitioning with respect to `a`:

$$pderiv(r_0, \mathtt{a}) = pderiv\Big(Alt(a, ab), \mathtt{a}\Big) \cdot Alt(b, \varepsilon).$$

Inside the first factor:

$$\begin{aligned} \mathit{pderiv}(a, \mathtt{a}) &= \Big[(\varepsilon,\ \lambda_.\, a)\Big] \\ \mathit{pderiv}(ab, \mathtt{a}) &= \Big[(\mathit{Seq}(\varepsilon, b),\ \lambda(v_1, v_2).\, (a, v_2))\Big]. \end{aligned}$$

After the alternation rule wraps each state with *Left* or *Right*:

$$\begin{aligned} \mathit{pderiv}(a + ab, \mathtt{a}) = \Big[&(\varepsilon,\ \lambda v.\, \mathit{Left}\,(\lambda_.\, a)(v)), \\ &(\mathit{Seq}(\varepsilon, b),\ \lambda(v_1, v_2).\, \mathit{Right}\,(a, v_2))\Big]. \end{aligned}$$

The outer sequence rule combines this with the untouched second factor:

$$\begin{aligned} S_1 = \Big[&(\mathit{Seq}(\varepsilon, b + \varepsilon),\ \lambda(v_1, v_2).\, (\mathit{Left}\, a,\ v_2)), \\ &(\mathit{Seq}(\mathit{Seq}(\varepsilon, b), b + \varepsilon),\ \lambda((v_1, v_2), v_2').\, (\mathit{Right}(a, v_2),\ v_2'))\Big]. \end{aligned}$$

Transitioning with respect to `b` steps both configurations. For the first, the left factor ε is nullable, so the sequence rule produces the state that jumps directly into the second factor:

$$\mathit{pderiv}(b + \varepsilon, \mathtt{b}) = \Big[(\varepsilon,\ \lambda v.\, \mathit{Left}\, b)\Big].$$

The second factor's injection inserts the character at the leftmost position of the pair:

$$\mathit{inj}_1(\mathit{Left}\, b) = (\mathit{Left}\, a,\ \mathit{Left}\, b).$$

For the second configuration, the leading $\mathit{Seq}(\varepsilon, b)$ has a non-nullable *Seq* structure and the derivative continues inside b:

$$\mathit{pderiv}(\mathit{Seq}(\varepsilon, b), \mathtt{b}) = \Big[(\varepsilon,\ \lambda_.\, ((), b))\Big],$$

and the outer sequence rule rebuilds the pair, giving the second configuration's residual $b + \varepsilon$. The two resulting state-set entries are:

$$\begin{aligned} S_2 = \Big[&(\varepsilon,\ \lambda_.\, (\mathit{Left}\, a,\ \mathit{Left}\, b)), \\ &(b + \varepsilon,\ \lambda(_, v_2).\, (\mathit{Right}(a, b),\ v_2))\Big]. \end{aligned}$$

Selection. The first configuration's residual is ε, which is nullable, and selection returns immediately:

$$\mathit{select}(S_2) = (\lambda_.\, (\mathit{Left}\, a, \mathit{Left}\, b))(\mathit{mkEps}_\varepsilon) = (\mathit{Left}\, a, \mathit{Left}\, b).$$

Flattening confirms the result:

$$\mathit{flatten}\Big((\mathit{Left}\, a, \mathit{Left}\, b)\Big) = \mathtt{ab}.$$

This is the greedy parse tree. The derivative-based parser from Chapter 5 returns $(\mathit{Right}(a, b), \mathit{Right}())$ for the same input, the POSIX tree. The two trees agree on which word they spell out, but differ on how the input is decomposed: the greedy tree commits to the shorter first alternative a in the first factor, while the POSIX tree commits to the longer alternative ab.

6.2 The Bit-Coded Partial-Derivative Parser

The standard partial-derivative parser carries a closure per configuration, which accumulates the reconstruction path during the forward pass. The bit-coded variant replaces closures with bit-sequences: each configuration's annotation records the choices made so far, and decoding the final bit-sequence against the original expression yields the parse tree.

Transitioning $w = \texttt{ab}$ from $r_0 = (a + ab)(b + \varepsilon)$ gives:

$$\begin{aligned} S_0 &= \big[(r_0,\ [\,])\big] \\ &\xrightarrow{\texttt{a}} S_1 = \big[(b + \varepsilon,\ [\mathit{false}]),\ (b\,(b + \varepsilon),\ [\mathit{true}])\big] \\ &\xrightarrow{\texttt{b}} S_2 = \big[(\varepsilon,\ [\mathit{false}, \mathit{false}]),\ (b + \varepsilon,\ [\mathit{true}])\big]. \end{aligned} \tag{6.4}$$

The first configuration of S_1 records that the character `a` was matched by the left branch of the first factor's alternation. The second configuration records that it was matched by the right branch, leaving the residual b to be consumed later. After the second character, only the first configuration is accepting, and its bits $[\mathit{false}, \mathit{false}]$ decode to $(\mathit{Left}\, a, \mathit{Left}\, b)$.

Figure 6.6 shows this transition together with the selection and decode steps.

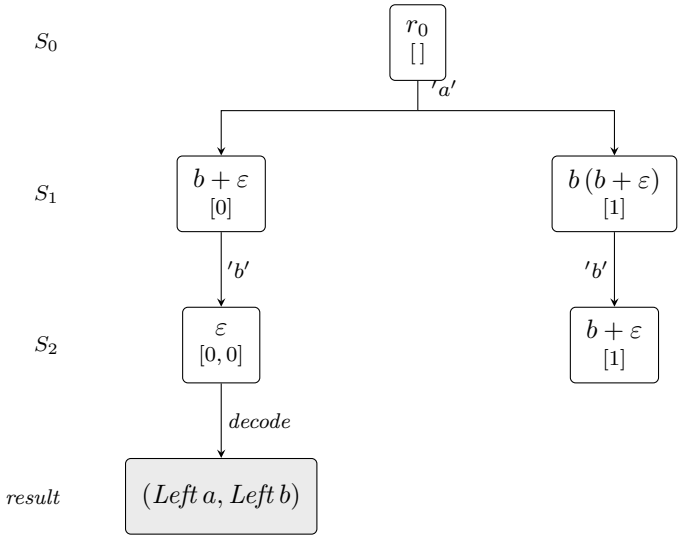


Figure 6.6: Overview of the bit-coded partial-derivative parser on the ambiguous example $r_0 = (a + ab)(b + \varepsilon)$ with input `ab`. Each box is a configuration: a residual expression and a bit-sequence, written 0 for *false* and 1 for *true*. Selection takes the first accepting configuration in S_2 and appends *mkEpsBC* of its residual, yielding the bit-string $[0, 0]$, which decode_{r_0} reads against the original expression.

The bit-coded parser keeps the same state-set structure as the standard one. Only the configuration's annotation changes, from a closure to a sequence of bits, and the final step changes from applying an injection to calling *decode*.

6.2.1 Bit-Coded Partial Derivatives

The bit-coded partial derivative $\mathit{pderivBC}(r, x)$ returns a list of (residual, bit-sequence) pairs. It follows the same case structure as (6.3), with one difference: where the standard

version builds an injection closure, the bit-coded version prepends a fixed bit-sequence to the configuration's existing bits.

The bit conventions are inherited from Chapter 5.2. Each alternation contributes a leading *false* for the left branch or a leading *true* for the right branch. Each star iteration contributes a leading *false*, and each star termination contributes a *true*. Concatenation contributes no bits of its own.

$$
\begin{aligned}
\mathit{pderivBC}(\phi, x) &= \emptyset \\
\mathit{pderivBC}(\varepsilon, x) &= \emptyset \\
\mathit{pderivBC}(c, x) &= \begin{cases} \{(\varepsilon,\, [\,])\} & c = x \\ \emptyset & \text{otherwise} \end{cases} \\
\mathit{pderivBC}(r_1 + r_2, x) &= \{\, (r_1',\, [\mathit{false}] +\!\!+\, bs) \mid (r_1', bs) \in \mathit{pderivBC}(r_1, x) \,\} \\
&\quad \cup \{\, (r_2',\, [\mathit{true}] +\!\!+\, bs) \mid (r_2', bs) \in \mathit{pderivBC}(r_2, x) \,\} \\
\mathit{pderivBC}(r_1\, r_2, x) &= \{\, (\mathit{seq}(r_1', r_2),\, bs) \mid (r_1', bs) \in \mathit{pderivBC}(r_1, x) \,\} \\
&\quad \cup \begin{cases} \{\, (r_2',\, \mathit{mkEpsBC}_{r_1} +\!\!+\, bs) \mid (r_2', bs) \in \mathit{pderivBC}(r_2, x) \,\} & n(r_1) \\ \emptyset & \text{otherwise} \end{cases} \\
\mathit{pderivBC}(r^*, x) &= \{\, (\mathit{seq}(r', r^*),\, [\mathit{false}] +\!\!+\, bs) \mid (r', bs) \in \mathit{pderivBC}(r, x) \,\}.
\end{aligned}
\tag{6.5}
$$

As in the standard version, the result is deduplicated by residual, keeping the first occurrence.

The bit-coded sequence rule is the counterpart of the injection rule in the standard version. When the first factor is nullable, the bits recorded inside its empty match are extracted with $\mathit{mkEpsBC}_{r_1}$ and prepended to the second factor's bits, so the choice made inside the first factor survives in the surviving configuration.

In Rust. Figure 6.7 implements (6.5) directly. The base cases and the alternation case mirror the standard version with bit-lists in place of closures. The sequence case inlines both the non-nullable and nullable paths. The star case prepends the leading *false* that marks one more iteration.

```
pub fn pderiv_bc(r: &Regex, x: char) -> Vec<(Regex, Vec<bool>)> {
    match r {
        Regex::Phi | Regex::Eps => vec![],

        Regex::Lit(y) => {
            if *y == x { vec![(Regex::Eps, vec![])] } else { vec![] }
        }

        Regex::Alt(r1, r2) => {
            let mut out: Vec<(Regex, Vec<bool>)> = pderiv_bc(r1, x)
                .into_iter()
                .map(|(r1_, mut bs)| {
                    let mut b = vec![false]; b.append(&mut bs);
                    (r1_, b)
                })
                .collect();
            out.extend(pderiv_bc(r2, x).into_iter().map(|(r2_, mut bs)| {
                let mut b = vec![true]; b.append(&mut bs);
                (r2_, b)
            }));
            nub2(out)
        }

        Regex::Seq(r1, r2) => {
            if nullable(r1) {
                let mut out: Vec<(Regex, Vec<bool>)> = pderiv_bc(r1, x)
                    .into_iter()
                    .map(|(r1_, bs)| (Regex::seq(r1_, *r2.clone()), bs))
                    .collect();

                let eps_bits = mk_eps_bits(r1);
                out.extend(pderiv_bc(r2, x).into_iter().map(|(r2_, bs)| {
                    let mut b = eps_bits.clone(); b.extend(bs);
                    (r2_, b)
                }));
                nub2(out)
            } else {
                pderiv_bc(r1, x)
                    .into_iter()
                    .map(|(r1_, bs)| (smart_seq(r1_, r2), bs))
                    .collect()
            }
        }

        Regex::Star(r1) => {
            let out: Vec<(Regex, Vec<bool>)> = pderiv_bc(r1, x)
                .into_iter()
                .map(|(r1_, mut bs)| {
                    let mut b = vec![false]; b.append(&mut bs);
                    (smart_seq(r1_, &Regex::star(*r1.clone())), b)
                })
                .collect();
            nub2(out)
        }
    }
}
```


Figure 6.7: `pderiv_bc`: one bit-coded partial-derivative transition

The `Seq` arm keeps the outer sequence constructor in the nullable case, matching the standard version's behaviour. It uses `smart_seq` only in the non-nullable case, where the derivative of the first factor is guaranteed to be a proper expression and no outer pair needs to be preserved. This asymmetry mirrors the paper's reference implementation and is needed so that the residual's shape stays compatible with the decoder.

6.2.2 Extracting Bits

When a configuration's residual is nullable, its accumulated bits must be extended with the bits of the residual's empty match. This is the role of $mkEpsBC$, which mirrors $mkEps$ from Chapter 5 but produces a bit-sequence instead of a parse tree.

$$\begin{aligned} mkEpsBC_{\varepsilon} &= [\,] \\ mkEpsBC_{r^*} &= [true] \\ mkEpsBC_{r_1 r_2} &= mkEpsBC_{r_1} +\!\!+ mkEpsBC_{r_2} \\ mkEpsBC_{r_1 + r_2} &= \begin{cases} [false] +\!\!+ mkEpsBC_{r_1} & n(r_1) \\ [true] +\!\!+ mkEpsBC_{r_2} & \text{otherwise.} \end{cases} \end{aligned} \tag{6.6}$$

The alternation rule reproduces the left preference of $mkEps$, emitting *false* when the left branch is nullable and *true* otherwise. The star rule emits a single terminating *true*, since a star matching the empty word uses zero iterations.

```
pub fn mk_eps_bits(r: &Regex) -> Vec<bool> {
    match r {
        Regex::Phi => panic!("mk_eps_bits called on Phi"),
        Regex::Lit(c) => panic!("mk_eps_bits called on Lit('{}')", c),
        Regex::Eps => Vec::new(),
        Regex::Alt(r1, r2) => {
            if nullable(r1) {
                let mut bits = vec![false];
                bits.extend(mk_eps_bits(r1));
                bits
            } else {
                let mut bits = vec![true];
                bits.extend(mk_eps_bits(r2));
                bits
            }
        }
        Regex::Seq(r1, r2) => {
            let mut bits = mk_eps_bits(r1);
            bits.extend(mk_eps_bits(r2));
            bits
        }
        Regex::Star(_) => vec![true],
    }
}
```

Figure 6.8: `mk_eps_bits`: the bit-string analogue of `mk_eps` for plain `Regex`.

The `Phi` and `Lit` arms panic, matching the precondition that *mkEpsBC* is applied only to nullable expressions. Neither empty language nor a literal can match the empty word.

6.2.3 State Sets and Selection

State-set transition for the bit-coded parser mirrors the standard version, with bit-list concatenation replacing closure composition:

```
pub(super) fn step_frontier(
    frontier: &[(Regex, Vec<bool>)], x: char
) -> Vec<(Regex, Vec<bool>)> {
    let mut out = Vec::new();
    for (r, bits) in frontier {
        for (r_next, extra) in pderiv_bc(r, x) {
            let mut combined = bits.clone();
            combined.extend(extra);
            out.push((r_next, combined));
        }
    }
    out
}
```

Selection walks the state set in priority order and returns the bits of the first accepting configuration, extended with *mkEpsBC* of the residual:

```
pub(super) fn select_bits(
    frontier: &[(Regex, Vec<bool>)]
) -> Option<Vec<bool>> {
    for (r, bits) in frontier {
        if nullable(r) {
            let mut result = bits.clone();
            result.extend(mk_eps_bits(r));
            return Some(result);
        }
    }
    None
}
```

The resulting bit-sequence is interpreted by the shared `decode` function from Chapter 5.2, which walks the original expression and consumes one bit per alternation and per star iteration.

6.2.4 Parser

The parser composes `step_frontier` over the input, checks that the state set is non-empty, and calls `select_bits` followed by `decode`.

```
pub fn parse_pderiv_bc(input: &str, r: &Regex) -> Option<ParseTree> {
    let mut frontier = vec![(r.clone(), Vec::new())];
    for c in input.chars() {
        frontier = step_frontier(&frontier, c);
        if frontier.is_empty() {
            return None;
        }
    }
    let bits = select_bits(&frontier)?;
    Some(decode(r, &bits))
}
```

Figure 6.9: The bit-coded partial-derivative parser.

The driver has the same structure as the standard one. Only the configuration type and the final step differ: bits and `decode` in place of closures and injection application.

6.2.5 Worked Example: Ambiguous Expression

Consider again the ambiguous example $r_0 = (a + ab)(b + \varepsilon)$ on input `ab`.

Forward pass. The initial state set holds a single configuration with empty bits:

$$S_0 = \Big[(r_0,\ [\,])\Big].$$

Transitioning by `a`, the first factor is not nullable, so *pderivBC* recurses into it:

$$\begin{aligned} \mathit{pderivBC}(a + ab, \mathtt{a}) &= \mathit{pderivBC}(a, \mathtt{a}).\mathit{map}(+[\mathit{false}]) \cup \mathit{pderivBC}(ab, \mathtt{a}).\mathit{map}(+[\mathit{true}]) \\ &= \Big[(\varepsilon,\ [\mathit{false}]),\ (b,\ [\mathit{true}])\Big]. \end{aligned}$$

The sequence rule then places the second factor after each derivative:

$$S_1 = \Big[(b + \varepsilon,\ [\mathit{false}]),\ (b\,(b + \varepsilon),\ [\mathit{true}])\Big].$$

Transitioning by `b`, both configurations are stepped. The first configuration reaches the second factor directly:

$$\mathit{pderivBC}(b + \varepsilon, \mathtt{b}) = \Big[(\varepsilon,\ [\mathit{false}])\Big],$$

giving bits $[\mathit{false}, \mathit{false}]$. The second configuration's derivative comes from b inside the leading sequence:

$$\mathit{pderivBC}(b, \mathtt{b}) = \Big[(\varepsilon,\ [\,])\Big],$$

and the outer sequence rule reconstructs the residual $b + \varepsilon$ with the bits unchanged:

$$S_2 = \Big[(\varepsilon,\ [\mathit{false}, \mathit{false}]),\ (b + \varepsilon,\ [\mathit{true}])\Big].$$

Selection and decoding. The first configuration is accepting, so *selectBits* returns its bits extended with $mkEpsBC_{\varepsilon} = [\,]$:

$$selectBits(S_2) = [false, false].$$

Decoding these bits against r_0:

$$decode_{r_0}\Big([false, false]\Big) = \Big(Left(a), Left(b)\Big).$$

The first *false* selects the left branch of the first factor's alternation, committing to the shorter alternative a. The second *false* selects the left branch of the second factor's alternation, committing to the shorter alternative b. The resulting tree is the greedy parse tree, identical to the one produced by the standard partial-derivative parser.

Flattening validates the result:

$$flatten\Big((Left(a), Left(b))\Big) = \texttt{ab}.$$

Both partial-derivative parsers therefore produce the same tree on this example, and both differ from the derivative-based parsers of Chapter 5, which commit to the longer first alternative and return $(Right(a, b), Right())$.

7 External To Internal Regex

The examples presented thus far have been expressed directly in terms of `Regex`, the six-constructor core algebra introduced in Section 2.3. This is the representation consumed by `deriv` and `pderiv`. External regular-expression patterns, however, are provided as strings using a PCRE-style syntax. For example, a pattern such as `"[a-c]+\d{2}"` must first be converted into the internal representation before it can be processed by the parsers.

The frontend in `src/frontend/` provides this conversion. It first parses the external pattern into a surface-syntax AST and then translates that AST into the `Regex` representation used by the parsers in Chapters 5 and 6. Its design is based on the `Text.Regex.PDeriv` implementation in `luzhuomi/regex-pderiv`, in particular the `ExtPattern`, `Parse`, and `Translate` components used in Kenny Lu's implementation of the partial-derivative approach of Sulzmann and Lu [2012]. This implementation provides a separate reference for the treatment of external syntax. The construction in Sulzmann and Lu [2014], by contrast, operates directly on the six-constructor `Regex` algebra and does not define a PCRE-oriented frontend.

The real-world corpus described in Section 8.2 depends on this frontend to convert Suricata, SpamAssassin, and RegexLib rule patterns into `Regex` values. This conversion includes parsing the surface syntax and desugaring constructs that are not represented by `Regex`; a construct the algebra cannot express at all is rejected rather than desugared.

This frontend accepts a strict subset of PCRE syntax. Three categories of construct are rejected outright: backreferences (`\1`–`\9`), lookaround assertions (`(?=...)`, `(?!...)`, `(?<=...)`, `(?<!...)`), and anchors, which this frontend takes to include both the string anchors `^`/`$` and the word-boundary assertions `\b`/`\B`. None of the three denote a regular language over `Regex`'s six constructors: a backreference requires matching against a previously captured substring, which no alphabet-indexed derivative can express; and lookaround and word boundaries test the input around the current position without consuming it, a property of a match *position* rather than of a language. String anchors are the same kind of problem one level up: they constrain *where* in the input a pattern may match, and Section 7.4 discusses why this frontend has nothing left to attach that constraint to once it performs no search at all. Rejecting all three explicitly, rather than silently ignoring or mistranslating them, is what lets corpus extraction (Section 8.2) identify unsupported patterns instead of miscounting them as supported.

The dataset-preparation tooling (`dataset_stats`, `demo_canonical_reject`) splits every rejection into seven causes. The first four above account for most of them directly; the remaining three are specific to `parse_pcre_rule`'s own flag handling and a catch-all, and are listed here for completeness:

Cause	Example	Reason
Backreference	`\1`	Requires matching a previously captured substring; no alphabet-indexed derivative expresses that.
Lookaround	`(?!...)`	Tests the input around the current position without consuming it.
Word boundary	`\b`	Position-dependent, for the same reason as lookaround.
Anchor	`^`, `$`	Constrains *where* a match may sit (Section 7.4).
Flag `m`	`/^a$/m`	Changes where `^`/`$` match, which no longer exist to be changed.
Flag `x`	`/a b/x`	Makes pattern whitespace insignificant, which this parser does not model.
Other	`.{1586,}`	Everything matching none of the above: an ambiguous POSIX class such as `[:punct:]` inside a character class, a bound exceeding `MAX_BOUND` (Section 7.3), or an ordinary syntax error such as an unmatched `(`.

Every example above except the last two flags is drawn from the real-world corpus (Section 8.2); flag `x` does not occur there at all, and the flag `m` example is simplified from the corpus pattern that triggers it.

7.1 Two ASTs

The frontend separates surface syntax from the internal regular-expression algebra by using two abstract syntax trees connected by a single translation step:

$$\text{pattern string} \xrightarrow{\texttt{parse_ext_pattern}} \texttt{ExtPat} \xrightarrow{\texttt{translate}} \texttt{Regex}$$

`ExtPat` (`ext_pattern.rs`) represents the external surface syntax. It includes constructs such as marking and non-marking groups, bounded repetitions, character-class shorthands such as `\d`, `\w`, and `\s`, anchors, and the `greedy: bool` field associated with quantifiers. These constructs remain explicit in `ExtPat` and are not desugared during parsing.

`Regex`, introduced in Section 2.3, is the internal six-constructor representation `Phi | Eps | Lit | Seq | Alt | Star`. It has no representation for groups, repetition bounds, anchors, or quantifier laziness. Constructs expressible in `ExtPat` but absent from `Regex` are desugared by `translate` (Section 7.3) whenever a `Regex` equivalent exists. Anchors have none: unlike groups or bounds, `^` and `$` are not desugared into an equivalent combination of the six constructors, they are rejected by `translate`, the same treatment the word-boundary assertions `\b`/`\B` already receive. Section 7.4 returns to this once the rest of translation has been presented.

The explicit intermediate representation also isolates parsing from translation. The unit tests in `translate.rs`, for example, can construct `ExtPat` values directly and verify their translated `Regex` representations without depending on the recursive-descent parser in `parse.rs`. This separation is analogous to the distinction between parse-tree construction and operations such as `flatten` and `decode` discussed in Section 4.2: each stage has a distinct input and output representation and can therefore be tested independently.

7.2 Parsing Surface Syntax

The parser in `frontend/parse.rs` is a recursive-descent parser operating on a `Chars` iterator. Its structure follows the grammar used by `Text.Regex.PDeriv.Parse`. The reference functions `p_ere`, `p_branch`, `p_exp`, `p_atom`, `p_group`, `p_charclass`, `p_enum`, and `p_bound` correspond to `parse_or`, `parse_concat`, `parse_postfixed`, `parse_atom`, `parse_group`, `parse_charclass`, `parse_class_enum`, and `parse_bound`. These functions implement the respective levels of the surface grammar.

The implementation differs from the reference parser in several respects.

- **Class shorthands are interpreted as character classes.** The reference implementation's `Translate.hs` treats a bare `EEscape c` as the literal character `c`. This frontend instead interprets `d`, `D`, `w`, `W`, `s`, and `S` as character-class shorthands (Section 7.3). This interpretation corresponds to their use in the external patterns processed by the project.
- **Backreferences, lookaround, and anchors are all rejected, at two different stages.** Constructs such as `\1`–`\9` and lookaround assertions have no `ExtPat` constructor to parse into at all, so `parse_ext_pattern` itself returns a parse error the moment it encounters one. Anchors are different: `^`, `$`, and the word-boundary escapes `\b`/`\B` each parse successfully, into `ExtPat::Carat`, `ExtPat::Dollar`, and `ExtPat::WordBoundary`; it is `translate` (Section 7.3) that rejects them, one stage later. Both stages exist for the same reason: none of the five constructs are represented by the regular-expression algebra used by the derivative constructions, and rejecting them explicitly, rather than silently ignoring or mistranslating them, is what corpus extraction (Section 8.2) relies on to identify unsupported patterns instead of miscounting them as supported.
- **Named groups are accepted.** Both `(?<name>...)` and `(?P<name>...)` are parsed and subsequently translated to their unnamed group contents. Group names have no representation in `Regex` and therefore have no effect after translation. Supporting the syntax nevertheless permits patterns containing named groups to be processed without introducing capture-group semantics into the internal representation.
- **Hexadecimal escapes are accepted.** The parser additionally accepts `\xHH` hexadecimal character escapes, which are not provided by the reference frontend.

The function `parse_bound` also recognizes lazy-quantifier markers, such as `a*?`, `a+?`, and `a{2,4}?`. The marker is stored in the `greedy: bool` field of `ExtPat`, although it does not affect the subsequent translation, as described in Section 7.3.

The precedence of the surface syntax follows the reference grammar. Alternation has

the lowest precedence, followed by concatenation and then postfix quantifiers, with each quantifier applying to a single atom. At the atom level, the parser handles literals, escapes, character classes, groups, and anchors.

7.3 Translating `ExtPat` into `Regex`

The function `translate` in `frontend/translate.rs` is defined by structural recursion over `ExtPat`. Each constructor has a corresponding match arm that either maps directly to a `Regex` constructor or explicitly desugars the surface construct:

`ExtPat`	`Regex`
`Empty`, `Eps`	`Eps`
`Never`	`Phi`
`Char(c)`	`Lit(c)`
`Group`/`GroupNonMarking(r)`	`translate(r)`
`Or([r1, ..., rn])`	left-folded `Alt` chain
`Concat([r1, ..., rn])`	left-folded `Seq` chain
`Opt(r)`	`Alt(translate(r), Eps)`
`Plus(r)`	`Seq(translate(r), Star(translate(r)))`
`Star(r)`	`Star(translate(r))`
`Dot`, `Any(cs)`, `NoneOf(cs)`	sorted, deduplicated `Alt` of `Lit`
`Bound(r, lo, hi)`	unrolled exact/optional repetition
`Carat`, `Dollar`	rejected
`WordBoundary(b)`	rejected

The translation therefore removes all surface constructs that are not present in the core algebra while preserving the language represented by the supported constructs, with one exception: the last two rows above have no `Regex` to remove them *into*, so `translate` reports an error instead of returning one at all. Section 7.4 discusses the consequences once the rest of translation has been presented.

Character classes are represented by alternation. `Any`, `NoneOf`, `Dot`, and class shorthands are translated via `alt_of_chars`, which sorts and deduplicates the character set into a left-folded `Alt` chain. The file `alphabet.rs` provides 98 characters (printable ASCII, tab, newline, carriage return). The functions `digit_chars`, `word_chars`, and `space_chars` define sets for `\d`, `\w`, and `\s`, with negated forms via `complement`.

The `Regex` representation contains no dedicated character-class constructor. Consequently, every character class is reduced to the six constructors of the core algebra. For example, a `\d` shorthand is represented as an alternation over its ten characters, which the derivative implementations subsequently process through the ordinary `Alt` structure.

Bounds are desugared by explicit repetition. The function `desugar_bound` constructs the exact lower-bound portion using `repeat_exact`. This function produces a left-folded `Seq` chain and returns `Eps` when `lo == 0`. For an unbounded upper limit, the remaining repetitions are represented by a `Star`. Thus, `r{2,}` becomes two explicit copies followed by `Star(r)`. For a finite upper bound, `hi - lo` additional copies are represented using nested optional expressions constructed by `optional_extra`.

Bounded repetition can substantially increase the size of the internal expression. A short source pattern such as `r{1586,}` would otherwise produce 1586 nested `Seq` nodes before the trailing `Star` is introduced. The frontend therefore imposes `MAX_BOUND = 1000`. This limit is specific to the implementation and is not inherited from the reference parser. Bounds exceeding the limit result in a descriptive translation error rather than the construction of an excessively large `Regex`.

The check is on the repetition count alone, not on the size of the expression it produces, and the two differ sharply depending on what is being repeated. `a{1586,}` desugars to 1586 copies of a single `Lit` node: about 1586 nodes in total. `.{1586,}` desugars to 1586 copies of `Dot`, and `Dot` alone already lowers to 195 nodes (an `Alt` chain over the 98-character alphabet), so the result is on the order of 309,000 nodes before the trailing `Star` is even added. `desugar_bound` inspects only `lo` and `hi`; it treats both patterns identically, since 1586 exceeds `MAX_BOUND` regardless of what is repeated. A separate, coarser safeguard in the dataset-preparation pipeline (`src/data/prepare.rs`'s `MAX_CORE_NODES = 20,000`, applied to the pattern as a whole after translation) would catch `.{1586,}`'s expansion on size alone, independently of `MAX_BOUND`. No equivalent check exists outside that pipeline, however: a bare `parse` or `match` invocation has no backstop beyond `MAX_BOUND` itself, so a bound that passes the cap but wraps an expensive sub-expression can still produce a disproportionately large `Regex`.

`demo_frontend_translation_sizes` measures the translated node count of a representative sample directly, and `test_thesis_figures.rs` pins the same numbers:

Pattern	Nodes
`a*`	2
`a?`	3
`a+`	4
`abc`	5
`a{2}`	5
`a{2,}`	8
`a{2,4}`	15
`\d`	19
`\d{2}`	41
`[a-c]+\d{2}`	54
`\w`	125
`[^a]`	193
`.`	195
`\w+`	252

The shorthand classes grow with alphabet coverage rather than with pattern length: `\d` (ten digits) costs 19 nodes, while `\w` (alphanumeric and underscore, 63 characters under this project's 98-character alphabet) costs 125, and `.` and the negated class `[^a]`, which must enumerate the alphabet's remaining 97 characters, cost 195 and 193 respectively – all three several times larger than `abc`, a literal three characters long. Case-folding adds a fixed two nodes per letter regardless of which letter: `/abc/` translates to 5 nodes, `/abc/i` to 11, since `case_fold` rewrites each `Lit(c)` into `Any([c, C])`, an alternation of two characters in place of one.

Greedy and lazy quantifiers are parsed but not distinguished. The translation of `Opt`, `Plus`, `Star`, and `Bound` does not inspect the `greedy` flag recorded by `parse_bound`. The internal `Regex` representation has no per-quantifier notion of greediness or laziness. Disambiguation is instead determined by the parser as a whole, with the derivative-based and partial-derivative-based constructions implementing their respective policies, as described in Section **??**.

Consequently, `a*` and `a*?` translate to identical `Regex` values. A pattern containing both greedy and lazy quantifiers can therefore be parsed successfully, but the distinction between the two quantifier forms is not preserved by translation. The frontend consequently accepts a broader surface syntax than can be distinguished by the internal representation.

7.4 Anchors and the Two Entry Points

The parsers in Chapters 5 and 6 perform full-string matching: after the input has been consumed, the resulting expression is tested for nullability. They do not provide a separate operation for determining whether a pattern occurs as a substring of a longer input. The derivative constructions of Sulzmann and Lu [2014] and the corresponding teaching material likewise define matching over the complete input, and this frontend makes no attempt to build a search operation on top of them: every pattern is matched against the whole of its input, nothing less and nothing more. The two entry points below, `parse_pattern` and `parse_pcre_rule`, both end by calling `translate` and handing its result straight to a parser; neither pads, searches, or otherwise changes what the translated pattern is matched against. Section 8.2 discusses what this costs against a real-world corpus whose rules generally do expect search semantics; this section is only concerned with what the frontend itself does, which is to reject any construct whose behaviour depends on *where* in the input it appears, since `Regex` has no notion of where at all.

`^`, `$`, and `\b`/`\B` are rejected at the same site, for the same reason. Figure 7.1 is the two match arms of `translate` (`frontend/translate.rs`) responsible, already marked as rejected in the table of Section 7.3. Both produce a descriptive `Err` rather than silently approximating the construct with something else, such as the `Eps` this frontend used to lower anchors to.

```
ExtPat::Carat | ExtPat::Dollar => {
    Err("anchor '^'/'$' is not supported by this frontend -- unsupported".to_string())
}
// ...
ExtPat::WordBoundary(_) => {
    Err("word boundary '\b'/'\B' is position-dependent, not a regular-language \
         construct -- unsupported".to_string())
}
```

Figure 7.1: The two rejecting arms of `translate`. Unlike backreferences and lookaround (Section 7.2), which have no `ExtPat` constructor and are rejected while parsing, these four constructs parse successfully into `ExtPat::Carat`, `ExtPat::Dollar`, and `ExtPat::WordBoundary` and are rejected one stage later, during translation.

Rejection is total, not local. `translate` is defined by structural recursion (Section 7.3), and every recursive call site either passes its result through Rust's `?` operator or, for `Group`/`GroupNonMarking`, returns it directly as a tail call. Figure 7.2 shows three representative arms: `Group` forwards whatever its inner call returns, unchanged; `Or` accumulates over a list, applying `?` to each element's translation in turn; `Star` applies `?` inline to its one recursive call. `Concat`, `Plus`, and `Bound`'s `desugar_bound` all propagate the same way, through a loop-accumulated or inline `?`. An `Err` returned by any recursive call is therefore returned immediately by its caller, without constructing a partial `Regex` around it, and that caller's own caller does the same, all the way back to `translate`'s original entry point.

```
ExtPat::Group(inner) | ExtPat::GroupNonMarking(inner) => translate(inner),
// ...
ExtPat::Or(alts) => {
    let mut iter = alts.iter();
    let mut acc = translate(iter.next().ok_or("empty alternation")?)?;
    for a in iter {
        acc = Regex::alt(acc, translate(a)?);
    }
    Ok(acc)
}
// ...
ExtPat::Star(inner) => Ok(Regex::star(translate(inner)?)),
```

Figure 7.2: Three of `translate`'s recursive arms. Every other compound arm (`Concat`, `Opt`, `Plus`, `Bound`) follows the same `?`-propagation shape.

The practical consequence is that an anchor anywhere in a pattern rejects the pattern as a whole, regardless of how deeply it is nested or whether it sits behind an optional quantifier that would, if taken, not require it at all. Figure 7.3 confirms this against the built CLI for five increasingly indirect placements of `^`: directly optional, optional behind a group, inside one branch of an alternation, under a star, and under an explicit bound.

`parse_pcre_rule` additionally unwraps PCRE delimiters, case-folds on `i`, and rejects two more flags outright. Suricata and SpamAssassin rule text is not a bare pattern: it carries its own `/PATTERN/FLAGS` wrapper, the same convention PCRE tools use to separate the pattern from modifiers such as case-insensitivity. `parse_pcre_rule` strips this wrapper with `strip_pcre_delimiters` before parsing, and, if the `i` flag is present, applies `case_fold` to the resulting `ExtPat` before translation. `case_fold` rewrites every literal character into an `Any` of both cases, so `a` becomes `Any(['a','A'])`; the fold happens at the `ExtPat` level rather than at match time because `Regex` itself has no case-insensitivity concept for a parser to consult. The `m` (multiline) and `x` (extended) flags are rejected outright by `PcreFlags::unsupported`, for the same underlying reason as anchors: `m` only changes where `^`/`$` match, which no longer exists to be changed, and `x` makes pattern whitespace insignificant, which this parser does not model. `R` (relative), `A` (anchored), and `E` (dollar-endonly) are treated differently again: each used to position or tune a search or an anchor this frontend no longer has, so each is now simply ignored, the same way an unrecognized buffer-selector flag such as Snort's `U` or `H` always was. `parse_pattern` passes any input through unchanged if it is not `/`-delimited, so a bare

```
$ cargo run -- parse "^?abc" "abc"
Regex parse error: anchor '^'/'$' is not supported by this frontend -- unsupported

$ cargo run -- parse "(^)?abc" "abc"
Regex parse error: anchor '^'/'$' is not supported by this frontend -- unsupported

$ cargo run -- parse "(^a|b)c" "bc"
Regex parse error: anchor '^'/'$' is not supported by this frontend -- unsupported

$ cargo run -- parse "(^a)*b" "b"
Regex parse error: anchor '^'/'$' is not supported by this frontend -- unsupported

$ cargo run -- parse "(^a){0,1}b" "b"
Regex parse error: anchor '^'/'$' is not supported by this frontend -- unsupported
```

Figure 7.3: Every placement is rejected identically, including the three where the anchor is reachable only through an optional, alternative, or bounded branch that need not be taken at all. `translate` never gets the chance to discover that: structural recursion visits the anchor node regardless of what quantifier or alternation surrounds it, and Figure 7.2's `?` chain reports it immediately.

pattern with no wrapper is accepted by either entry point exactly as written.

One flag remains: `s` (dot-all) is honoured rather than rejected or ignored. `parse_pattern` has no flags to read at all, so `.` matches `\n` under it unconditionally. `parse_pcre_rule` instead applies `strip_dot_newline` to the parsed `ExtPat` whenever `s` is *absent*, removing `\n` from `.`'s expansion; when `s` is present, the pass is skipped and `.` expands over the full alphabet exactly as `parse_pattern` already does. `PcreFlags` tracks exactly these four letters – `i`, `s`, `m`, `x` – and nothing else; every other letter reaching `parse_pcre_rule` is an unrecognized buffer-selector by elimination, handled identically to `R`, `A`, and `E`:

Flag	Meaning	Handling
`i`	case-insensitive	Honoured: `case_fold` rewrites every `Lit(c)` to `Any([c, C])` before translation.
`s`	dot-all	Honoured: absent, `strip_dot_newline` excludes `\n` from `.`; present, `.` expands over the whole alphabet.
`m`	multiline	Rejected: would change where `^`/`$` match, which no longer exist to be changed.
`x`	extended	Rejected: makes pattern whitespace insignificant, which this parser does not model.
`R`, `A`, `E`	relative, anchored, dollar-endonly	Ignored: positioned or tuned a search or an anchor this frontend no longer has.
other (`U`, `H`, `P`, `C`, ...)	buffer selectors	Ignored: name which buffer a rule is tested against, not what the pattern itself matches.

A third, narrower entry point recovers one further anchor shape, for the dataset pipeline only. Suricata and SpamAssassin rule text rarely omits `^`/`$` by accident: a pattern wrapped in `^...$` at its own top level is the source author's own explicit statement that the whole pattern, not a part of it, is what is meant to match. Once matching is full-string only, that wrap is provably redundant – full-string matching already requires consuming the entire input, so stripping the two anchor nodes and translating what remains changes which inputs match not at all. `core_regex_tagged` (`src/data/prepare.rs`) performs exactly this recovery, and only this recovery: it recognizes the one shape where `^` is the pattern's literal top-level-first element and `$` its literal top-level-last element – seeing through `Group`/`GroupNonMarking` wrappers to find them, but nothing else – and strips exactly those two nodes before handing the rest to `translate`. A lone `^` or `$`, one nested inside an `Or` branch or under a `Star`/`Bound`, or any placement other than this one, falls through unchanged and is rejected by `translate` exactly as above.

This recovery exists only for `core_regex_tagged`: neither `translate` itself nor the two entry points `parse_pattern` and `parse_pcre_rule` strip or special-case this shape. A pattern such as `^abc$` fed directly to `parse_pcre_rule` is still rejected, because `translate` still visits `ExtPat::Carat` and `ExtPat::Dollar` and still returns an error the moment it does. Only the dataset-preparation pipeline (Section 8.2) calls `core_regex_tagged` when converting corpus rule text into `Regex` values for benchmarking.

`core_regex_tagged` returns not just a `Regex` but a `MatchIntent`, recording which of the two accepted shapes a pattern had: `ExplicitFullMatch` for the `^...$`-wrapped shape just described, and `UnanchoredAmbiguous` for a pattern with no anchor at all, translated as-is. The two names mark a real asymmetry rather than a cosmetic one. Suricata's and SpamAssassin's own tools treat an anchor-free pattern as a substring search within a buffer by default, not a full match; this project's parsers perform full-string matching

only, so every `UnanchoredAmbiguous` pattern is benchmarked as a full match regardless – this project's own interpretive choice, not the source tool's stated convention, which the tag name is meant to keep visible rather than let a reader assume the pattern's author intended a full match. `ExplicitFullMatch` carries no such ambiguity: the source author's own `^...$` already says full match, so that tag records only that the pattern reached `Regex` by a different route, not that its match semantics are in question.

The following gives both tiers' share of every pattern this frontend accepts from the real-world corpus (Section 8.2). `ExplicitFullMatch` is the smaller tier for both sources, but not negligible: roughly one accepted pattern in eleven states its own full-match intent explicitly rather than leaving it to this project's own reading.

	`UnanchoredAmbiguous`		`ExplicitFullMatch`	
Source	Count	%	Count	%
Suricata	5573	91.0	550	9.0
SpamAssassin	139	85.8	23	14.2
Total	**5712**	**90.9**	**573**	**9.1**

8 Evaluation

Chapters 5 and 6 developed four parser combinations, `deriv_std` (recursive and loop variants), `deriv_bc`, `pderiv_std` and `pderiv_bc`, each claimed to compute either the POSIX or the leftmost-first parse tree, by four structurally different constructions. This chapter evaluates those four parsers against each other, first for correctness and then for performance, on two kinds of input: 1. Small, constructed patterns taken from the source paper to isolate one algorithmic property at a time; 2. A corpus of real-world regular expressions assembled from three sources: Snort/Suricata, SpamAssassin and RegexLib, that Mamouras et al.'s disambiguation-robustness study [Mamouras et al., 2024] uses.

That paper studies exactly the POSIX-versus-Greedy question this thesis studies, via real intrusion-detection signatures, and finds concrete real-world patterns where the two policies visibly diverge. It provides no public dataset artifact of its own, so Section 8.2 describes how the corpus was independently assembled and prepared.

Section 8.1 checks the four parsers, and the three Boolean matchers of Chapter 3, against each other and against the paper's own worked examples. Section 8.2 describes how the real-world corpus specifically is assembled and prepared into a benchmarkable dataset; the small hand-constructed patterns used directly in Section 8.3's own benchmark families need no such preparation and are introduced there instead. Section 8.3 measures where the four parsers' costs diverge on each and the final Section 8.4 draws the two together.

8.1 Parser Agreement

Paper A1 Example:. $r_1 = (a + ab)(b + \varepsilon)$ and $w =$ `ab`,

```
$ cargo run -- parse "(a|ab)(b|)" "ab" --parser all
Parser       | Policy    | Result
-------------+-----------+--------------------------------
DERIV_REC    | POSIX     | (Right (a, b), Right ()) -> "ab"
DERIV_LOOP   | POSIX     | (Right (a, b), Right ()) -> "ab"
DERIV_BC     | POSIX     | (Right (a, b), Right ()) -> "ab"
PDERIV_STD   | NON-POSIX | (Left a, Left b) -> "ab"
PDERIV_BC    | NON-POSIX | (Left a, Left b) -> "ab"

POSIX parsers agree
PDERIV parsers agree
NON-POSIX agrees with POSIX on membership
```

Figure 8.1: Comparison of the parser results for the regular expression $(a|ab)(b|)$.

Paper A2 Example:. $r_2 = (a + b + ab)^*$ against $w =$ `ab`,

8.2 Dataset Preparation

This section builds the one dataset the measurements later in this chapter run against: real-world regular expressions drawn from Suricata, SpamAssassin, and RegexLib rule text, carried through extraction, acceptance, input generation, and verification. The small hand-constructed patterns used elsewhere in this thesis – the $(a + ab)(b + \varepsilon)$ worked example, $(a^*)^*$-style pathological expressions, and similar – are not part of it: each exists to isolate one specific algorithmic property for one specific figure or trace, and pooling them with a real-world corpus would blur exactly the property each one is there to isolate. Only the real-world corpus is prepared as a reusable, benchmarkable dataset; it is the sole input to everything that follows in this chapter.

8.2.1 The Real-World Corpus

This thesis's corpus is independently assembled from three families of real-world regular-expression sources, chosen to cover different "shapes" of real regex use rather than three samples of the same thing:

- **Snort/Suricata** network intrusion-detection signatures (`pcre:"..."` rule options, sourced from the Emerging Threats Open ruleset), literal-heavy, moderate complexity, the closest of the three to "everyday" regex use.
- **SpamAssassin** email spam-filtering rules (`body`/`header` Perl-regex rules), content matching over unstructured text, a different shape than network signatures.
- **RegexLib** crowd-sourced general-purpose patterns (validators, extractors), the opposite end, where anchors and chained lookahead assertions are both idiomatic, stress-testing the frontend's rejection path hardest of the three.

Assembling a real-world corpus this way, rather than relying only on hand-constructed patterns, follows the motivation in Mamouras et al.'s disambiguation-robustness study [Mamouras et al., 2024]: that paper studies exactly the POSIX-versus-Greedy question this thesis studies, via real intrusion-detection signatures, and finds concrete real-world patterns where the two policies visibly diverge. It publishes no dataset artifact of its own, so the corpus used here was assembled independently.

Extraction. A separate extractor reads each source's native text format directly, rather than a shared one assuming a structure the three formats do not actually have in common. `extract_suricata` scans each line of a `.rules` file for a `pcre:"..."` option, pulling out the pattern together with that rule's `content:` fields, `msg`, and `sid` as context input generation uses later. `extract_spamassassin` reads a `.cf` file's `body NAME /pattern/flags` and `header NAME Field =~ /pattern/flags` lines; the two genuinely differ in syntax, since a `body` rule carries no `=~` operator at all, and requiring one, as an earlier version of this extractor did, silently dropped every `body` rule rather than just failing loudly on it. `extract_regexlib` reads RegexLib's comment-delimited export format directly: a block of `#`-prefixed description lines followed by the bare pattern, one entry per blank-line-separated block. All three produce the same `ExtractedRule`: a source tag, the raw pattern text, and whatever source-specific context only that source provides.

Acceptance. An extracted rule's raw pattern is handed to `core_regex` (`data/prepare.rs`), which repeats `parse_pcre_rule`'s own steps (Section 7.4) – delimiter stripping, the `m/x` flag check, parsing, case-folding, `dotall` – rather than calling it directly, so that two further, pipeline-only safety caps can run around translation: a pattern whose *estimated* translated depth already exceeds 2 000 is rejected before translation is attempted at all, and a pattern that translates to more than 20 000 `Regex` nodes is rejected after. Both exist for the same reason `MAX_BOUND` does in Section 7.3: a character-class range can expand from a few source characters into thousands of code points – one RegexLib pattern in this corpus does exactly this, with a range spanning from `-` to the en dash – and translating a pattern that large is itself the risk these caps exist to avoid, independently of whether `translate` would otherwise succeed on it. A pattern surviving every check `core_regex` applies – the rejections Section 7.4 already described, plus these two size caps – is *accepted*; every other pattern is *unusable* and takes no further part in the pipeline. There is no accepted-but-approximate middle case left to report: a pattern is either translated exactly or not translated at all.

The following reports both numbers against this project's own sample corpus, `data/raw/_Samples/`: six files split across the three sources, small enough to check into the repository itself. The same extraction and acceptance code applies unchanged at whatever scale the corpus is eventually grown to; only the counts below are specific to this sample.

Source	Files	Extracted	Accepted	Unusable
Suricata	2	177	42 (23.7%)	135
SpamAssassin	3	156	49 (31.4%)	107
RegexLib	1	500	108 (21.6%)	392
Total	**6**	**833**	**199 (23.9%)**	**634**

Splitting the unusable count by cause shows the three sources are not rejected for the same reason:

Source	Backref.	Lookaround	Word-bound.	Anchor	Other	Too large[†]
Suricata	4	8	1	109	5	8
SpamAssassin	1	13	76	16	1	0
RegexLib	8	55	5	291	31	2
Total	**13**	**76**	**82**	**416**	**37**	**10**

[†]the two pipeline-only size caps above, not a rejected construct. Anchors dominate Suricata's and RegexLib's rejections, as Section 7.4 already expects of signature and validator patterns; SpamAssassin inverts this, with the word-boundary assertions `\b`/`\B` responsible for most of its rejections instead, 76 of 107, consistent with spam rules matching whole words rather than positions in a line. This is exactly the kind of difference the three sources were chosen to expose, not a flaw in any one of them.

Generating Input. Every accepted pattern's `Regex` is handed to `generate_sample` (`data/generate.rs`), which samples up to `count` distinct positive matches directly from the pattern's own structure – walking each constructor, choosing an `Alt` branch by its relative leaf count so a large character class is sampled uniformly rather than biased toward whichever side of a left-folded chain happens to be smaller, and taking a star between zero and a fixed four iterations – plus up to `negative_count` negatives, each a positive sample with its last character replaced by a character outside the working alphabet. This replaces the three-tier Best/Neutral/Worst difficulty split used earlier in

this project with one plain setting: generating a realistic match is the goal here, not adversarial growth, which Section 8.3's own benchmark families handle separately and deliberately. `dataset_prepare` calls this with `count = negative_count = 5` for every pattern.

Suricata patterns additionally draw on `generate_from_content_fields`, which assembles a candidate from the rule's own `content:` bytes with a short sampled filler between consecutive fields – real evidence of what a triggering payload contains, not a structural guess. SpamAssassin patterns additionally draw on `generate_from_corpus_dir` when a local ham/spam corpus is configured, keeping only messages whose *entire* content matches; without the search padding Section 7.4 removed, this finds far fewer candidates against a real corpus than it once did; kept for the rare whole-message pattern it still finds, not removed, since it costs nothing when it finds nothing.

Verification. Nothing `generate_sample` or its source-specific companions produce is trusted as written: `verify_candidate` (`data/prepare.rs`) re-derives each candidate's claimed outcome against the real bit-coded parser, bounded by a 100 000-node cap so a pathological candidate is dropped rather than left to run unbounded, and keeps the candidate only if the claim holds. A claimed negative is held to a second, stronger standard: its failure must fall at or past two-thirds of the input, since a candidate that fails on its first character exercises the frontend barely at all. Verification therefore drops some fraction of what was generated, and that fraction is not the same across sources. Against this project's sample corpus:

Source	Candidates Generated	Candidates Verified	Dropped
Suricata	513	404	109 (21.3%)
SpamAssassin	440	440	0 (0.0%)
RegexLib	1 026	987	39 (3.8%)
Total	**1 979**	**1 831**	**148 (7.5%)**

Suricata's much larger drop is not noise: it is the only source using `generate_from_content_fields`, whose candidates come from a rule's raw `content:` bytes rather than from the translated pattern's own structure, so nothing guarantees they satisfy it, unlike a `generate_sample` candidate, which is sampled from that structure directly and almost always verifies. SpamAssassin's and RegexLib's smaller, nonzero drops come from the same two sources as any `generate_sample`-only pipeline: a corrupted negative that happens to fail too early, and, rarely, a candidate against a pattern expensive enough to hit the node cap.

Figure 8.2 shows one source run end to end against the real sample files, matching the totals above.

What survives verification is written to `prepared.jsonl` per source and read back by `load_prepared_cases` – 1 831 cases in this sample corpus, each an input together with the pattern it was generated from, its provenance, and its verified outcome. This is the dataset the remainder of this chapter benchmarks all four parsers against.

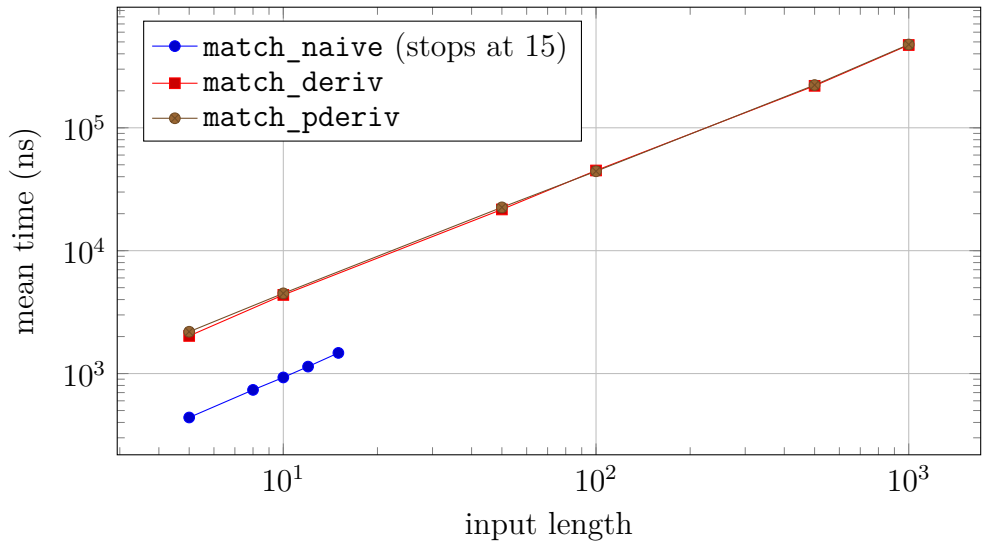
```
$ cargo run --example dataset_prepare -- regexlib \
    data/raw/_Samples/regexlib-manual-processed.sample.txt
=== RegexLib (variants per pattern: 5) ===
rules extracted:      500
patterns unusable:    392 (rejected -- anchors, lookaround, backreferences,
                       word boundaries, unsupported flags, or too large to
                       translate safely)
patterns accepted (used): 108
candidates generated: 1026
candidates verified:  987
-> data/processed/regexlib/prepared.jsonl
```

Figure 8.2: `dataset_prepare` run against the RegexLib sample file, reporting the same extraction, acceptance, and verification counts tabulated above for this source.

8.3 Performance Analysis

8.3.1 Performance on Synthetic Patterns

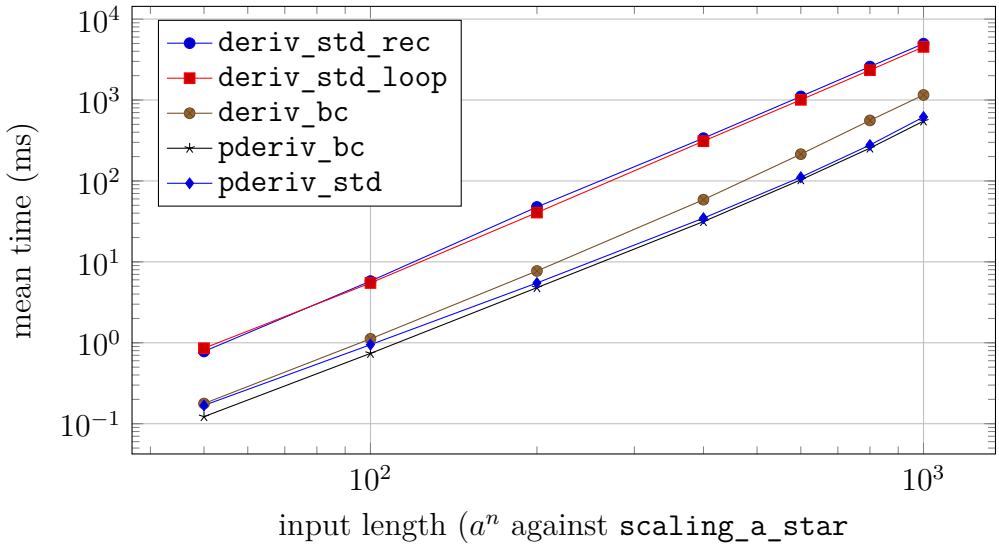
Comparing The Three Matchers.



Stack depth: recursive vs. iterative construction. Comparing recursive and loop-based standard derivative parser.

Heap allocation: `deriv_std` vs. `deriv_bc`. Savings without backward pass.

Derivative-based vs. partial-derivative-based.



Benchmark results.

Parser	n=5	n=10	n=15	n=18
deriv_std_rec	8.0 µs	110.7 µs	1.427 ms	6.052 ms
deriv_bc	11.4 µs	183.8 µs	4.175 ms	49.491 ms
pderiv_bc	13.4 µs	210.6 µs	2.207 ms	9.236 ms
pderiv_std	22.4 µs	323.9 µs	3.374 ms	14.263 ms

8.3.2 Evaluation on the Mamouras Regex Dataset

- **Corpus sample sweep**
- **Neutral-case probe**
- **Worst-case probe**
- **Best-case probe**

8.4 Discussion of Results

8.4.1 Optimization Challenges and Insights

Character classes in place of literal alternation.

Frontier-wide deduplication.

9 Related Work and Conclusion

9.1 Related Work

9.2 Conclusion

Bibliography

Valentin Antimirov. Partial derivatives of regular expressions and finite automaton constructions. *Theoretical Computer Science*, 155(2):291–319, 1996. doi: 10.1016/0304-3975(95)00182-4.

Janusz A. Brzozowski. Derivatives of regular expressions. *Journal of the ACM*, 11(4): 481–494, 1964. doi: 10.1145/321239.321249.

Russ Cox. Regular expression matching can be simple and fast (but is slow in Java, Perl, PHP, Python, Ruby, ...). https://swtch.com/~rsc/regexp/regexp1.html, 2007. Accessed: 2026-08-05.

Konstantinos Mamouras, Alexis Le Glaunec, Wu Angela Li, and Agnishom Chattopadhyay. Static analysis for checking the disambiguation robustness of regular expressions. *Proceedings of the ACM on Programming Languages*, 8(PLDI):231, 2024. doi: 10.1145/3656461.

Martin Sulzmann. Programming paradigms: Regular expressions. https://sulzmann.github.io/ProgrammingParadigms/pp-regular-expressions.html, 2026. Accessed: 2026-08-08.

Martin Sulzmann and Kenny Zhuo Ming Lu. Regular expression sub-matching using partial derivatives. In *PPDP*, pages 79–90, 2012. doi: 10.1145/2370776.2370788.

Martin Sulzmann and Kenny Zhuo Ming Lu. Posix regular expression parsing with derivatives. In *Functional and Logic Programming (FLOPS 2014)*, volume 8475. Springer, 2014.

The Rust Project Developers. Using `Box<T>` to point to data on the heap. The Rust Programming Language, 2026a. URL https://doc.rust-lang.org/book/ch15-01-box.html. Accessed: 2026-09-11.

The Rust Project Developers. Types. The Rust Reference, 2026b. URL https://doc.rust-lang.org/reference/types.html. Accessed: 2026-09-11.

A Diagnostics

`cargo run -- parse "(a|ab)(b|)" <INPUT> --parser <PARSER> --diag 3`

Field	Value
Expression	`"(a|ab)(b|)"`, that is $(a + ab)(b + \varepsilon)$
Inputs	`"ab"` (match), `"b"` (no match)
Parsers	`deriv_std_rec`, `deriv_bc`, `pderiv_std`, `pderiv_bc`
Output file	`reports/report.txt`
Omitted	the `Timestamp` line
Wrapped	long lines
∂	`d`
·	`.`
$\oplus$	`(+)`
ε	`epsilon`
ϕ	`phi`
✓	`[OK]`
`Policy`	the implementation's internal label, not a claim about disambiguation policy

A.1 `deriv_std_rec`, match

```
$ cargo run -- parse "(a|ab)(b|)" "ab" \
    --parser deriv_std_rec --diag 3
================================
REGEX ENGINE DEBUG REPORT
================================
Mode:                 POSIX Standard (recursive parser)
Regex:                (a|ab)(b|)
Input:                "ab"
Result:               MATCH
Policy:               POSIX
================================
TIMING
================================
Parse time:           0.012ms
Expressions stored:   3 (r0 -> r1 -> r2)
================================
FORWARD PASS (Derivatives)
================================
```

```
r0 = Seq(Alt(Lit('a'), Seq(Lit('a'), Lit('b'))), Alt(Lit('b'), Eps))

Step 1: d(r0, 'a')
  rule: d(r1.r2, 'a')  [nullable(r1)=false]
       = d(r1, 'a').r2
r1 = Seq(Alt(Eps, Seq(Eps, Lit('b'))), Alt(Lit('b'), Eps))
  nullable(r1) = true  [OK] position 1 matched

Step 2: d(r1, 'b')
  rule: d(r1.r2, 'b')  [nullable(r1)=true]
       = d(r1, 'b').r2  +  d(r2, 'b')
r2 = Alt(Seq(Alt(Phi, Alt(Seq(Phi, Lit('b')), Eps)), Alt(Lit('b'),
  Eps)), Alt(Eps, Phi))
  nullable(r2) = true  [OK] position 2 matched

--------------------------------====
NULLABILITY CHECK
--------------------------------====
nullable(r2) = true -> proceed to backward pass
--------------------------------====
BACKWARD PASS (mkEps + inject)
--------------------------------====
mkEps(r2):
  mkEps(r1 + r2): nullable(r1)=true -> Left(mkEps(r1))
v2 = Left (Right Right (), Right ())

inject(r1, 'b', v2):
inject(Seq(Alt(Eps, Seq(Eps, Lit('b'))), Alt(Lit('b'), Eps)), 'b',
    Left (Right Right (), Right ()))
  -> (Right ((), b), Right ())
v1 = (Right ((), b), Right ())

inject(r0, 'a', v1):
inject(Seq(Alt(Lit('a'), Seq(Lit('a'), Lit('b'))), Alt(Lit('b'),
    Eps)), 'a', (Right ((), b), Right ()))
  -> (Right (a, b), Right ())
v0 = (Right (a, b), Right ())

--------------------------------====
RESULT
--------------------------------====
Parse tree:           (Right (a, b), Right ())
Flattened:            "ab"
--------------------------------====
END OF REPORT
--------------------------------====
```

A.2 `deriv_bc`, match

```
$ cargo run -- parse "(a|ab)(b|)" "ab" \
    --parser deriv_bc --diag 3
--------------------------------====
REGEX ENGINE DEBUG REPORT
--------------------------------====
Mode:                 POSIX Bitcoded
Regex:                (a|ab)(b|)
Input:                "ab"
Result:               MATCH
Policy:               POSIX
--------------------------------====
TIMING
--------------------------------====
Parse time:           0.015ms
Derivative steps:     2
--------------------------------====
INTERNALIZE
--------------------------------====
internalize((a|ab)(b|))
  rule: internalize(r1 . r2) = []@(internalize r1)(internalize r2)
ri0 = []@([]@([false]@'a' (+) [true]@([]@'a' . []@'b')) .
  []@([false]@'b' (+) [true]@epsilon))
--------------------------------====
FORWARD PASS (deriv_bc + simp)
--------------------------------====
Step 1 (char: 'a', position 1)
ri0 = []@([]@([false]@'a' (+) [true]@([]@'a' . []@'b')) .
    []@([false]@'b' (+) [true]@epsilon))
  deriv_bc(ri0, 'a'):
    rule: (bs@ri1.ri2) \ 'a'  [nullable_bc(ri1)=false]
         = bs@(ri1\'a'.ri2)
simp -> []@([]@([false]@epsilon (+) [true]@'b') . []@([false]@'b' (+)
    [true]@epsilon))
ri1 = []@([]@([false]@epsilon (+) [true]@'b') . []@([false]@'b' (+)
    [true]@epsilon))
  nullable_bc(ri1) = true  [OK]  bits accumulating

Step 2 (char: 'b', position 2)
ri1 = []@([]@([false]@epsilon (+) [true]@'b') . []@([false]@'b' (+)
    [true]@epsilon))
  deriv_bc(ri1, 'b'):
    rule: (bs@ri1.ri2) \ 'b'  [nullable_bc(ri1)=true]
         = bs@((ri1\'b'.ri2) (+) (fuse (mkEpsBC ri1) (ri2\'b')))
simp -> []@([true]@([false]@'b' (+) [true]@epsilon) (+) [false,
    false]@epsilon)
ri2 = []@([true]@([false]@'b' (+) [true]@epsilon) (+) [false,
```

```
    false]@epsilon)
  nullable_bc(ri2) = true  [OK]  bits accumulating

--------------------------------====
NULLABILITY CHECK
--------------------------------====
nullable_bc(ri2) = true -> proceed to mkEpsBC + decode
--------------------------------====
MKEEPSBC + DECODE
--------------------------------====
mkEpsBC(ri2) = [11]

decode((a|ab)(b|), [11]):
  = (Right (a, b), Right ())
--------------------------------====
RESULT
--------------------------------====
Bits:                  [11]
Parse tree:            (Right (a, b), Right ())
Flattened:             "ab"
--------------------------------====
END OF REPORT
--------------------------------====
```

A.3 pderiv_std, match

```
$ cargo run -- parse "(a|ab)(b|)" "ab" \
    --parser pderiv_std --diag 3
--------------------------------====
REGEX ENGINE DEBUG REPORT
--------------------------------====
Mode:                  Standard Partial-Derivative (pderiv_std)
Regex:                 (a|ab)(b|)
Input:                 "ab"
Result:                MATCH
Policy:                GREEDY
--------------------------------====
TIMING
--------------------------------====
Parse time:            0.017ms
Steps:                 2
--------------------------------====
INITIAL FRONTIER
--------------------------------====
[r0]  where r0 = Seq(Alt(Lit('a'), Seq(Lit('a'), Lit('b'))),
  Alt(Lit('b'), Eps))
--------------------------------====
FORWARD PASS (pderiv_tree per residual, frontier flattened)
```

```
--------------------------------====
Step 1 (char: 'a', position 1)
  frontier before (1 residual(s)):
[0] Seq(Alt(Lit('a'), Seq(Lit('a'), Lit('b'))), Alt(Lit('b'), Eps))
  pderiv_tree('a', .) applied to each:
    from [0]:
      rule: pderiv_tree('a', r1.r2)  [nullable(r1)=false]
= [(r1'.r2, inj: continue r1) | (r1',_) <- pderiv_tree(c,r1)]
-> Seq(Eps, Alt(Lit('b'), Eps))  (+ injection closure, not printable)
-> Seq(Seq(Eps, Lit('b')), Alt(Lit('b'), Eps))  (+ injection closure,
       not printable)
  frontier after (2 residual(s)):
    [0] Seq(Eps, Alt(Lit('b'), Eps))
    [1] Seq(Seq(Eps, Lit('b')), Alt(Lit('b'), Eps))
  a residual in this frontier is nullable  [OK]  parse tree available

Step 2 (char: 'b', position 2)
  frontier before (2 residual(s)):
    [0] Seq(Eps, Alt(Lit('b'), Eps))
    [1] Seq(Seq(Eps, Lit('b')), Alt(Lit('b'), Eps))
  pderiv_tree('b', .) applied to each:
    from [0]:
      rule: pderiv_tree('b', r1.r2)  [nullable(r1)=true]
= nub_tree $ [(r1'.r2, inj: continue r1) | (r1',_) <-
             pderiv_tree(c,r1)]
++ [(r2', inj: mk_eps(r1) paired with r2's result) | (r2',_) <-
                        pderiv_tree(c,r2)]
      -> Eps  (+ injection closure, not printable)
    from [1]:
      rule: pderiv_tree('b', r1.r2)  [nullable(r1)=false]
= [(r1'.r2, inj: continue r1) | (r1',_) <- pderiv_tree(c,r1)]
-> Seq(Eps, Alt(Lit('b'), Eps))  (+ injection closure, not printable)
  frontier after (2 residual(s)):
    [0] Eps
    [1] Seq(Eps, Alt(Lit('b'), Eps))
  a residual in this frontier is nullable  [OK]  parse tree available

--------------------------------====
SELECTION (first nullable residual, priority/list order)
--------------------------------====
any nullable residual in final frontier = true -> proceed to injection
  (already applied by select)
--------------------------------====
RESULT
--------------------------------====
Parse tree:           (Left a, Left b)
Flattened:            "ab"
--------------------------------====
```

```
END OF REPORT
--------------------------------====
```

A.4 pderiv_bc, match

```
$ cargo run -- parse "(a|ab)(b|)" "ab" \
    --parser pderiv_bc --diag 3
--------------------------------====
REGEX ENGINE DEBUG REPORT
--------------------------------====
Mode:                  Bitcoded Partial-Derivative (pDerivBC)
Regex:                 (a|ab)(b|)
Input:                 "ab"
Result:                MATCH
Policy:                GREEDY
--------------------------------====
TIMING
--------------------------------====
Parse time:            0.012ms
Steps:                 2
--------------------------------====
INITIAL FRONTIER
--------------------------------====
[(r0, [])]  where r0 = Seq(Alt(Lit('a'), Seq(Lit('a'), Lit('b'))),
  Alt(Lit('b'), Eps))
--------------------------------====
FORWARD PASS (pDerivBC per residual, frontier flattened)
--------------------------------====
Step 1 (char: 'a', position 1)
  frontier before (1 residual(s)):
[0] Seq(Alt(Lit('a'), Seq(Lit('a'), Lit('b'))), Alt(Lit('b'), Eps))
      bits-so-far = []
  pDerivBC('a', .) applied to each:
    from [0]:
      rule: pDerivBC('a', r1.r2)  [nullable(r1)=false]
           = [(smartC(r1',r2), bs) | (r1',bs) <- pDerivBC(c,r1)]
      -> (Alt(Lit('b'), Eps), [0])
      -> (Seq(Lit('b'), Alt(Lit('b'), Eps)), [1])
  frontier after (2 residual(s)):
    [0] Alt(Lit('b'), Eps)   bits = [0]
    [1] Seq(Lit('b'), Alt(Lit('b'), Eps))   bits = [1]
  a residual in this frontier is nullable  [OK]  bits accumulating

Step 2 (char: 'b', position 2)
  frontier before (2 residual(s)):
    [0] Alt(Lit('b'), Eps)   bits-so-far = [0]
    [1] Seq(Lit('b'), Alt(Lit('b'), Eps))   bits-so-far = [1]
  pDerivBC('b', .) applied to each:
```

```
    from [0]:
rule: pDerivBC('b', r1+r2) = tag(0, pDerivBC('b',r1)) ++ tag(1,
        pDerivBC('b',r2)), nub2
      -> (Eps, [0])
    from [1]:
      rule: pDerivBC('b', r1.r2)  [nullable(r1)=false]
           = [(smartC(r1',r2), bs) | (r1',bs) <- pDerivBC(c,r1)]
      -> (Alt(Lit('b'), Eps), [])
  frontier after (2 residual(s)):
    [0] Eps   bits = [00]
    [1] Alt(Lit('b'), Eps)   bits = [1]
  a residual in this frontier is nullable  [OK]  bits accumulating

--------------------------------====
SELECTION (first nullable residual, priority/list order)
--------------------------------====
any nullable residual in final frontier = true -> proceed to mkEpsBC +
  decode
--------------------------------====
MKEEPSBC + DECODE
--------------------------------====
selected residual's accumulated bits ++ mkEpsBC(residual) = [00]

decode((a|ab)(b|), [00]):
  = (Left a, Left b)
--------------------------------====
RESULT
--------------------------------====
Bits:                 [00]
Parse tree:           (Left a, Left b)
Flattened:            "ab"
--------------------------------====
END OF REPORT
--------------------------------====
```

A.5 deriv_std_rec, no match

```
$ cargo run -- parse "(a|ab)(b|)" "b" \
    --parser deriv_std_rec --diag 3
--------------------------------====
REGEX ENGINE DEBUG REPORT
--------------------------------====
Mode:                 POSIX Standard (recursive parser)
Regex:                (a|ab)(b|)
Input:                "b"
Result:               NO MATCH
Policy:               POSIX
--------------------------------====
```

```
TIMING
--------------------------------====
Parse time:           0.005ms
Expressions stored:   2 (r0 -> r1)
--------------------------------====
FORWARD PASS (Derivatives)
--------------------------------====
r0 = Seq(Alt(Lit('a'), Seq(Lit('a'), Lit('b'))), Alt(Lit('b'), Eps))

Step 1: d(r0, 'b')
  rule: d(r1.r2, 'b')  [nullable(r1)=false]
       = d(r1, 'b').r2
r1 = Seq(Alt(Phi, Seq(Phi, Lit('b'))), Alt(Lit('b'), Eps))
  nullable(r1) = false  [FAIL] position 1

--------------------------------====
NULLABILITY CHECK
--------------------------------====
nullable(r1) = false -> no parse tree exists for full input
--------------------------------====
PARTIAL RECOVERY
--------------------------------====
No prefix matched.
--------------------------------====
ERROR SUMMARY
--------------------------------====
Failure at position:  1 (1-indexed)
Found:                'b'
Expected:             'a'
  b
  ^
--------------------------------====
END OF REPORT
--------------------------------====
```

A.6 `deriv_bc`, no match

```
$ cargo run -- parse "(a|ab)(b|)" "b" \
    --parser deriv_bc --diag 3
--------------------------------====
REGEX ENGINE DEBUG REPORT
--------------------------------====
Mode:                 POSIX Bitcoded
Regex:                (a|ab)(b|)
Input:                "b"
Result:               NO MATCH
Policy:               POSIX
--------------------------------====
```

```
TIMING
--------------------------------====
Parse time:           0.008ms
Derivative steps:     1
--------------------------------====
INTERNALIZE
--------------------------------====
internalize((a|ab)(b|))
  rule: internalize(r1 . r2) = []@(internalize r1)(internalize r2)
ri0 = []@([]@([false]@'a' (+) [true]@([]@'a' . []@'b')) .
  []@([false]@'b' (+) [true]@epsilon))
--------------------------------====
FORWARD PASS (deriv_bc + simp)
--------------------------------====
Step 1 (char: 'b', position 1)
ri0 = []@([]@([false]@'a' (+) [true]@([]@'a' . []@'b')) .
    []@([false]@'b' (+) [true]@epsilon))
  deriv_bc(ri0, 'b'):
    rule: (bs@ri1.ri2) \ 'b'  [nullable_bc(ri1)=false]
         = bs@(ri1\'b'.ri2)
  simp -> phi
  ri1 = phi
  nullable_bc(ri1) = false  [FAIL]

--------------------------------====
NULLABILITY CHECK
--------------------------------====
nullable_bc(ri1) = false -> no full parse tree exists
--------------------------------====
PARTIAL RECOVERY
--------------------------------====
No prefix matched.
--------------------------------====
ERROR SUMMARY
--------------------------------====
Failure at position:  1 (1-indexed)
Found:                'b'
Expected:             'a'
  b
  ^
--------------------------------====
END OF REPORT
--------------------------------====
```

A.7 pderiv_std, no match

```
$ cargo run -- parse "(a|ab)(b|)" "b" \
    --parser pderiv_std --diag 3
```

```
--------------------------------====
REGEX ENGINE DEBUG REPORT
--------------------------------====
Mode:                  Standard Partial-Derivative (pderiv_std)
Regex:                 (a|ab)(b|)
Input:                 "b"
Result:                NO MATCH
Policy:                GREEDY
--------------------------------====
TIMING
--------------------------------====
Parse time:            0.007ms
Steps:                 1
--------------------------------====
INITIAL FRONTIER
--------------------------------====
[r0]  where r0 = Seq(Alt(Lit('a'), Seq(Lit('a'), Lit('b'))),
  Alt(Lit('b'), Eps))
--------------------------------====
FORWARD PASS (pderiv_tree per residual, frontier flattened)
--------------------------------====
Step 1 (char: 'b', position 1)
  frontier before (1 residual(s)):
[0] Seq(Alt(Lit('a'), Seq(Lit('a'), Lit('b'))), Alt(Lit('b'), Eps))
  pderiv_tree('b', .) applied to each:
    from [0]:
      rule: pderiv_tree('b', r1.r2)  [nullable(r1)=false]
= [(r1'.r2, inj: continue r1) | (r1',_) <- pderiv_tree(c,r1)]
      -> []
  frontier after (0 residual(s)):
  no nullable residual in this frontier  [FAIL]

--------------------------------====
SELECTION (first nullable residual, priority/list order)
--------------------------------====
any nullable residual in final frontier = false -> no full parse tree
  exists
--------------------------------====
PARTIAL RECOVERY
--------------------------------====
No prefix matched.
--------------------------------====
ERROR SUMMARY
--------------------------------====
Failure at position:  1 (1-indexed)
Found:                'b'
Expected:             'a'
  b
```

```
  ^
--------------------------------====
END OF REPORT
--------------------------------====
```

A.8 pderiv_bc, no match

```
$ cargo run -- parse "(a|ab)(b|)" "b" \
    --parser pderiv_bc --diag 3
--------------------------------====
REGEX ENGINE DEBUG REPORT
--------------------------------====
Mode:                  Bitcoded Partial-Derivative (pDerivBC)
Regex:                 (a|ab)(b|)
Input:                 "b"
Result:                NO MATCH
Policy:                GREEDY
--------------------------------====
TIMING
--------------------------------====
Parse time:            0.006ms
Steps:                 1
--------------------------------====
INITIAL FRONTIER
--------------------------------====
[(r0, [])]  where r0 = Seq(Alt(Lit('a'), Seq(Lit('a'), Lit('b'))),
  Alt(Lit('b'), Eps))
--------------------------------====
FORWARD PASS (pDerivBC per residual, frontier flattened)
--------------------------------====
Step 1 (char: 'b', position 1)
  frontier before (1 residual(s)):
[0] Seq(Alt(Lit('a'), Seq(Lit('a'), Lit('b'))), Alt(Lit('b'), Eps))
      bits-so-far = []
  pDerivBC('b', .) applied to each:
    from [0]:
      rule: pDerivBC('b', r1.r2)  [nullable(r1)=false]
           = [(smartC(r1',r2), bs) | (r1',bs) <- pDerivBC(c,r1)]
      -> []
  frontier after (0 residual(s)):
  no nullable residual in this frontier  [FAIL]

--------------------------------====
SELECTION (first nullable residual, priority/list order)
--------------------------------====
any nullable residual in final frontier = false -> no full parse tree
  exists
--------------------------------====
```

```
PARTIAL RECOVERY
--------------------------------====
No prefix matched.
--------------------------------====
ERROR SUMMARY
--------------------------------====
Failure at position:  1 (1-indexed)
Found:                'b'
Expected:             'a'
  b
  ^
--------------------------------====
END OF REPORT
--------------------------------====
```